Red-pen update of Tarmin_LN_7 against csf_final

Red text = our corrections and additions, collected on the red pages after each page (tags **[Rn]** on the original page show where each one goes).

Red line through text = proposed deletion; tag **[Sn]** gives the reason on the red page after it.

Basis: the csf_final artefact (lightning.spthy, lightning_aux.spthy, Makefile), all results reproduced on Sana's laptop with Tamarin 1.10: make verify 55/55, make general 24/24, make aux 21/21, make necessity (T3 counterexample verified, 28 steps). Co-authors' margin comments are left untouched.

Open questions for Sana and Simon

- [A] Should we claim a 'first'? None of the notes do. Positioning against [8], [9] (FC 2026) and Kiayias-Litos (CSF 2020) has to be settled first (also affects the p.1 sentence 'does not yet feature any complete ... formalisation').
- [A] Simon's question on p.2: keep or drop the two paragraphs 'Concretely, the two parties open a channel...' and 'Channels can be closed either cooperatively...'? If kept, they need the CSV/CLTV fix in the note at 'wait for a time lock'.
- [A] The model-vs-protocol sentences added in Sec. II (notes on pp. 2-4): keep them in the overview, or move them to Sec. IV as the (still valid) orange box on p.2 suggests?
- [B] Restriction labels: keep T1, T2b, T3 (T2a is now the lemma ForwardTimeGap), or renumber them T1-T3? Other sections cite T2b and T3.
- [B] OneResolutionPerHTLC is the only link between HTLC resolution and the channel layer. State it only as an assumption, or also give an informal argument (resolving an HTLC twice would need a revoked commitment, cf. No_Punishment_Without_Cheating)? Nothing in csf_final mechanizes that argument.
- [B] Necessity_OTF_Timeout_Order_Broken is in lightning.spthy (-D=NO_OTF) but is not run by make necessity and has no result: remove it from the shipped file, or keep it and say so?
- [B] Space: put the five restriction/lemma formulas from the page-6 notes in Sec. IV-A itself, or in Appendix A with only the informal statements in the body?
- [C] Fig. 4: keep the lemma's party names (Alice, Att1, Indiana, Att2, Bob), so the figure can be checked against the file, or map them to the running example of Fig. 3a?
- [C] Wormhole_Fee_Theft_Reachable is 29 lines verbatim: print it in Sec. V-E, or only in Appendix A with the prose description in V-E?
- [C] Present Intermediary_Never_Loses_Strong as Lemma 2' next to Lemma 2, or replace Lemma 2 by it?
- [C] Table II: keep row 0 (channel layer) and the general-mode and necessity rows, so it accounts for every lemma, or restrict it to routing groups 1-9?
- [C] Keep the pre-fix ('WRONG') Publish_Revoked_A listing, which is not in the artefact?
- [D] PTLC subsection: keep it as Sec. VI-E (inside bypass resistance, our proposal) or move it next to the wormhole in Sec. V-E?
- [D] Kiayias-Litos: please confirm the positioning sentence and add pages/DOI; and say what [9] verifies (we did not characterise it).
- [D] Maude 3.2 is reported unsupported by Tamarin 1.10: re-run with a supported Maude (e.g. 3.2.1 or 3.4) before submission, or report as is (current text)?
- [D] Skipped_Requires_Compromise is quoted verbatim both in Sec. V-E (p. 11 markup) and in the new Sec. VI-A: keep it in one place only?
- [D] Hop3..Hop6 statements: only by name in Sec. VI (current text) or verbatim in Appendix A?
- [D] Artifact URL and anonymisation for review.
- [E] SJ's TODO on Appendix A: keep the appendix (now re-anchored to csf_final, all listings verbatim) or drop it before submission?
- [E] Renumber the catalogue (Lemmas 9, 10, 12, 17–24, 27, 29 removed) and merge the additions into Sections A–H, or keep them as a block at the end of the appendix?
- [E] Keep lightning.spthy line numbers in the listings, or drop line numbers altogether?

Formal Verification of the Lightning Network[R1] in TAMARIN

***Abstract*—**
TODO[R2]

I. INTRODUCTION

Bitcoin [1] is the most important cryptocurrency in terms of market capitalisation. It is, however, very lacking in many usability and scalability aspects: it is costly, has low bandwidth (only a few dozen transactions per second), and high latency. Many protocols strive to alleviate those shortcomings.

a) Payment Channel Networks: Notably, payment channels enable a pair of users to quickly and cheaply perform arbitrarily many payments by relying on the blockchain only to resolve disputes. Both agents pool together funds and then iteratively update their balance. The scripts involved and the protocol itself ensure that no honest agent loses money, by relying on a revocation mechanism: any party closing a channel using an outdated favourable state can be punished, consequently losing all funds invested in the channel. However, payment channels are constrained to pairs of users. Two unconnected users still need to establish a channel in order to perform a payment; this is often not cost effective. Thus arose the concept of payment channel networks: one transitively propagates funds across payment channels to perform a payment between two users that are part of the same network but not connected directly. The most popular of such protocols is perhaps the Lightning Network [2], with about 170M$ locked on the network [3].

Due to the sheer complexity of its specification [4], it suffers from many attacks [5], [6], [7]. And despite promising ongoing efforts [8], [9] it does not yet feature any complete and unbounded verified formalisation. This paper attempts to alleviate that issue.[R3]

b) Protocol Verification: Designing and analysing cryptographic protocols is a notoriously difficult and error-prone task, as already illustrated by the various attacks and tentative fixes on the Lightning Network. Formal methods provide hope for a more robust verification of protocols, and automation.

Protocol verification is further split between two main classes of models: the computational model—inherited from cryptography, where the attackers and the participants are polynomial-time algorithms and results are typically probabilistic—and the symbolic (or *Dolev-Yao*) model [10]—where the attacker has infinite computational power but is limited to a set term algebra. The symbolic model notably abstracts away from cryptography and focuses on the logic of a protocol, where much of the attacker activity is typically found.

Although some tools exist to help the cryptographer in the computational model, such as CRYPOTVERIF [11], EASYCRYPT [12], SQUIRREL [13] or CRYPTOVAMPIRE [14], they all are brittle, hard to use and/or very scoped. In contrast, symbolic verification enjoys very successful automated verifiers in the form of PROVERIF [15] and TAMARIN [16]. In particular, TAMARIN is a multiset rewriting-based automated protocol verifier with a large pedigree of widely used protocols verified, such as WIREGUARD [17], TLS 1.3 [18] among many others.

Simon: Is this a good intro to TAMARIN?

c) Contributions:[R4]

~~AI: Pointers (also applies to the abstract): (1) Lead with claims a reviewer can check, not model features. The current bullets say "we built a Tamarin file with properties X, Y"; say instead WHAT IS PROVEN, for WHICH runs, against WHICH adversary. Note that "unbounded" now covers both channels and paths in ONE theory – one file, one adversary, one shared topology – but be precise about the coupling: the layers share the theory, the adversary and the channel topology (routing runs only on channels the channel layer opened), while the contract between them (each HTLC resolves at most once) is a NAMED assumption justified by channel-layer lemmas, not a mechanized composition. Claim exactly that; do not claim a composition theorem. (2) Suggested prongs: (a) coverage: first symbolic model of LN in a single theory covering the full channel lifecycle (open/update/revoke/close/punish, unbounded updates) AND unbounded path HTLC routing with per-hop amounts and a block clock, under Dolev-Yao + long-term-key compromise; (b) end-to-end results for arbitrary path length: payment atomicity, intermediary safety under liveness, end-to-end value conservation, punishment-only-on-cheating; (c) the wormhole modeled explicitly AND quantitatively: steals exactly the path fees, skipped intermediaries refunded – this is a characterization, not just a witness, worth its own bullet; (d) methodology: every abstraction is a named restriction with provenance (clock well-formedness / discharged by the other layer / liveness), plus necessity results (drop the liveness restriction and intermediary safety fails with a counterexample). (3) The commented-out draft right below is closer to this than the visible bullets – mine it. (4) Position against Grundmann (TLA+, bounded search) and the verified-LN work: unbounded path length + full channel layer in one theory is the delta.**[S1]

Simon: There seems to be 50000 models of WireGuard.

~~We provide a novel modelisation of the Lightning Network~~[S2]

Red updates for page 1

[S1] struck *"AI: Pointers (also applies to the ... one theory is the delta."*

Orange AI box. Its valid advice (state what is proven, for which runs and against which adversary; thin coupling; no composition theorem) is implemented by the new contributions list (note at 'c) Contributions:'). Prongs (a)-(c) describe a different model: csf_final has no block clock in the main model and no end-to-end value-conservation or 'steals exactly the path fees' lemma.

[S2] struck *"We provide a novel modelisation ofthe Lightning Network"*

Old contributions sentence (continues on p.2). Replaced by the new contributions list (note at 'c) Contributions:').

[R1] *at "Formal Verification of the Lightning Network"*

TODO (legend; remove before submission): **red** = updates that re-anchor the paper to the verified csf_final artefact (`lightning.spthy` + `lightning_aux.spthy`, TAMARIN 1.10), reproduced on Sana's laptop. **Red strikethrough** = proposed deletions. All changes are proposals and still need to be agreed with Simon.

[R2] *at "Abstract— TODO"*

Abstract—The Lightning Network routes payments over chains of payment channels using hash time-locked contracts (HTLCs). We present a TAMARIN model that captures, in one theory, the channel layer (unboundedly many updates, revocation, punishment) and generic multi-hop routing over routes of any length, against a Dolev-Yao adversary who can also compromise long-term keys. We prove that only parties that cheated or whose key was compromised are punished, that settlements are authenticated by the receiver's invoice, and that preimages, which our model never sends on the network, stay secret. Under explicit liveness assumptions, imposed as TAMARIN restrictions that remove every run violating them (for instance, every run in which an uncompromised intermediary whose outgoing HTLC was redeemed does not claim its incoming HTLC in time), an intermediary whose key is not compromised never has its outgoing HTLC redeemed while its incoming HTLC is refunded, and payment atomicity holds in runs without key compromise. The assumption that intermediaries claim their incoming HTLC is necessary: without it, TAMARIN finds a trace without key compromise in which an intermediary that never claims its incoming HTLC loses funds. We give the wormhole attack as a TAMARIN-checked trace: colluding intermediaries bypass an honest one, which is refunded but earns no fee. Conversely, we prove for routes of any length that a hop of a settled payment whose hash is offered once can be bypassed only if some long-term key is compromised. In a fixed-route model with a symbolic stand-in for point time-locked contracts (PTLCs), per-hop locks prevent the wormhole around an honest intermediary when the sender is honest, whereas plain HTLCs admit it. All results are machine-checked with TAMARIN 1.10 and reproduced with one command.

[R3] *at "This paper attempts to alleviate that issue."*

TODO: tone down "complete": our model abstracts the HTLC outputs of commitment transactions (Sec. IV). Suggested: "... it still lacks a machine-checked model that covers both the channel layer and multi-hop routing for unboundedly many channels, updates and payments and for routes of any length." This sentence must be consistent with the positioning against [8], [9] and Kiayias–Litos (CSF 2020) in the related work (see open question on a 'first' claim).

[R4] *at "c) Contributions:"*

We make the following contributions. All results are checked with TAMARIN 1.10. The all-traces statements hold for every trace of the model that satisfies its restrictions (Sec. IV-A), that is, for any number of parties, channels, channel updates and payments.

- **One theory for both layers (Sec. IV).** A single TAMARIN theory (`lightning.spthy`) with a single adversary covers two layers. The first is the channel layer: opening, unboundedly many updates, revocation, cooperative and unilateral closing, and punishment. The second is multi-hop HTLC routing, where one generic rule (`Forward_HTLC`) handles every hop, so routes of any length over the channels opened by the channel layer are included. The layers share the channel topology (`!ChannelConnect`) and a per-channel slot (`Mutex`: one in-flight HTLC per channel at a time). The contract between them, that each HTLC is resolved at most once, is the named restriction `OneResolutionPerHTLC`, which we justify informally. We do not claim a composition theorem.
- **Adversary (Sec. IV-A).** A Dolev-Yao network adversary that can, in addition, compromise the long-term key of any party at any time (`Compromise_Ltk`). Two compromised parties on the route of the same payment can relay the preimage around the parties between them (`AttackerSkip`).
- **Safety (Sec. V).** We prove 32 all-traces lemmas. Among them: a party is punished only if it published a revoked state or its key was compromised (`No_Punishment_Without_Cheating`); a sender settles only against an invoice created by the receiver, unless the receiver's key was compromised (`Invoice_Authenticates_Settlement`); the adversary never learns a payment preimage (`Preimage_Never_Known`; in our model the preimage is never sent on the network); and no HTLC is both redeemed and refunded (`OneOutcomePerHTLC`). Intermediary safety and payment atomicity rely on three named liveness restrictions (T1, T2b, T3; Sec. IV-A). Being restrictions, they remove

every trace that violates them, including traces in which an uncompromised forwarder could not have acted in time; these results say nothing about such traces. An intermediary whose outgoing HTLC is redeemed has its incoming HTLC refunded only if its key was compromised before that incoming HTLC timed out (`Intermediary_Never_Loses_Strong`; a weaker variant is `Intermediary_Never_Loses_Under_Liveness`). In runs without key compromise, if two consecutive forwarders relay a payment and the outgoing HTLC of the second is redeemed, the incoming HTLC of the first is never refunded (`Payment_Atomicity_Under_Liveness`). Ten exists-trace lemmas (nine found automatically, one replayed from a saved proof) show that channels open, update and close, that multi-hop payments complete, and that the attacks discussed below are reachable.

- **The wormhole as a trace and as a theorem (Secs. V-E and VI).** We give a five-party trace (Alice, Att1, Indiana, Att2, Bob) in which the sender settles and the receiver is paid, while the honest intermediary Indiana is refunded on its outgoing HTLC and earns no fee (`Wormhole_Fee_Theft_Reachable`; TAMARIN checks it by replaying a saved 116-step proof). Conversely, we prove that if the sender settles a payment, a forwarding hop on its route is refunded on its outgoing HTLC only if the long-term key of some party is compromised. We prove this for each of the first six hops of a route of any length (`First_Hop_Skip_Requires_Compromise`, `Second_Hop_Skip_Requires_Compromise`, `Hop3_Skip_Requires_Compromise` to `Hop6_Skip_Requires_Compromise`), and for every hop of a route of any length, assuming the payment hash is offered only once (`Any_Hop_Skip_Requires_Compromise`). The latter is proved with the three liveness restrictions removed; since removing restrictions only adds traces, it also holds in the full model. Neither result says which party is compromised, or when. The fixed-position theorems are proved in the full model, i.e., under the liveness restrictions.
- **Necessity of the liveness assumptions (Sec. V-D).** We remove the liveness restrictions one at a time. Without `IntermediaryMustClaim` (T3), TAMARIN finds a trace without key compromise in which an intermediary's outgoing HTLC is redeemed while its incoming HTLC is refunded (`Necessity_IMC_Honest_Intermediary_Loses`). Hence T3 is necessary for intermediary safety. Without `RedeemBeforeTimeout` (T1), all five liveness lemmas we check remain provable; without `OutgoingTimesOutFirst` (T2b), both intermediary-safety lemmas remain provable. The CLTV-delta ordering of timeouts (`ForwardTimeGap`) is not an assumption but a proved lemma, implied by T2b.
- **Countermeasure.** An auxiliary model (`lightning_aux.spthy`) uses the fixed route S→H1→H2→H3→R. In it, per-hop locks are derived along the route with per-hop secret tweaks, a symbolic stand-in for point time-locked contracts (PTLCs) [6]. With these locks, the sender's HTLC is never redeemed while H2's outgoing HTLC is refunded, unless H2 or the sender is compromised (`No_Wormhole_Around_Honest_Hop`). With a common payment hash, the same attack is reachable (`Wormhole_Around_Honest_Hop`).
- **Reproducibility.** A single command (`make all`) reproduces all results: the 55 lemmas of the main model, the 24-lemma run of the any-length theorem, the 21 lemmas of the auxiliary models, and the necessity runs.

The TLA+ model checking of [8] explores finite instances. In contrast, our all-traces results hold for unboundedly many channels, updates and payments, and for routes of any length.

~~in TAMARIN featuring~~

- ~~Unbounded channels;~~
- ~~an idealized routing layer capturing fees and timing constrains over arbitrary paths.~~[S3]

II. PROTOCOL OVERVIEW AND THREAT MODEL

The Lightning Network can be still into two main parts: the *Payements Channels* and the *Routing*. The first one enables to transfer funds quickly en efficiently between two participants with reduced interaction with a blockchain. However, the setup is comparatively expensive.

The routing protocol organises and secures transfers going beyond a single channel, scaling a mean of localized pairwise payments to a whole network.

Each part carries its own cryptographic mechanism. In the payment channel, revocation keys protect against *outdated states*. In the routing protocol, a payment hash and a deadline control the *transfer*. They meet only in the commitment transaction, where an in-flight payment is an extra output.[R5] It is subject to the same revocation as the rest of the channel. ~~We included it in our model as well. The model has a channel layer and a routing layer. The wormhole attack of sec. V-E touches only the payment hash. So the channel layer properties all remain intact.~~[S4]

AI: This paragraph is the seed of the whole paper story – make it the thesis, then pay it off in the methodology section. In the PROTOCOL, the two layers interact at exactly one place: the HTLC output of a commitment transaction, subject to the same revocation as the rest of the state. Hence everything the routing layer needs from the channel layer reduces to 'each HTLC is resolved at most once, and channel balances hold'. CRITICAL for honesty: the MODEL does not literally fold HTLCs into channel states – in Lightning.spthy the two layers couple only through the channel topology facts (!ChannelConnect, minted when the channel opens and consumed by the routing rules), the Mutex slot, and the named assumption OneResolutionPerHTLC. So the sentence below 'It is subject to the same revocation as the rest of the channel. We included it in our model as well.' overclaims: the model captures this coupling as the restriction OneResolutionPerHTLC, not as HTLC-carrying channel states. Reword to something like: in our model this meeting point becomes an explicit contract (the routing layer assumes each HTLC resolves at most once; the channel-layer lemmas justify it). Also: the model claims (about the wormhole etc.) belong in the model section; keep this overview purely about the protocol.

This section gives an overview of these two parts.

A. Payment Channel

The Payment Channel protocol fundamentally enables two agents to *commit* and *revoke* a shared blockchain-related "state". This state typically tracks the balance of some multi-sig [19] *common fund*, effectively transferring coins between its two participants. The state is represented by valid transactions spending that common fund, and a payment channel ensures these transactions are valid and that no agent can publish an old, favourable state. The latter property comes from careful signing (restricting transactions to agreed-upon shapes) and *punishing*: a participant may unconditionally retrieve *all* funds locked in a channel if their counterparty puts an old state on chain.

Simon: Do we need the 2 next paragraph. They seem to duplicate content from the 'subsubsection'.

Concretely, the two parties open a channel. They use an `on_chain` funding transaction that pays into a 2-of-2 output. After that, they make off-chain payments. They send signed commitment transactions back and forth. They can update the state by exchanging new commitments. When the state is being updated, parties exchange revocation secrets for the previous commitment and revoke the old state. If one party publishes an old commitment after revocation, the other party may claim all funds. In between state updates, any commitment that has not been revoked yet is considered a valid unilateral close. If it's being published by either party, there will be no penalty. The punishment will be triggered only when a revoked secret commitment is published.

Channels can be closed either cooperatively, in which both parties agree on the final balance and settle in a single chain transaction, or unilaterally. In a unilateral closing, one party broadcasts an unrevoked commitment. Both parties have to wait for a time lock[R6] (CLTV; Common abbreviation for the `CheckLockTimeVerify` opcode) to expire; this gives the other party time to detect whether the published secret is revoked or not. If a revoked state is published, then the other party can unilaterally punish the cheater and claim the entirety of the channel funds protected by a relative timelock (CSV; Common abbreviation for the `CheckSequenceVerify` opcode).

Simon: If I understood correctly it should be CLTV here as this is the timeout for the in flight HTLC which is absolute

In the context of the Lightning Network, the state also tracks in-flight payments which will propagate further than this channel. These are carried by *hash time-locked contracts* (HTLCs). We discuss HTLCs fully in sec. II-B. The protocol relies on two things: careful handling of revocation keys and careful handling of transaction signatures. Fig. 1 shows the channel lifecycle.

Simon: if we keep this paragraph, we need to remove the "CSV..." from below

1) The Transactions: Although we abstract it away in our model, one should keep in mind the shape of the transaction in the Lightning Network. Fig. 2 gives the shape of the commitment transaction held by Alice in an Alice-Bob Channel. This transaction notably enables unilateral closing.[R7] ~~It connects the parts in our rules to the real protocol.~~[S5]

Simon: What do you mean by that?

Outputs 3 and 4 enables the simple redistribution of the common fund between Alice and Bob, while Outputs 1 and 2 lock the fund of in-flight payment specific to the Lightning Network. ~~The model represents only the in-flight outputs and revocation path on Output 4~~[S6]

The figure also illustrates the punishment mechanism: unlike Bob's Output 3, Alice needs to wait for a relative timelock (CSV; Common abbreviation for the `CheckSequenceVerify` opcode) before redeeming Output 4. That gives Bob the opportunity to reddeem Output 4 using Alice's *revocation key* (rk_A). This revocation key is initially hidden from Bob.

Red updates for page 2

[S3] struck *"in TAMARIN featuring Unbounded channels; an ... timing constrains over arbitrary paths."*

Old contributions bullets (continued from p.1). Replaced by the new contributions list on p.1.

[S4] struck *"We included it in our model ... channel-layer properties all remain intact."*

Overclaim: in csf_final, commitment transactions carry no HTLC outputs; the layers couple only through !ChannelConnect, Mutex(ptr) and OneResolutionPerHTLC. Replaced by the note at 'extra output.'

[S5] struck *"It connects the parts in ourrules to the real protocol."*

Unclear (Simon's margin question). Replaced by the rule-to-output mapping in the note at 'enables unilateral closing.'

[S6] struck *"The model represents only the in-flight ... revocation path on Output 4"*

Wrong for csf_final: the modelled commitment has no in-flight Outputs 1/2, while Outputs 3/4 and the revocation path are modelled. Replaced by the same mapping note.

[R5] *at "where an in-flight payment is an extra output."*

Our model (Sec. IV) does not place HTLCs inside commitment transactions. Instead, this meeting point becomes an explicit interface: routing runs only on channels opened by the channel layer, holds at most one in-flight HTLC per channel at a time, and assumes that each HTLC is resolved at most once. The wormhole attack (Sec. V-E) exploits only the payment hash shared along a route and involves no channel-layer rule. Since the channel-layer lemmas are proved in the same theory, they also hold in the presence of the wormhole.

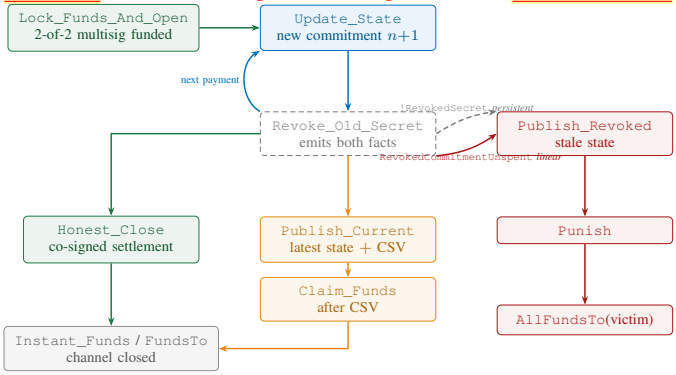
[R6] *at "wait for a time lock"*

TODO (protocol accuracy; answers Simon's margin comment): in a unilateral close only the publisher waits, and its delay is the relative timelock CSV (`to_self_delay`) on its own output (Output 4). The counterparty's Output 3 is spendable immediately, as Sec. II-A2d says. CLTV is the absolute timeout of the in-flight HTLC outputs (Outputs 1 and 2). Suggested text: "In a unilateral close, one party broadcasts its latest commitment. Its own output is locked by a relative timelock (CSV), which gives the counterparty time to check whether the published state was revoked and, if so, to claim the entire channel balance through the revocation path." If this paragraph is kept, the second CSV gloss below becomes redundant.

[R7] *at "This transaction notably enables unilateral closing."*

Fig. 2 also fixes the vocabulary of our channel-layer rules. A commitment is modelled as the tuple `<'commit', ptr, A, B, n, h(s)>` together with its signature, where `h(s)` commits to the revocation secret `s` of state `n`. Publishing Alice's current commitment (`Publish_Current_A`) creates two outputs. Bob's Output 3 is spendable immediately (`OnChainInstant`, spent by `Instant_Funds_B`). Alice's Output 4 is delayed (`OnChainDelayed`, spent by `Claim_Funds_A`, which marks the end of the CSV delay with the action `CSVPassed`; the main model has no clock). Publishing a revoked commitment (`Publish_Revoked_A`) instead opens the revocation path, through which Bob takes all funds (`Punish_A`, `AllFundsTo`). The in-flight Outputs 1 and 2 are not part of the modelled commitment; HTLCs are handled by the routing layer (Sec. II-B, Sec. IV).

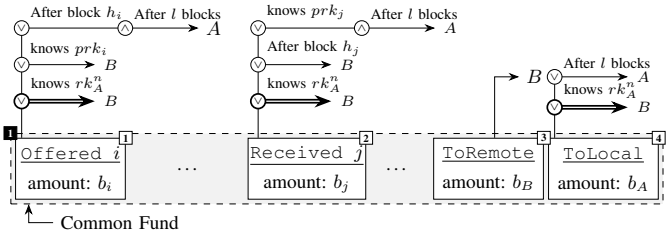
[TODO] Fix margin legend [END TODO]



Green: open and cooperative close.
Blue: update loop.
Orange: unilateral close with its CSV delay.
Red: revoked publish and punishment.

`Revoke_Old_Secret` emits *two* facts, and both gate punishment: the persistent `!RevokedSecret` (dashed, checked but never consumed) and the linear `RevokedCommitmentUnspent` (solid, consumed once). Gating on the persistent fact alone is the defect of Section V-B0a. [R8] Cooperative and unilateral closes end in settlement; punishment terminates in `AllFundsTo`. Grace-period and pending-publish paths, and the counterparty's instant output on a unilateral close, are elided.

Fig. 1: Channel lifecycle.



where the CSV is set to l, and the timeout (CLTV) of the kth HTLC is set to height h_k for the `Offered` and `Received` outputs; similarly prk_k is the corresponding pre-image of the payment hash; ⟹ denotes the punishment path; and 1 refers to an output. [NOTE] I removed the two staged thing [END NOTE]

Fig. 2: Model of the Transaction held by A [R9]

Alice only releases as a proof she will not use a given state. This mechanism enables declaring states as "old".

Fig. 2 also describes the local mechanism for in-flight payments. Output 1 (resp. 2) capture in-flight funds flowing towards Bob (resp. Alice). Besides the punishment mentioned above, these outouts are further governed by *payement hashes* and a timeout. To *fullfill* an in-flight transfer, the receiver (Bob in Output 1's case) needs the pre-image of the payment hash associated with that transfer. To prevent funds from being locked up, the output effectively reverts to a regular output once a block height is reached (e.g., h_i in 1's case). Real-world Lightning Network uses a two staged system to separate the CLTV and CSV. Our model collapses this complexity into what is shown in Fig. 2.

Simon: I commented out the rest as I believe it will better fit in the part where the model is explained directly

2) The Protocol: The lifecycle of a channel has five phases: opening, update, revocation, closing, and punishment, which correspond, left to right, to Fig. 1.

Simon: align the paragraph names to the figure

a) Opening: The opening phase aims to establish a first committed state and the common fund; in that order to prevent griefing attacks. After an unmodeled negotiation for the inputs of the common fund, Alice and Bob each sample a random revocation key (rk^0_A and rk^0_B respectively) and exchange their hashes. Relying on SegWit [20], both parties can compute the common fund TXID and exchange the various signatures needed to spend it: it will be a multisig [19] transaction locked by both party. For instance, Bob signs the various outputs of Fig. 2 (modeled in TAMARIN as a single signature of the whole transaction). [R11] Then, the witnessses for the common fund are exchanged and it is published. Once on chain, the chanel is ready.

Sana: Corrected

Simon: meh explained

b) Update: Updating involve setting up a new state then revoking the old one. The party first commits to a new state via the same mechanism as in the opening phase: sampling revocation keys (rk^n_A and rk^n_B respectively for the nth state); exchanging their hashes; exchanging the signatures. Only then do they revoke the old state (here the $n-1^{\text{th}}$ state) by disclosing to the other party their revocation keys (e.g., A sends rk^{n-1}_A to B).

If the steps were reversed, the party would have nothing to publish. The old state would be revoked, and the new one would not be signed. It could not close the channel if the counterparty stopped responding.

Sana: I divided it into two paras; it can be in one, idk

c) Revocation: A state is retired by handing its revocation key to the counterparty: Alice revokes state n by giving rk^n_A to Bob. The key is useless unless Alice later publishes that state, at which point it lets Bob take the whole channel balance. Revoking is therefore a proof of honesty, as publishing that old state could then result in punishment. Thus, updating a state is done via committing to a new state by both parties, then revoking the previous state.

d) Closing: A channel can close in two ways. In the *cooperative* close, the parties sign one transaction together. This transaction sends the final balance directly to their wallets. Only one on-chain transaction is needed. There are no scripts and no delay. This is the cheapest outcome for both. In unilateral closing, either party can close. One party publishes its current commitment alone. The publisher's own output (out. 4) is restricted by the CSV delay and by the revocation path. The party cannot spend its own output until the CSV delay ends. During that window, the counterparty can take it outright if the published state was revoked. The counterparty's (out. 3) is spendable immediately. Therefore, unilateral Closing disadvantages the publisher and favours the counterparty. This creates a punishment window, which is why an honest party closes unilaterally only when its counterparty has stopped responding.

e) Punishment: If an old state is published on the blockchain by one party (i.e., a state whose revocation key is known to counter-party), the counterparty can defend themselves using the punishment paths shown in Fig. 2. Punishment works as [R10]

Red updates for page 3

[R8] *at "the defect of Section V-B0a"*

TODO: broken cross-reference "V-B0a" (a paragraph label). Point it to the paragraph of Sec. V that explains why `Publish_Revoked_A/B` must consume the linear `RevokedCommitmentUnspent`.

[R9] *at "Model of the Transaction held by A"*

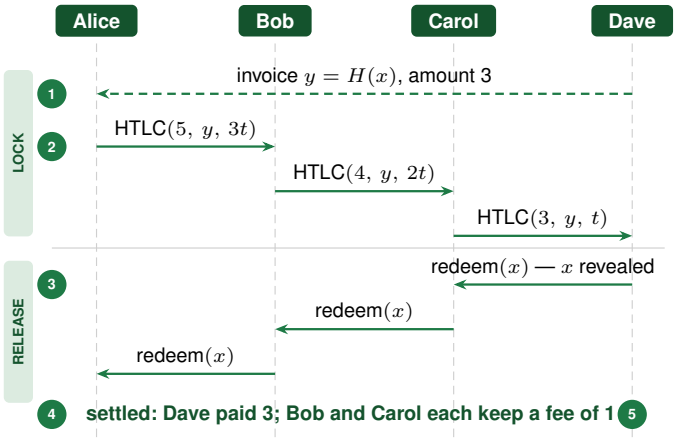
TODO: rename to "Fig. 2: Commitment transaction held by A". The figure shows the protocol transaction; our model keeps only Outputs 3 and 4 and the revocation path (see Sec. II-A1), not the in-flight Outputs 1 and 2.

[R10] *at "Our model collapses this complexity into"*

If this paragraph is kept, correct it: our model does not represent Outputs 1 and 2 at all. HTLCs are not outputs of the modelled commitment; they are resolved in the routing layer. The single on-chain spend of an HTLC output is replaced by the restriction `OneResolutionPerHTLC`: each HTLC is redeemed or refunded at most once (Sec. IV).

[R11] *at "as a single signature of the whole transaction"*

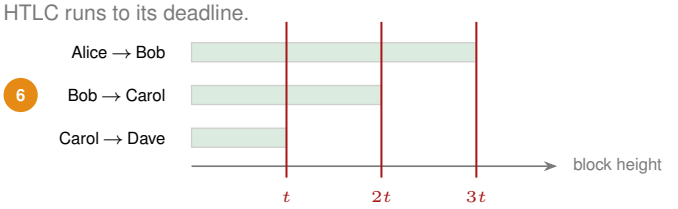
In our model, each party signs its own commitment (e.g., `sigA0 = sign(commitA0, ltkA)` in `Create_State_A`). Both signatures are verified (`Both_States_Committed`) before the funds are locked (`Lock_Funds_And_Open`). The counter-signature of the real protocol is abstracted into this check.



Amounts decrease $5 \to 4 \to 3$; each one-unit drop is that intermediary's fee. Deadlines decrease $3t > 2t > t$, so every forwarder can redeem its incoming HTLC before its outgoing one expires. Circled numbers index the lemma groups of Table II.[R12]

(a) Successful three-hop payment

HTLCs are locked as in Fig. 3a; Dave then never releases x, so every HTLC runs to its deadline.



Each window strictly contains the next, so a forwarder always redeems its incoming HTLC before its outgoing one expires.

Capital is locked for the full window and the payment fails, but the nested deadlines $t < 2t < 3t$ mean every honest party is refunded: this is a denial of service, not a loss of funds (`Loss_Requires_Inaction`).[R13]

(b) Griefing

Fig. 3: In-flight Payements

an *active* defense: the defender must act (i.e., notice the fraud; send the punishement transaction; and wait for it to be accepted) within the window given by the CSV. Failure to react makes the protocol trivially insecure [21].

This is why our model must encode *liveness*: no agent can be forcebly put offline. We do so with restriction T3sec. IV-A and show in sec. V-D. This assumption is load-bearing. Removing it falsifies `Intermediary_Never_Loses_Under_Liveness`.[R14]

Simon: Since we are more precise later, maybe we can move that in the "Liveness" section?

B. Multi-hop HTLC Routing

When the sender and recipients do *not* share a direct channel, the payment is routed through intermediaries along the path as in Fig. 3: it is a *multi-hop* payment. Because intermediaries are untrusted, each hop relay in an HTLC is conditioned on a single shared secret, the *payment hash*. As a result, the payment either completes along the entire path or fails. Furthermore, to provide incentive, the intermediaries are rewarded with a routing fee.[R15]

Example 1 (Path)**.** *We will use the running example where S sends money to R with the following path:*

$$S \to F_1 \to \cdots \to F_n \to R \tag{1}$$

a) Locking: For this, the receiver (R in Ex. 1) generates a preimage x and sends an invoice to the sender (S) containing the hash $y = H(x)$ and the requested amount v, keeping x strictly secret; the sender adds an HTLC to its channel with F_1 locking funds v_0 on that channel conditional on y before the deadline d_0, and after d_0 the payer may reclaim it. The condition to unlock funds is to provide the preimage x' with $H(x') = y$ before the deadline (as in Fig. 2); This moves forward with each forwarder locking funds with the routing fee $v_{k-1} - v_k$ given to forwarder F_k. Each HTLC uses the same hash but with a decreasing deadline and funds.

Simon: todo better unify the two section. This is repeating what is above; but I believe it might make more sense to have it here.

$$v_0 > v_1 > \cdots > v_n = v \tag{2}$$

$$d_0 > d_1 > \cdots > d_n \tag{3}$$

At this steps fund are still locked by the payment hash; they have not moved yet. Each HTLC becomes an additional conditional output[R16] in the two parties' current commitment transaction, as explained in Sec. II-A. The intermediaries F_i can safely lock funds, since an incoming HTLC has a larger amount and has a later deadline. If the outgoing HTLC is claimed, F_i learns x and still has time to unlock the incoming funds, keeping the difference as its fee.

b) Release round: In the redeem round, the receiver reveals x, the preimage, to the last forwarder to claim the final HTLC. Since every HTLC has the same preimage, each intermediary claims on its incoming channel using the same x, each learning x when the next hop claims from it. To claim v_{n-1} from F_{n-1}, F_n provides the secret x for its incoming HTLC, and it keeps the $v_{n-1} - v_n$ as a fee. The preimage thus propagates backward hop by hop until the sender HTLC is settled. If any party in between fails to forward it and the timelock expires, the funds return to the party that locked them.[R17] ~~If honest forwarders submit their claims on time, everyone gets paid; otherwise, no one does.~~[S7]

Remark 1 (Claiming and Liveness)**.** *Claiming can happen by mutual negotiation or by unilateral closing. Notably, unilateral closing also publish x as it is leaked by the closing transactions redeeming the HTLC.*[R18]

Remark 2 (Decreasing deadlines and Liveness)**.** *Consider F_i holding an incoming HTLC at d_{i-1} and an outgoing HTLC at d_i. If F_{i+1} unilaterally claims the outgoing HTLC at d_i, F_i learns x as late as d_i and is left the window (d_i, d_{i-1}) to settle its incoming HTLC. The gap between d_{i-1} and d_i is the CLTV delta, which is precisely the time reserved for F_i to recover the funds.*

This means that, in case of unilateral close, forwarders need liveness in order to claim their incoming HTLC within their claim window [21].[R19]

~~[TODO] re-add the wormhole attack later. Likely when we talk about the model [END TODO]~~[S8]

Sana: I commented it out...but I think if we refer it to that section in model two lines are ok...if you think it's ok add it

Red updates for page 4

[S7] struck *"If honest forwarders submit their claims ... paid; otherwise, no one does."*

'Otherwise, no one does' is wrong: an intermediary that fails to claim loses its incoming HTLC while the parties downstream are paid (Necessity_IMC_Honest_Intermediary_Loses). Replaced by the note at 'locked them.'

[S8] struck *"[TODO]re-add the wormhole attack later. Likely ... about the model [END TODO]"*

Co-author TODO resolved by the two-sentence wormhole pointer in the note at 'claim window [21].', as Sana's margin comment suggests.

[R12] *at "index the lemma groups of Table II."*

TODO: Table II is being rebuilt for csf_final; check that the circled numbers 1–6 still index its lemma groups. Fig. 3 depicts the protocol. In the model, the invoice carries no amount and deadlines are not numbers (see the note in Sec. II-B).

[R13] *at "not a loss of funds"*

TODO: delete "(Loss_Requires_Inaction)" from this caption. In csf_final that lemma reads:

```
lemma Loss_Requires_Inaction:
    "All P Prev Next Up ptrIn ptrOut y #i #j #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #i
      & Redeemed(ptrOut, y) @ #j
      & Refunded(Up, ptrIn, y) @ #k
     ==> not (Ex #l. Redeemed(ptrIn, y) @ #l)"
```

It states that if a forwarder's outgoing HTLC is redeemed and its incoming HTLC is refunded, the incoming HTLC was never redeemed. It says nothing about griefing. `Refund_Possible` (exists-trace) only shows that a sender can be refunded in a run without key compromise. That honest parties will be refunded is a progress property, which trace lemmas do not express.

[R14] *at "Removing it falsifies Intermediary_Never_Loses_Under_Liveness."*

More precisely: once T3 (`IntermediaryMustClaim`) is removed, TAMARIN finds a trace without any key compromise in which an intermediary's outgoing HTLC is redeemed while its incoming HTLC is refunded (`Necessity_IMC_Honest_Intermediary_Loses`, exists-trace, 28 steps). This trace violates `Intermediary_Never_Loses_Under_Liveness`. (Also fix "T3sec." to "T3 (Sec. IV-A)".)

[R15] *at "As a result, the payment either completes"*

TODO: replace this sentence by: "As a result, the payment is meant to either complete along the entire path or fail. The wormhole attack (end of this section and Sec. V-E) shows that colluding intermediaries can break this for the hops between them."

[R16] *at "Each HTLC becomes an additional conditional output"*

(This describes the protocol. In our model, HTLCs are not outputs of commitment transactions; see Sec. IV.) Our model also simplifies amounts and deadlines. Amounts are local to a hop: `Forward_HTLC` reads the incoming amount, the fee and the outgoing amount from the network, and its only constraint on them is `Eq(%vIn, %vOut %+ %fee)`. The invoice `<'invoice', R, S, y>` and the sender's offer carry no amount. Amounts are therefore not chained along the route, and we make no claim of end-to-end value conservation. Deadlines are not numbers either: the ordering (3) is captured by the restriction `OutgoingTimesOutFirst` (T2b) and the lemma `ForwardTimeGap` derived from it. Auxiliary models with explicit block heights (`lightning_aux.spthy`, flags CLOCK and CLTV) check the numeric claim-window argument (e.g., `Claim_Window_Exists`, `Claim_Window_Nonempty`, `Staggered_Path_Safe`).

[R17] *at "funds return to the party that locked them."*

Once the receiver has released *x*, if every intermediary claims its incoming HTLC in time, all HTLCs on the path are settled. An intermediary that fails to do so after its outgoing HTLC was redeemed loses the forwarded amount, while the parties downstream are paid. This is why forwarders need liveness (Remark 2).

[R18] *at "redeeming the HTLC."*

In our model, HTLCs are resolved without on-chain transactions and the preimage is never sent on the network, so the adversary never learns it (`Preimage_Never_Known`, Sec. IV-A). Compromised parties can reuse a preimage only through the explicit rule `AttackerSkip`.

[R19] *at "within their claim window [21]."*

Finally, all HTLCs of a payment are locked by the same hash *y*. Two colluding intermediaries on the route can therefore pass *x* around the hops between them, so that the payment completes while the bypassed hops are refunded and earn no fee: this is the wormhole attack [6]. Sec. V-E gives such a trace, checked by TAMARIN, and Sec. VI proves that, for routes of any

length, a hop of a payment that the sender settles, and whose hash is offered once, can be bypassed only if some party's long-term key is compromised.

III. Symbolic Model and Notation

We modeled our protocol using Tamarin [16]. It is symbolic tool for security-protocol analysis, which tracks a *system* represented as a multiset of rewriting facts—refered to as state S. It evolves following user defined transition *rules* of the following shape:

$$[L] \text{ —— } [A] \longrightarrow [R] \tag{4}$$

with the premises (L), action facts (A), and conclusions (R).

(4) can be triggered when L unifies with facts in S. It captures that L should be rewritten as R within S while remembering the action facts A.[R20] Tamarin features linear and persistent facts; the former are consumed upon rule application, while the latter can be reused arbitrarily many times. The tool also provides built-in symbols $In()$, $Out()$ and $Fr()$ capturing respectively sending and recieving over a public (i.e., attacker controlled) channel and generating fresh and unique value (e.g., for nonces). We further use the `hashing`, `signing`, and `natural-numbers` built-ins[R21]

Simon
Not sure the "emembering the facts" is the right way to say it.

Simon
maybe point to somewhere to say what these do concretly

An execution produces a trace of action facts. Security goals are then first-order formulas over these trace, enably security queries—queries over all traces—, and reachability queries—ensuring a certain trace exists.

IV. Formal Modeling Methodology[R22]

~~AI: Restructure proposal — make this section about the ASSUMPTION INTERFACE, in three moves: (1) Why not monolithic: sig + equational theory + nat induction + unbounded clock exhausts memory (you have first-hand evidence; Grundmann must bound the search in TLA+). One or two sentences, cite, move on. (2) What we did instead: ONE theory, one adversary, one clock vocabulary; but be honest about the coupling: the layers share only the channel TOPOLOGY (!ChannelConnect minted at open, consumed by routing; the Mutex slot) – HTLC state is deliberately NOT folded into channel states, and the real bridge is the named assumption OneResolutionPerHTLC. Its DISCHARGE ARGUMENT should be spelled out in the paper as the centerpiece: resolving an HTLC twice requires publishing a revoked state; publishing a revoked state only happens after cheating (No_Punishment_Without_Cheating, a channel-layer theorem in the same theory); under punishment-liveness a rational party never does it. Argued, not mechanized — say so, in one sentence, then classify the restriction set by provenance: (a) encoding well-formedness (Equality, Inequality, LessThan, TickOnce, TicksInOrder, ExpiryBlockReached, OneChannelPerPtr) – symbolic plumbing, uncontroversial; (b) the layer interface: OneResolutionPerHTLC — this is THE bridge; justified by the channel-layer lemmas (resolving an HTLC twice requires publishing a revoked state, which only happens after cheating, No_Punishment_Without_Cheating); plus OneHTLCPerHashPerChannel; (c) liveness: ForwarderResolvesInTime (honest forwarders only, prefix-closed) – the CLTV-delta contract made formal. (3) Necessity: each load-bearing assumption stress-tested by removal: drop ForwarderResolvesInTime and Intermediary_Never_Loses fails with a counterexample; timeout.spthy shows the early-timeout race reachable; AttackerSkip shows the wormhole still reachable after the single-spend fix (so the fix is not too strong). A table (assumption / statement / who discharges it / what fails without it) IS the section. Do not apologize for the space: this table is the paper's claim to rigor.[S9]~~

~~As the Lightning Network is a very complex protocol, our model is very much split in two parts: the underlying Payment Channel protocol (Sec. II-A) and the Routing Layer (Sec. II-B). The latter assuming the security of the former and abstracting it away.[S10]~~

A. Threat model

Tamarin assumes the standard Dolev-Yao adversary model [10]; it typically controls the whole network; it can read, modify, and inject messages of arbitrary complexity; it is however constrained to the term algebra under consideration.[R23] ~~Such threat model is too strong for Blockchain protocols, in particular, the attacker no longer has full control over the network: 1) the blockchain grows following honnest behaviour[S11]~~

Red updates for page 5

[S9] struck *"AI: Restructure proposal make this section ... the paper's claim to rigor."*

Orange AI box describes a different model: csf_final has none of LessThan, TickOnce, TicksInOrder, ExpiryBlockReached, ForwarderResolvesInTime or timeout.spthy, and No_Punishment_Without_Cheating does not discharge OneResolutionPerHTLC. Its valid core (one theory, thin coupling, named interface, argued not mechanized) is implemented in the Sec. IV intro note.

[S10] struck *"As the Lightning Network is a ... former and abstracting it away."*

Wrong for csf_final: the model is one theory, and routing runs on channels opened by the channel layer instead of abstracting it away. Replaced by the Sec. IV intro note at the section heading.

[S11] struck *"Such threat model is too strong ... blockchain grows following honnest behaviour"*

Wrong for csf_final: the adversary keeps full Dolev-Yao control of the network and the main model has no blockchain or clock; liveness is imposed by restrictions on honest forwarders. Replaced by the threat-model note at 'term algebra under consideration.' The sentence continues on p.6 ('and 2) the participants ... liveness of the participant.'), which is struck on p. 6 as well.

[R20] *at "remembering the action facts A."*

TODO (wording, answers Simon): "... captures that *L* is rewritten as *R* in *S*, and the action facts *A* are appended to the trace."

[R21] *at "and natural-numbers built-ins"*

The hashing built-in provides `h`, used for payment hashes `y = h(x)` and for the commitment `h(s)` to a revocation secret. The signing built-in provides `sign`, `verify` and `pk`, used for the channel handshake, commitments and invoices. The natural-numbers built-in provides the sort of amounts (variables `%v`) with addition `%+`, used for the per-hop constraint `Eq(%vIn, %vOut %+ %fee).`

[R22] *at "IV. FORMAL MODELING METHODOLOGY"*

We model the protocol as a single TAMARIN theory (`lightning.spthy`, 37 rules) with a single adversary. It contains both the payment-channel protocol (Sec. II-A) and multi-hop routing (Sec. II-B). One generic rule, `Forward_HTLC`, handles every hop, so routes of any length are covered. The routing layer does not abstract the channel layer away, but the coupling is deliberately thin: commitment transactions carry no HTLC outputs, and the layers interact only through three explicit interfaces.

- **Topology.** `Lock_Funds_And_Open` mints the persistent facts `!ChannelConnect(ptr, $A, $B)` and `!ChannelConnect(ptr, $B, $A)`, which `Sender_Offer_HTLC`, `Forward_HTLC` and `Open_Timeout` require. HTLCs are therefore added only on opened channels (`HTLC_On_Opened_Channel`).
- **Slot.** Opening also mints one linear fact `Mutex(ptr)`. Adding an HTLC consumes it, and settling or refunding that HTLC returns it. A channel therefore holds at most one in-flight HTLC at a time and can be reused afterwards.
- **Resolution.** Each HTLC is resolved (redeemed or refunded) at most once:

  ```
  restriction OneResolutionPerHTLC:
      "All ptr y #i #j. Old(ptr, y) @ #i & Old(ptr, y) @ #j ==> #i = #j"
  ```

  In the protocol this holds because an HTLC output of the current commitment can be spent only once on chain. Spending it from an older state requires publishing a revoked commitment, which the counterparty can punish (`Publish_Revoked_A/B`, `Punish_A/B`). This argument is not mechanized, and we do not claim a composition theorem.

All other abstractions are also explicit, named restrictions (Sec. IV-A).

[R23] *at "constrained to the term algebra under consideration."*

Our adversary keeps this full control of the network. Channel proposals and acknowledgements and invoices are sent over the network (`Out`/`In`). The adversary also chooses the values that honest rules read from the network: the balances of a channel update (`Update_State`) and, at each hop, the incoming amount, the fee and the outgoing amount (`Forward_HTLC`, whose only constraint on amounts is `Eq(%vIn, %vOut %+ %fee)`). HTLC offers and fulfilments between adjacent hops are modelled as linear facts (`Route`, `Fulfilled`), that is, as authenticated, non-replayable messages between channel partners.

Key compromise. Rule `Compromise_Ltk` reveals the long-term signing key of any party at any time (action `LtkCompromised(A)`, fact `!Compromised(A)`). The adversary can then sign as that party, e.g., forge its invoices (reachable: `Forged_Invoice_Requires_Key_Compromise`). The preimage *x* of an invoice is never sent on the network, and the adversary never learns it (`Preimage_Never_Known`). What compromised parties can do with it is given explicitly by

rule `AttackerSkip`: two compromised parties that offered or forwarded HTLCs with the same payment hash may move the fulfilment of one party's outgoing HTLC to the other party's outgoing HTLC. In effect, they relay x around the parties between them (the wormhole, Sec. V-E). Apart from their keys, `AttackerSkip` and the exemptions below, compromised parties follow the protocol rules.

Time. Time is abstract: the timeout of channel `ptr` is the event `TimedOut(ptr)` of rule `Open_Timeout`, which can happen at any point after the channel is opened and at most once per channel (`OneTimeoutPerPtr`). The adversary therefore chooses when timeouts happen relative to honest actions. The liveness restrictions T1, T2b and T3 below constrain this choice. They do more than constrain the adversary: as TAMARIN restrictions, they remove every trace that violates them, including traces in which an uncompromised forwarder could not have claimed or timed out in the required order; our results say nothing about such traces. T1 applies to every HTLC; T2b and T3 bind only forwarders whose key was not compromised before their incoming HTLC timed out.

and 2) the participants can always see the blockchain and interact with it. Item 2 notably enforces the *liveness* of the participant.[S12]

Consequently, our model uses TAMARIN`restrinctions`[R24] to capture the limitation of the attacker.

a) Liveness: **[NOTE]** Just to make sure before I rewrite things:

- We encode things like the timeout choices with restrictions (T2*)
- We prevent Redeeming before a timeout via restriction (T1). Which sort of raises question marks to me. I need to look again at why we need it because you likely explained it to me at some point.
- We counter Rmk. 1 with Restr. T2b.
- We **don't** encode liveness of punishement (Sec. II-A2e) via restrictions.

Notably the last point is quite important because this is how this section is framed. I might have rewritten some stuff wrongly because I had the wrong mental model...

Also this is quite space inefficient. But I believe it is important enough to be worth the space. **[END NOTE]**[S13]

As mentioned before, the security of the Lightning Network relies on the liveness of the agents, whether to defer cheating behaviour (Sec. II-A2e) or to claim HTLCs (Rmk. 1). However, a standard Dolev-Yao adversary has full control over the network. It can notably keep an agent from seeing the network (e.g., preventing it from noticing a fraud) or sending a message (e.g., sending the defending transaction), leading to trivial attacks on the Lightning Network [21].

To make the model relevant, we restrict[R25] the attacker's control over the network to allow for liveness. We also rule out pathological honnest behaviour. Concretely, we impose the following restrictions, which constrain the *routing* layer. So, in our model, the punishment's liveness(sec. II-A2e) is not encoded but results when the counterparty acts within its window.

AI: OUTDATED: this T1/T2a/T2b/T3 taxonomy describes the pre-merge restrictions. On more-lemmas, RedeemBeforeTimeout was dropped (redeem and refund now genuinely race; only ForwarderResolvesInTime makes honest forwarders win in time), ForwardTimeGap became a LEMMA, and T2a/T2b/T3 collapsed into the clock restrictions (TickOnce, TicksInOrder, ExpiryBlockReached) plus ForwarderResolvesInTime restricted to honest forwarders. Rewrite this block against the actual restriction set of Lightning.spthy, classified per the comment at the top of this section. The T3 necessity story survives almost verbatim: ForwarderResolvesInTime is the only obligation (not an ordering), and removing it falsifies Intermediary_Never_Loses_Under_Liveness.[S14]

Restriction T1 (`RedeemBeforeTimeout`)**.**[R26] *An HTLC must be redeemed before its timeout. This is not a liveness assumption but a clock well-formedness.*[S15]

Restriction T2a (`ForwardTimeGap`)**.**[R27] *If both of a forwarding hop's HTLCs time out, the outgoing one hits its timeout first.*[S16]

Restriction T2b (`HonestPartiesActBeforeIncoming Timeout`)**.**[R28] *If the incoming HTLC of a forwarding hop times out, then the outgoing one timed out earlier.*[S17]

Restriction T3 (`IntermediaryMustClaim`)**.**[R29] *An intermediary claims its incoming HTLC once the outgoing HTLC is redeemed.*[S18]

[NOTE] V1 **[END NOTE]** Restr. T1 keeps the clock consistent, while Restrs. T2a and T2b capture the CLTV delta invariant. These three are structural. On the other hand, Restr. T3 is critically important for security. and Sec. V-D shows removing it falsifies `Intermediary_Never_Loses_Under_Liveness`. We proved the following properties: Payment Atomicity, Intermediary Safety, Routing Causality, and Value Conservation Sec. V-C, along with the timing results of Sec. V-D.[R30][S19]

[NOTE] V2 **[END NOTE]** Restrs. T1 to T2b are structural constraints of the CLTV clock and manage timing soundness. On the other hand, Restr. T3 is critically important for security and Sec. V-D shows removing it falsifies `Intermediary_Never_Loses_Under_Liveness`. We proved the following properties: Payment Atomicity, Intermediary Safety, Routing Causality, and Value Conservation Sec. V-C, along with the timing results of Sec. V-D.[S20]

[NOTE] This restriction feels a bit out of place currently **[END NOTE]**[S21]

Restriction Eq. *`EQ(a,b)` is `true` iff. `a` and `b` unify. This enables encoding equality.*[R31]

Simon
this is already said before, so potentially we can remove.

Simon
[for me] frame better

Simon
Do we need that one? Can't it simply be proven from T2b? I am struggling to give it a short description that doesn't overlapp too much with T2b...

Sana
I changed it a lil bit to avoid overdefining T1. but grouped 3 as structural. we have to keep one version either above or below.

Simon
see LaTeX

Red updates for page 6

[S12] struck *"and 2) the participants can always ... the liveness of the participant."*

End of the sentence struck on page 5 ('Such threat model is too strong ...'); replaced by the threat-model note on page 5.

[S13] struck *"[NOTE]Just to make sure before I ... worth the space. [END NOTE]"*

Simon's self-check note, resolved by this update: T2a is now a lemma (note at 'Restriction T2a'), T1 is well-formedness, T2b/T3 encode Rmk. 2 and the claiming liveness, punishment liveness is not encoded (TODO note at 'To make the model relevant').

[S14] struck *"AI: OUTDATED: this T1/T2a/T2b/T3 taxonomy describes ... and removing it falsifies Intermediary_Never_Loses_Under_Liveness."*

Wrong for csf_final: describes another model (RedeemBeforeTimeout dropped, TickOnce/TicksInOrder/ExpiryBlockReached, ForwarderResolvesInTime). Its T3 claim does hold for csf_final, but only through the separate counterexample Necessity_IMC_Honest_Intermediary_Loses, which violates Intermediary_Never_Loses_Under_Liveness (the direct proof attempt without T3 gives no result). Its one valid point (ForwardTimeGap is a lemma) is adopted in the T2a note.

[S15] struck *"An HTLC must be redeemed before ... tion but a clock well-formedness."*

Reads as an obligation; replaced by the exact csf_final restriction in the note at 'Restriction T1 (RedeemBeforeTimeout).'

[S16] struck *"Restriction T2a (ForwardTimeGap). If both of ... one hits its timeout first."*

ForwardTimeGap is a proven lemma in csf_final, not a restriction; replaced by the note at this spot.

[S17] struck *"Restriction T2b (HonestPartiesActBeforeIncoming Timeout). If the ... outgoing one timed out earlier."*

Named OutgoingTimesOutFirst in csf_final and guarded by non-compromise; replaced by the note at this spot.

[S18] struck *"An interme- diary claims its incoming ... the outgoing HTLC is redeemed."*

Omits csf_final's timeout premise and non-compromise guard; replaced by the note at 'Restriction T3 (IntermediaryMustClaim).'

[S19] struck *"[NOTE]V1 [END NOTE] Restr. T1 keeps ... timing results of Sec. V-D."*

Version V1 of a duplicated paragraph (resolves Sana's 'keep one version'). T2a is no longer a restriction, removing T3 falsifies Intermediary_Never_Loses_Under_Liveness only through the separate counterexample Necessity_IMC_Honest_Intermediary_Loses (the direct check gives no result), and there is no Value Conservation lemma. Replaced by the single paragraph in the note at 'Restr. T1 keeps the clock'.

[S20] struck *"[NOTE]V2 [END NOTE] Restrs. T1 to ... timing results of Sec. V-D."*

Version V2 of the same paragraph, with the same errors. Replaced by the single paragraph in the note at 'Restr. T1 keeps the clock'.

[S21] struck *"[NOTE]This restriction feels a bit outof place currently [END NOTE]"*

Resolved: Restr. Eq is now introduced together with the other structural restrictions (note at 'Restriction Eq.').

[R24] *at "Consequently, our model uses TAMARINrestrinctions"*

Replace by: Our model therefore uses TAMARIN restrictions to impose liveness on honest forwarders, as explained next.

[R25] *at "To make the model relevant, we restrict"*

TODO (answers Simon's struck [NOTE], checked against csf_final): (1) T1 (`RedeemBeforeTimeout`) is not needed by any of the five liveness lemmas (necessity runs, Sec. V-D), so it is presented as well-formedness. (2) The decreasing deadlines of Rmk. 2 are T2b (`OutgoingTimesOutFirst`); the claiming liveness stated after Rmk. 2 is T3 (`IntermediaryMustClaim`). (3) Correct: punishment liveness is not encoded; none of the nine restrictions of csf_final mentions `Cheat` or `Punish`.

[R26] *at "Restriction T1 (RedeemBeforeTimeout)."*

Replace the struck description by: *If an HTLC is redeemed and its channel times out, the redemption comes first.* This is not a liveness assumption but a well-formedness condition on timeouts: with T1 removed, all five liveness lemmas of Sec. V-D are still proved.

```
restriction RedeemBeforeTimeout:
    "All ptr y #i #j.
        Redeemed(ptr, y) @ #i & TimedOut(ptr) @ #j ==> #i < #j"
```

[R27] *at "Restriction T2a (ForwardTimeGap)."*

Replace by: **Lemma (ForwardTimeGap, formerly Restr. T2a).** *If both the incoming and the outgoing HTLC of a forwarding hop time out, and the forwarder was not compromised before its incoming timeout, then the outgoing one times out first.* This is no longer assumed: it follows from Restr. T2b and `OneTimeoutPerPtr`, and Tamarin proves it (`make verify`).

```
lemma ForwardTimeGap [reuse]:
    "All P Prev Next ptrIn ptrOut y #f #j #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #f
      & TimedOut(ptrOut) @ #j
      & TimedOut(ptrIn)  @ #k
      & not (Ex #c. LtkCompromised(P) @ #c & #c < #k)
     ==> #j < #k"
```

TODO (answers Simon's margin question): yes, in csf_final it is proven from T2b, so only T1, T2b and T3 remain as liveness restrictions.

[R28] *at "Restriction T2b (HonestPartiesActBeforeIncoming"*

Replace by: **Restriction T2b** (`OutgoingTimesOutFirst`). *If the incoming HTLC of a forwarding hop times out and the forwarder was not compromised before, then its outgoing HTLC timed out earlier.* This is the decreasing-deadline (CLTV-delta) invariant of Rmk. 2. Forwarders compromised before their incoming timeout are not constrained.

```
restriction OutgoingTimesOutFirst:
    "All P Prev Next ptrIn ptrOut y #f #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #f
      & TimedOut(ptrIn) @ #k
      & not (Ex #c. LtkCompromised(P) @ #c & #c < #k)
     ==> Ex #m. TimedOut(ptrOut) @ #m & #m < #k"
```

[R29] *at "Restriction T3 (IntermediaryMustClaim)."*

Replace the struck description by: *If a forwarder's outgoing HTLC is redeemed and its incoming HTLC times out, and the forwarder was not compromised before that timeout, then its incoming HTLC is redeemed too.* This is the claiming liveness of Rmks. 1 and 2. It only constrains traces in which the incoming HTLC times out, which are the only traces in which the upstream party can be refunded (`Refund_Requires_Timeout`). By T1, the claim then comes before the timeout. As a restriction, T3 does not penalise a missing claim but removes every such trace, including traces in which the forwarder could not have claimed in time; lemmas proved under T3 say nothing about those runs.

```
restriction IntermediaryMustClaim:
    "All P Prev Next ptrIn ptrOut y #f #i #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #f
      & Redeemed(ptrOut, y) @ #i
      & TimedOut(ptrIn)  @ #k
      & not (Ex #c. LtkCompromised(P) @ #c & #c < #k)
     ==> Ex #l. Redeemed(ptrIn, y) @ #l"
```

[R30] *at "Restr. T1 keeps the clock"*

Replace both versions (V1, V2) by: Restr. T1 is a well-formedness condition on timeouts. Restr. T2b encodes the decreasing deadlines of Rmk. 2, and `ForwardTimeGap` (formerly T2a) is derived from it. Restr. T3 is the liveness assumption on which intermediary safety rests. The necessity runs of Sec. V-D remove one restriction at a time. Without T1, all five liveness lemmas (`ForwardTimeGap`, `Timeout_Race_Blocked`, `Intermediary_Never_Loses_Under_Liveness`, `Intermediary_Never_Loses_Strong`, `Payment_Atomicity_Under_Liveness`) are still proved. Without T2b, `Timeout_Race_Blocked` and both intermediary-safety lemmas are still proved. Without T3, Tamarin finds a trace with no compromised party in which an intermediary's outgoing HTLC is redeemed while its incoming one is refunded (`Necessity_IMC_Honest_Intermediary_Loses`): an honest intermediary loses money. The remaining checks give no result within 600 s, and we draw no conclusion from them. Under these restrictions we prove payment atomicity, intermediary safety, routing causality, invoice authentication and preimage secrecy (Sec. V-C), the timing results of Sec. V-D, and that a forwarder at positions 1 to 6 of a route can be bypassed only if some key was compromised (Secs. V-E and VI). The bypass theorem for any position of a single-offer payment (Sec. VI) is proved without T1, T2b and T3.

[R31] *at "Restriction Eq. EQ(a,b) is true iff."*

Add after this paragraph: Restr. Eq (`Equality`) is one of six structural restrictions. `Equality` and `Inequality` turn the `Eq` and `Neq` action facts into constraints. `OneChannelPerPtr` and `OneTimeoutPerPtr` let a channel be opened at most once and time out at most once, and `OneHTLCPerHashPerChannel` lets an HTLC with a given payment hash be added at most once per channel. The sixth, `OneResolutionPerHTLC`, lets each HTLC be resolved (redeemed or refunded, action fact `Old`) at most once; it stands for a guarantee of the channel layer, which in our model does not carry HTLC outputs (Sec. IV-B).

```
restriction Equality:
    "All x y #i. Eq(x, y) @ #i ==> x = y"

restriction Inequality:
    "All x y #i. Neq(x, y) @ #i ==> not(x = y)"
```

B. Structural Limitations

~~AI: This subsection was written for the old five-theory split and now reads apologetic ('not a formal decomposition, no composition theory'). The merge changes it, but do not overshoot in the other direction: the honest statement is 'one theory, one adversary, one shared topology, and a THIN coupling' — HTLCs never appear in channel states, and the contract between layers is the named assumption OneResolutionPerHTLC, discharged by an argued proof sketch (PublishRevoked implies Cheat, plus punishment-liveness and rationality), not by a mechanized composition. Framing advice: (i) say plainly that folding HTLC state into channel states is exactly what explodes the state space – the thin coupling is a deliberate engineering decision, not an oversight; (ii) present the discharge argument for the interface assumption (see the comment at the top of this section) as the substitute for a composition theorem; (iii) list the NON-GOALS positively (onion privacy, on-chain fees, fee policy, wallet capacity, per-HTLC CSV stage), one clause each; (iv) a mechanized discharge (composition or refinement, Grundmann-style) is the natural follow-up work — one sentence, framed as future work, not as a confession. Reframe 'we acknowledge what our theories do not establish' as 'we state precisely what each assumption buys' — that is a result, not a confession.~~[S22]

The Lightning Network is a complex protocol. Verifying it as a single monolithic block quickly hits scalability problems with verification tools [8], TAMARIN is no exception.

This is not an assumption but an observation. When different built-ins are combined in the TAMARIN, the search space explodes. When the signature, equational theory, natural-number induction, and an unbounded clock are combined, the TAMARIN exhausts memory; any two of them remain traceable when combined.[R32] ~~To overcome this problem, we split the model into multiple theories, each verified individually.~~[R33][S23]

~~Splitting a model comes with limitations as well. We therefore acknowledge what our five theories do *not* establish. They are not a formal decomposition. In particular, there is no composition theory to claim that the five theories together yield a property of one combined theory. Each theory is its own self-contained model. In table I, the part of the protocol each theory covers is stated. Each theory has its own lemmas, each proving a specific part of the protocol.~~[R34]

Simon: I'll think of way to say this in a less weak sounding way.

~~The theories share structure and it justifies reading them together.~~[S24]

V. VERIFICATION OF THE MODEL

[NOTE] So this section seems to be doing 2 things.

1) Describe how the model was built
2) Describe the lemma verified

.

I am reframing things this way. **[END NOTE]**

A. The Core Model

[NOTE] NB: there are a bunch of things to be fished out everything after Sec. V-B **[END NOTE]** ~~As stated in Sec. IV-B our models splits the channels from the routing.~~[R35][S25]

1) Payment Channels: **[NOTE]** I am still not happy with this. But it is harder for me to know why... **[END NOTE]**

A channel is identified by a fresh pointer `ptr` to its founding transaction. Effectively our models tracks two *mirror* facts about this channel: `StateA` and `StateB`, one held by each party, each carrying the two balances (`fA`,`fB`), a state index `n` and that party's own *revocation secret* together with the corresponding signed commitment.[R36]

Simon: NB: This was mostly LLM generated. But I am fairly ok with it. It needs polishing though

The heart of the model is the *update dance*: commit a new state first, then revoke the old one. `Update_State` samples the next state index and both fresh revocation secrets, reads the two current mirrors and the new balances from the environment, and produces two *pending* mirrors without destroying anything.

```
rule Update_State:[R37]
   let newCommitA = <'commit', ptr, $A, $B,
       ~n_new, h(~sAnew)>
      sigAnew = sign(newCommitA, ltkA)
      ...
   in
   [ ...
    StateA($A, $B, ptr, n_old, sAold, oldCommitA,
        fA, fB),
    StateB($A, $B, ptr, n_old, sBold, oldCommitB,
        fA, fB),
    In(<fA_new, fB_new>) ]
   −[ NewStateBuilt($A, $B, ptr, ~n_new, fA_new,
      fB_new) ]→
   [ PendingA($A, $B, ptr, n_old, sAold,
      oldCommitA,
          ~n_new, ~sAnew, newCommitA, sigAnew,
              fA_new, fB_new),
    PendingB($A, $B, ptr, n_old, sBold, oldCommitB,
          ~n_new, ~sBnew, newCommitB, sigBnew,
              fA_new, fB_new) ]
```

Only afterwards does `Revoke_Old_Secret` supersede the old state: it consumes the two pending mirrors, installs them as the new state, and, crucially, publishes the *old* revocation secrets.[R38]

Simon: TODO: simplify the listing

```
rule Revoke_Old_Secret:[R39]
   [ PendingA($A, $B, ptr, ..., n_new, sAnew,
       newCommitA, sigAnew, ...),
    PendingB($A, $B, ptr, ..., n_new, sBnew,
       newCommitB, sigBnew, ...),
    !Pk($A, pkA), !Pk($B, pkB) ]
   −[ StateUpdate($A, $B, ptr, n_new),
      Revoke($A, n_old, sAold), Revoke($B, n_old,
         sBold),
      Eq(verify(sigAnew, newCommitA, pkA), true),
      Eq(verify(sigBnew, newCommitB, pkB), true)
         ]→
   [ StateA($A, $B, ptr, n_new, sAnew, newCommitA,
       fA_new, fB_new),
    StateB($A, $B, ptr, n_new, sBnew, newCommitB,
       fA_new, fB_new),
    !RevokedSecret($A, n_old, sAold, oldCommitA),
    !RevokedSecret($B, n_old, sBold, oldCommitB),
    RevokedCommitmentUnspent(ptr, $A, $B, n_old,
       oldCommitA),
    RevokedCommitmentUnspent(ptr, $B, $A, n_old,
       oldCommitB) ]
```

Red updates for page 7

[S22] struck *"AI: This subsection was written for ... a result, not a confession."*

Wrong for csf_final: written for another merge; its 'one theory' omits the five auxiliary models of lightning_aux.spthy, and the 'argued discharge' of OneResolutionPerHTLC is not in the artifact. Its valid points (thin coupling, no HTLCs in commitments, refinement as future work) are kept in the notes at 'To overcome this problem' and 'Splitting a model'.

[S23] struck *"To overcome this problem, we split ... multiple theories, each verified individually."*

csf_final is not split into independent theories; replaced by the note at this spot.

[S24] struck *"Splitting a model comes with limitations ... it justifies reading them together."*

Describes the old five-theory split; replaced by the note at this spot (one main theory plus five auxiliary models).

[S25] struck *"As stated in Sec. IV-B our ... the channels from the routing."*

In csf_final both layers are in one theory; replaced by the note at this spot.

[R32] *at "any two of them remain traceable when combined."*

TODO: "any two of them remain traceable" understates what we verify (and should read "tractable"): the main theory `lightning.spthy` combines signatures, hashing and natural numbers (`builtins: hashing, signing, natural-numbers`) and is verified in full; only the unbounded clock is kept out of it. Suggested: "...; without the unbounded clock, the remaining ones are tractable together."

[R33] *at "To overcome this problem, we split"*

Replace the struck sentence by: We therefore keep the block clock out of the main theory. `lightning.spthy` combines signatures, hashing and natural-number amounts, but represents timeouts as untimed events whose order is constrained by the restrictions of Sec. IV-A. The block clock and the CLTV arithmetic are analysed in small auxiliary models that use hashing and natural numbers only.

[R34] *at "Splitting a model comes with limitations"*

Replace the two struck paragraphs by:

Our artifact consists of one main theory and five small auxiliary models. The main theory, `lightning.spthy`, contains both the payment-channel layer (Sec. V-A1) and multi-hop routing over routes of any length (Sec. V-A2), under a single Dolev-Yao adversary that can also compromise long-term keys. The auxiliary models, all in `lightning_aux.spthy` and each selected by a preprocessor flag, cover what the main theory leaves out: the PTLC countermeasure (`-D=PTLC`) and its HTLC baseline (`-D=HTLC`), a block-clock model of CLTV deadlines (`-D=CLOCK`), CLTV-delta arithmetic (`-D=CLTV`) and the early-timeout race (`-D=TIMEOUT`). Each auxiliary model is self-contained and is not composed with the main theory; Table I states what each run establishes.

Inside the main theory, the two layers share only the long-term keys and the channel topology: `Lock_Funds_And_Open` mints the persistent `!ChannelConnect` facts and the per-channel slot `Mutex(ptr)` that the routing rules use. HTLC state is not folded into the commitment transactions: `StateA` and `StateB` carry balances and revocation data but no HTLC outputs. That an HTLC is resolved (redeemed or refunded) at most once is imposed by Restr. `OneResolutionPerHTLC` rather than derived from the channel mechanics. A mechanized link between HTLC resolution and the channel layer, for example by refinement, is future work.

[R35] *at "As stated in Sec. IV-B"*

Replace by: As stated in Sec. IV-B, the main theory contains both layers, as two groups of rules that share only the long-term keys and the channel topology.

[R36] *at "corresponding signed commitment."*

Replace "signed commitment" by "commitment": `StateA` and `StateB` hold the commitment `<'commit', ptr, $A, $B, n, h(sA)>` without its signature; signatures appear only in the pending mirrors and are checked in `Revoke_Old_Secret`.

[R37] *at "rule Update_State:"*

TODO: checked; the listing matches rule `Update_State` of csf_final. It elides only `newCommitB`, `sigBnew` and the premises `Fr(~n_new), Fr(~sAnew), Fr(~sBnew), !Ltk($A, ltkA), !Ltk($B, ltkB).`

[R38] *at "publishes the old revocation secrets."*

Replace "publishes the old revocation secrets" by "records the old revocation secrets as persistent facts": `!RevokedSecret` is a state fact, and no rule sends a revocation secret over the network.

[R39] *at "rule Revoke_Old_Secret:"*

TODO: checked; the listing matches rule `Revoke_Old_Secret` of csf_final. It elides only `n_old, sAold, oldCommitA` (resp. `n_old, sBold, oldCommitB`) and `fA_new, fB_new` in the two `Pending` premises.

Two details deserve emphasis. First, the signature checks of Sec. II-A are compiled into the trace constraint `Eq(verify(sigAnew, newCommitA, pkA), true);` a Tamarin rule cannot fail, so Restr. Eq turns these action facts into hard prerequisites. Second, the old revocation secret becomes *persistent* knowledge (`!RevokedSecret`), while the old commitment's spend becomes a *linear* one-shot token (`RevokedCommitmentUnspent`). This keeps the secret available for later punishment while guaranteeing a given revoked commitment is spent at most once, following the single-spend discipline of Sec. V-B0a. The order—commit, then revoke—is what protects a party that aborts mid-update: the old state is never revoked until a fully signed new one exists.

The remaining phases reuse the same facts, so we give only the closing path here and trust the pattern carries over. ~~`Lock_Funds_And_Open` mints the two mirrors together with the persistent `!ChannelConnect` and the one-shot slot `Free(ptr)`.~~[S26] A unilateral close spends the current mirror: `Publish_Current_A` turns `StateA` into an `OnChainDelayed` and an `OnChainInstant` output,

```
rule Publish_Current_A:                                   [R41]
   [ StateA($A, $B, ptr, n, sA, commitA, fA, fB),
     !ValidOnChain(ptr) ]
   –[ Publish($A, n, commitA), ... ]→
   [ OnChainDelayed($A, n, commitA),
       OnChainInstant($B, n, commitA) ]

rule Claim_Funds_A:
   [ OnChainDelayed($A, n, txContent) ]
   –[ ClaimFunds($A, n), CSVPassed($A, n),
       SettledTx($A, txContent) ]→
   [ FundsTo($A, n) ]
```

notably the `CSVPassed` action lets us capture that time has passed.

A cooperative close (`Honest_Close`) signs one shared settlement transaction instead, and punishment (`Publish_Revoked_A` consuming `RevokedCommitmentUnspent`, then `Punish_B`) transfers the whole channel to the honest party.[R42]

2) Lightning Network routing: The routing layer of Sec. II-B is a chain of HTLC offers in the forward direction and of fulfills in the backward one, and our model gives each direction its own fact. ~~Forward obligations travel as `Route($Prev,$Me,ptrIn,y)`; once an HTLC is fulfilled, the preimage travels back as `Fulfill(ptr,x)`. The one-shot slot `Free(ptr)` of Sec. V-A1 doubles as a per-channel lock: it is consumed whenever an HTLC is added, so no two forwardings can race on the same channel.~~[R43][S27]

The core of the model is the generic forwarding rule, one rule for any number of hops.

```
rule Forward_HTLC:                                        [R44]
   [ Route($Prev, $Me, ptrIn, y),
     !ChannelConnect(ptrOut, $Me, $Next),
     Free(ptrOut),
     In(%vIn), In(%fee), In(%vOut) ]
   –[ ForwardHTLC($Me, ptrIn, ptrOut, y),
      FeeCharged($Me, y, %fee),
      Eq(%vIn, %vOut %+ %fee),
      Neq(ptrIn, ptrOut), Neq($Prev, $Me),
          Neq($Me, $Next), ... ]→
   [ IntermediaryPending($Me, $Prev, $Next, ptrIn,
       ptrOut, y,
                %vIn, %vOut, %fee),
     Route($Me, $Next, ptrOut, y) ]                       [S28]
```

The premise `Route($Prev,$Me,ptrIn,y)` is consumed[R45] and a `Route($Me,$Next,ptrOut,y)` is produced: `Forward_HTLC` advances the payment one hop on the forward path, spending the slot of the outgoing channel via `Free(ptrOut)`. The incoming amount, the fee and the outgoing amount are all chosen by the environment; the constraint `Eq(%vIn, %vOut %+ %fee)` forces the two amounts and the fee to be consistent, which is exactly what the wormhole attack of Sec. V-E exploits. The `Neq` action facts keep the three parties and the two channels distinct. This rule is the concrete counterpart of the abstract `Forward_HTLC` of Listing 1 over the idealised routing layer.[S29]

The settle round runs in the other direction on separate facts. `Receiver_Create_Invoice` mints the invoice[R46] and keeps the preimage private in `ReceiverInv`; `Receiver_Fulfill_HTLC` reveals it, checked by the constraint `Eq(h(x),y)`. `Intermediary_Settle` then consumes `IntermediaryPending` and a `Fulfill(ptrOut,x)` to emit `Fulfill(ptrIn,x)` one hop upstream, recording `FeeEarned` when the intermediary actually collects; `Sender_Settle` finally closes the payment with `PaymentComplete`. The gap between the `FeeCharged` of the forwarding rule and this `FeeEarned` is precisely the window in which a wormhole steals the fee.[S30]

Finally, the model lets a payment fail without punishment. `Open_Timeout` marks a channel as timed out, and `Sender_Timeout_Refund` and `Intermediary_Timeout_Refund` consume the corresponding pending state to return funds (recording `Refunded`). ~~The complementarity of claiming and refunding is pinned down by the restrictions of the theory, e.g. `RedeemBeforeTimeout` and `IntermediaryMustClaim`, which encode the liveness requirement of Rmk. 1 symbolically.~~[R47][S31]

3) Interactions with the Blockchain: [R48] **[NOTE]** I want to more or less put Sec. V-B0a but it needs some work to be moveable here. **[END NOTE]**

B. Verified Lemma[R49]

[TODO] ~~Better section name and framing~~ **[END TODO]**[S32]

AI: these two listings are fished out of the orphaned paragraphs later in this file; that duplicate block should be removed once this section settles.

Simon: I agree with the AI, leave the comment until it is resolved.

Simon: NB: LLM generated, under review

AI: the claim/refund and liveness story here is borrowed from the (commented-out) guided tour of `model-tamari`; please double-check the restriction names and the attribution to Rmk. 1.

Red updates for page 8

[S26] struck *"Lock_Funds_And_Open mints the two mirrors to- ... the one- shot slot Free(ptr)."*

csf_final mints a reusable Mutex(ptr) and further persistent facts; replaced by the note at this spot.

[S27] struck *"Forward obligations travel as Route($Prev,$Me,ptrIn,y); once ... race on the same channel."*

Fulfill/Free(ptr) are Fulfilled/Mutex(ptr) in csf_final and the slot is given back on resolution; replaced by the note at this spot.

[S28] struck *"rule Forward_HTLC: 2 Route($Prev, $Me, ptrIn, ... 12 Route($Me, $Next, ptrOut, y)"*

Not the csf_final rule (Free, ForwardHTLC; no HTLCForwarded, HTLCAdded, !RouteInfo); replaced by the verbatim csf_final rule in the note at this spot.

[S29] struck *"The premise Route($Prev,$Me,ptrIn,y) is consumed and ... over the idealised routing layer."*

Wrong for csf_final: the slot is Mutex(ptrOut); the wormhole uses AttackerSkip, not the Eq amount constraint; Neq does not make Prev and Next distinct; there is no abstract Listing-1 counterpart. Replaced by the note at this spot.

[S30] struck *"Receiver_Create_Invoice mints the invoice and keeps ... a wormhole steals the fee."*

Fulfill and PaymentComplete do not exist in csf_final (Fulfilled, SenderSettled), and the preimage is never output; replaced by the note at this spot.

[S31] struck *"The complementarity of claiming and refunding ... requirement of Rmk. 1 symbolically."*

Wrong attribution (answers the AI margin request): T1 is not a liveness requirement, T2b encodes Rmk. 2 and T3 the claiming liveness. Replaced by the note at this spot.

[S32] struck *"[TODO]Better section name and frameing [ENDTODO]"*

Resolved: new title and opening framing in the note at 'B. Verified Lemma'.

[R40] *at "Lock_Funds_And_Open mints the two mirrors"*

Replace by: `Lock_Funds_And_Open` mints the two mirrors together with the persistent facts `!ValidOnChain(ptr)`, `!ChannelRecord(ptr, $A, $B)` and `!ChannelConnect` (one per direction), and the per-channel slot `Mutex(ptr)`, which the routing rules take and give back (Sec. V-A2).

[R41] *at "rule Publish_Current_A:"*

TODO: checked; both listings match csf_final. `Publish_Current_A` hides the action facts `PublishState($A, n)`, `OnChainDelayedProduce($A, n)`, `OnChainInstantProduce($B, n)` behind "..."; `Claim_Funds_A` is verbatim. The duplicate full listings on p. 9 are struck.

[R42] *at "then Punish_B) transfers the whole"*

Replace `Punish_B` by `Punish_A`: in csf_final, `Punish_A` consumes the `OnChainCheat($A, $B, ...)` and `RevealedSecret($A, ...)` facts produced by `Publish_Revoked_A` and yields `AllFundsTo($B)` (action `Punish($B, $A, n_old)`).

[R43] *at "Forward obligations travel as"*

Replace by: Forward obligations travel as linear facts `Route($Prev, $Me, ptrIn, y)`, and every added HTLC is also recorded in a persistent fact `!RouteInfo`. Once an HTLC is redeemed, the preimage travels back as `Fulfilled(ptr, x)`. The slot `Mutex(ptr)`, minted when the channel opens (Sec. V-A1), acts as a per-channel lock: it is consumed whenever an HTLC is added to the channel and given back when that HTLC is redeemed or refunded. Hence no two HTLCs are pending on the same channel at the same time, while the channel stays reusable for later payments.

[R44] *at "rule Forward_HTLC:"*

Replace the listing by the csf_final rule (verbatim):

```
rule Forward_HTLC:
    [ Route($Prev, $Me, ptrIn, y),
      !ChannelConnect(ptrOut, $Me, $Next),
      Mutex(ptrOut),
      In(%vIn), In(%fee), In(%vOut) ]
    --[ HTLCForwarded($Me, $Prev, $Next, ptrIn, ptrOut, y),
        HTLCAdded(ptrOut, y),
        FeeCharged($Me, y, %fee),
        Eq(%vIn,  %vOut %+ %fee),
        Neq(ptrIn, ptrOut), Neq($Prev, $Me), Neq($Me, $Next) ]->
    [ IntermediaryPending($Me, $Prev, $Next, ptrIn, ptrOut, y, %vIn, %vOut, %fee),
      Route($Me, $Next, ptrOut, y),
```

```
      !RouteInfo($Me, $Next, ptrOut, y) ]
```

[R45] *at "The premise Route($Prev,$Me,ptrIn,y) is consumed"*

Replace by: `Forward_HTLC` consumes `Route($Prev, $Me, ptrIn, y)` and produces `Route($Me, $Next, ptrOut, y)`. It advances the payment one hop, takes the slot `Mutex(ptrOut)` of the outgoing channel, and stores in `IntermediaryPending` everything the hop needs to settle or to be refunded later. `$Next` is any party with which `$Me` shares a channel, so routes are built hop by hop and have no fixed length; this single rule serves every forwarding hop of every route. The incoming amount, the fee and the outgoing amount are chosen by the environment and constrained only by `Eq(%vIn, %vOut %+ %fee)`. The action `HTLCForwarded($Me, $Prev, $Next, ptrIn, ptrOut, y)` identifies the hop in our lemmas, `HTLCAdded(ptrOut, y)` feeds Restr. `OneHTLCPerHashPerChannel`, and `FeeCharged` records the fee the hop expects. The `Neq` facts make the forwarder distinct from both of its neighbours and the two channels distinct from each other.

[R46] *at "Receiver_Create_Invoice mints the invoice"*

Replace by: `Receiver_Create_Invoice` signs the tagged invoice `<'invoice', $Receiver, $Sender, y>` with `y = h(~x)`, sends it with its signature, and keeps the preimage private in `ReceiverInv`. `Sender_Offer_HTLC` checks that signature before offering the first HTLC. `Receiver_Fulfill_HTLC` consumes the last `Route` fact, checks `Eq(h(x), y)`, emits `Redeemed(ptrIn, y)` and passes the preimage upstream as `Fulfilled(ptrIn, x)`. The preimage is never sent over the network, and the adversary never learns it (`Preimage_Never_Known`). `Intermediary_Settle` then consumes `IntermediaryPending` and `Fulfilled(ptrOut, x)`, emits `Fulfilled(ptrIn, x)` one hop upstream and records `FeeEarned` when the intermediary collects its fee. Both rules give the slot `Mutex(ptrIn)` back. `Sender_Settle` finally consumes `SenderPending` and `Fulfilled(ptrSF, x)` and closes the payment with `SenderSettled`.

[R47] *at "The complementarity of claiming and refunding"*

Replace by: Both refund rules also give the slot `Mutex` back. Timeouts are untimed channel events: `Open_Timeout` may fire at any point once the channel exists, at most once per channel (`OneTimeoutPerPtr`), and makes the pending HTLCs on that channel refundable. The interplay of claiming and refunding is pinned down by the three liveness restrictions of Sec. IV-A: `RedeemBeforeTimeout` forbids redeeming after the timeout, `OutgoingTimesOutFirst` encodes the decreasing deadlines of Rmk. 2, and `IntermediaryMustClaim` encodes the claiming liveness of Rmks. 1 and 2.

[R48] *at "Interactions with the Blockchain:"*

TODO: add at the end of Sec. V-A2, just before this heading:

Besides the Dolev-Yao network and key compromise (`Compromise_Ltk`), the adversary has one routing rule, used for the wormhole:

```
rule AttackerSkip:
    [ Fulfilled(ptrExit, x),
      !RouteInfo(ExitAtt, Next, ptrExit, y)[-,no_precomp],
      !RouteInfo(EntryAtt, Skipped, ptrSkip, y)[-,no_precomp],
      !Compromised(ExitAtt),
      !Compromised(EntryAtt) ]
    --[ FulfilledProduced(ptrSkip, x),
        Skipped(ptrSkip, x),   // annotation: marks a wormhole skip
        Eq(h(x), y) ]->
    [ Fulfilled(ptrSkip, x) ]
```

If two compromised parties, `ExitAtt` and `EntryAtt`, have each added an HTLC for the same payment hash `y`, on channels `ptrExit` and `ptrSkip`, then `AttackerSkip` takes the preimage released on `ptrExit` and delivers it on `ptrSkip`. `EntryAtt` can then settle upstream, while the hops between `ptrSkip` and `ptrExit`, starting with `Skipped`, never see their outgoing HTLC fulfilled and can at best be refunded. The rule emits `Skipped(ptrSkip, x)` but neither `Redeemed` nor `FeeEarned`. It yields the wormhole trace of Sec. V-E (`Wormhole_Fee_Theft_Reachable`), and the hop-bypass theorems (Sec. V-E, VI) show that, if the sender settles, a forwarder at positions 1 to 6 of the route, or at any position of a single-offer payment, is bypassed only if some key was compromised.

[R49] *at "B. Verified Lemma"*

Rename to **B. Verification Results** and open the subsection with: Table I lists every verification run of our artifact, one per Makefile target; `make all` runs them in order. All results were obtained with Tamarin 1.10 (which reports the Maude version used, 3.2, as unsupported) and reproduced on a laptop with a 6 GB heap limit (`MEM=6G`).

Theory File	Aspect Modeled	Safety	Reachability	Builtins
multihop.spthy	Lifecycle, 3-hop routing, value/fees, wormhole	33	10	Hashing, Signing, Nat
Clock.spthy	Block clock lifecycle & staggered CLTV claim windows	6	3	Natural numbers
Cltv.spthy	Pure CLTV block-arithmetic (delta positivity)	3	0	Natural numbers
timeout.spthy	Early-timeout race (T2b)	0	1	—
Distinct properties		42	14	
Modif.spthy	*Re-proof* of above over N-hop abstraction, fee/amount chaining	25	6	Hash, Sign, Multiset
Grand total (all 5 theories, 87 lemma checks)		67	20	

The first four theories establish 56 distinct properties, and the N-hop theory adds a further 5, for 61 in total. Of the N-hop theory's 31 lemmas, 26 restate properties already proved in multihop.spthy over the abstract routing layer and 5 are new to it (generic forwarding causality and the unbounded-path value lemmas), giving 87 lemma checks in total.

TABLE I: The five-theory modular architecture.[S33]

AI: OUTDATED NUMBERS: the 43/31/87 lemmas across five theories accounting describes the pre-merge files (multihop_modified.spthy, Modif.spthy, Clock/Cltv/timeout). After the merge there is ONE theory: Lightning.spthy with 46 all-traces lemmas proven automatically in a single run (make verify re-proves them) plus 8 exists-trace witnesses with hand-saved, replayed proof scripts, and the separate 4-rule timeout.spthy toy for the race counterexample. This is a big SIMPLIFICATION win for the paper: the '26 of the 31 lemmas re-prove what multihop already proved' bookkeeping disappears entirely, and so does the awkward 'five theories, no composition result' framing. Regenerate figures/theory-table.tex accordingly (two rows: Lightning.spthy, timeout.spthy). Also re-anchor the lemma listings to the merged model: several lemma statements changed shape (ForwardHTLC now carries amounts and expiries; lemma names mostly survive). The appendix catalogue has the same re-anchoring problem, but that is mechanical.[S34]

[TODO] We may want to rename the files. Modif.spthy doesn't feel very professionnal g. Also we are inconsistent in the file we name. [END TODO][S35]

[NOTE] SJ: I feel like we need a paragraph to explain the table, but this one is rather weak. We likely want to explain in plainer english which file models and verifies what instead of how many things it verifies. Potentially stating things like "mulithop.spthy is our main file" [END NOTE][S36]

We verified 87 lemmas accross the fives theories shown in Table I. The concrete theory multihop_modified.spthy contributes 43 lemmas: 33 all-traces and 10 exists-trace witnesses (non-vacuity checks and the attack traces below); 5 of the all-traces lemmas are [reuse] helpers. The N-hop theory, Modif.spthy, re-verifies 26 of these multihop_modified.spthy lemmas over the abstract routing layer and adds 5 new to it, for 31 lemmas in total. The remaining lemmas come from the three small timing theories, Clock.spthy, Cltv.spthy, and timeout.spthy.[S37]

Simon: isn't this basically what is already written in the table?

[TODO] Move to the right place. This is the old "fee" section from above. I believe it fits better here where we talk about the details of the mode. I am not sure where exactly yet though. [END TODO][S38]

Our model includes the value of each HTLC[R50] and the fee at each hop. The reason is that the wormhole attack in sec. V-E steals routing fees. Amounts are natural numbers terms, and each forwarding rule takes its outgoing amount and its fee from the network (In(%fee), In(%vOut)), constrained only by the action fact Eq(%vIn, %vOut %+ %fee). The fee is a per-hop quantity rather than Lightning's base_fee + fee_rate formula. We can not prove anything about fee *policy*.[S39]

Fees are charged and earned by two separate events. FeeCharged is emitted when a forwarder locks its outgoing HTLC; FeeEarned is emitted only when it later claims its incoming one. The wormhole is exactly the gap between them, which is what Wormhole_Steals_Exactly_The_Fees measures.[S40][R51]

We skip three things deliberately.[R52] Forward1_HTLC has no Wallet premise. So, a channel can carry any amount, so a forward never fails for lack of funds. The linear fact Free(ptr) is spent on each added HTLC. Therefore, a channel holds no more than one HTLC at a time. We also did not include onion routing. Each intermediary knows the full path explicitly (Sender, Forward1, Forward2). So, no claims about hiding who pays whom are made. The wormhole appears purely as fee theft; [6] also characterize it.[S41] StateA and StateB carry a state index and a revocation secret, not the output structure from Fig. 2. The two-stage HTLC claim becomes a single transition. The second stage adds a CSV delay and a revocation path. These are already enforced by Publish_Current and Claim_Funds.

```
rule Publish_Current_A:
[ StateA($A, $B, ptr, n, sA, commitA, fA, fB),
  !ValidOnChain(ptr) ]
 –[ Publish($A, n, commitA),
    PublishState($A, n),
    OnChainDelayedProduce($A, n),
    OnChainInstantProduce($B, n) ]→
[ OnChainDelayed($A, n, commitA),
  OnChainInstant($B, n, commitA) ]

rule Claim_Funds_A:
[ OnChainDelayed($A, n, txContent) ]
 –[ ClaimFunds($A, n), CSVPassed($A, n),
    SettledTx($A, txContent) ]→
[ FundsTo($A, n) ][S42]
```

[NOTE] End orphaned pragaraphs [END NOTE]

Fig. 4 shows the concrete 3-hop payment: lock, release,

Red updates for page 9

[S33] struck *"Theory File Aspect Modeled Safety Reachability ... I: The five-theory modular architecture."*

Five-theory table of the old lineage; replaced by the new Table I (insert after this page).

[S34] struck *"AI: OUTDATED NUMBERS: the 43/31/87-lemmas-across- five-theories ... problem, but that is mechanical."*

Wrong for csf_final: describes another model (Lightning.spthy with 46 all-traces + 8 exists-trace, separate timeout.spthy). The csf_final numbers are in the new Table I.

[S35] struck *"[TODO]We may want to rename the ... file we name. [END TODO]"*

Resolved: csf_final consists of lightning.spthy and lightning_aux.spthy, named consistently in the new Table I (Modif.spthy is gone).

[S36] struck *"[NOTE]SJ: I feel like we need ... our main file" [END NOTE]"*

Resolved: the new Table I comes with a plain-English paragraph saying which file models what ('lightning.spthy is our main model'); see the insert after this page.

[S37] struck *"We verified 87 lemmas accross the ... theories, Clock.spthy, Cltv.spthy, and timeout.spthy."*

Old-lineage counts and file names; replaced by the paragraph under the new Table I (insert after this page).

[S38] struck *"[TODO]Move to the right place. This ... exactly yet though. [END TODO]"*

Resolved: the corrected amounts/fees paragraph goes at the end of Sec. V-A2 (note at 'Our model includes the value').

[S39] struck *"Our model includes the value of ... prove anything about fee policy."*

csf_final reads all three amounts per hop (incl. In(%vIn)) and does not chain them along the path; replaced by the note at this spot.

[S40] struck *"The wormhole is exactly the gap ... is what Wormhole_Steals_Exactly_The_Fees mea- sures."*

Lemma Wormhole_Steals_Exactly_The_Fees does not exist in csf_final; replaced by the note at this spot.

[S41] struck *"We skip three things deliberately. Forward1_HTLC ... theft; [6] also characterize it."*

Forward1_HTLC, the one-shot Free(ptr) and the fixed path (Sender, Forward1, Forward2) belong to the old model; replaced by the note at this spot.

[S42] struck *"1 rule Publish_Current_A: 2 StateA($A, $B, ... SettledTx($A, txContent) 5 FundsTo($A, n)"*

Duplicates the Publish_Current_A and Claim_Funds_A listings on p. 8, which match csf_final (see the note there).

[R50] *at "Our model includes the value of each HTLC"*

Replace by (end of Sec. V-A2, after the `Forward_HTLC` paragraph): Amounts and fees are modelled per hop only. `Forward_HTLC` reads the incoming amount, the fee and the outgoing amount from the network (`In(%vIn)`, `In(%fee)`, `In(%vOut)`, natural-number terms) and enforces `Eq(%vIn, %vOut %+ %fee)`, so incoming = outgoing + fee holds at every hop by construction. The three values are kept in `IntermediaryPending`. `Sender_Offer_HTLC`, the invoice and the `Route` facts carry no amount, so amounts are not chained along the path, and the model makes no statement about end-to-end value, such as the amount the receiver obtains or the total fee the sender pays. We keep per-hop fees because the wormhole of Sec. V-E is a fee theft. The fee is a single per-hop quantity rather than Lightning's `base_fee + fee_rate` formula, and we prove nothing about fee policy.

[R51] *at "The wormhole is exactly the gap"*

Replace by: In the wormhole trace, the bypassed honest hop emits `FeeCharged` but never `FeeEarned`, although the sender settles and the receiver is paid (`Wormhole_Fee_Theft_Reachable`, Sec. V-E). Conversely, a hop whose key is never compromised earns a fee only after it has forwarded the payment and its outgoing HTLC has been redeemed (`Fee_Only_On_Successful_Forward`).

[R52] *at "We skip three things deliberately."*

Replace by: We deliberately leave out three things. First, channel capacity: `Forward_HTLC` has no `Wallet` premise and HTLCs do not change the balances in `StateA` and `StateB`, so a forward never fails for lack of funds. Second, concurrent HTLCs on one channel: `Mutex(ptr)` is taken when an HTLC is added to channel `ptr` and given back when that HTLC is redeemed or refunded, so a channel carries at most one pending HTLC at a time; later payments can reuse it, each with a payment hash not used on that channel before (Restr. `OneHTLCPerHashPerChannel`). Third, onion routing: the sender chooses only the first hop, each forwarder chooses its next hop among its own channels, and route information is kept in local state rather than sent in layered encryption. We therefore make no claims about hiding who pays whom, and the

wormhole appears purely as fee theft, as also characterized in [6].

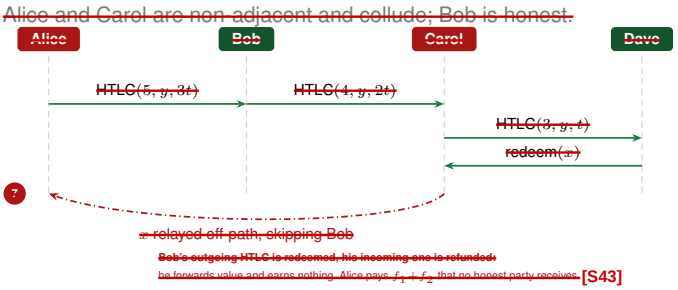
New Table I and its explanatory paragraph (replaces the struck old Table I, its caption, and 'We verified 87 lemmas ...')

TABLE I: Verification runs of the artifact (Tamarin 1.10), one row per Makefile target; `make all` runs them in this order. A = all-traces lemmas, E = exists-trace lemmas.

Target	File and flags	Content	Built-ins	A	E	Result
`make verify`	`lightning.spthy`	Main model: channel layer, multi-hop routing over routes of any length (one generic `Forward_HTLC`), wormhole rule `AttackerSkip`, key compromise. 32 safety lemmas; 13 hop-bypass lemmas (`First_Hop_`, `Second_Hop_`, `Hop3_` to `Hop6_Skip_Requires_Compromise`, plus 7 helpers); 10 exists-trace lemmas (9 found automatically, `Wormhole_Fee_Theft_Reachable` replayed from a saved 116-step proof)	hashing, signing, natural-numbers	45	10	55/55 verified
`make general`	`lightning.spthy -D=GENERAL -D=NO_IMC -D=NO_OTF -D=NO_RBT -D=NECESSITY`	Liveness restrictions T1, T2b, T3 removed. `Any_Hop_Skip_Requires_Compromise` (any route length, single offer per payment hash; 7 steps), its 7 helpers, and 16 default-mode lemmas re-proved in this mode	hashing, signing, natural-numbers	24	0	24/24 verified
`make aux`	`lightning_aux.spthy -D=PTLC`	PTLC countermeasure on the fixed route S, H1, H2, H3, R: `No_Wormhole_Around_Honest_Hop` (25 steps), 2 control traces, 2 sanity traces	hashing, natural-numbers	1	4	5/5 verified
	`lightning_aux.spthy -D=HTLC`	HTLC baseline on the same route: `Wormhole_Around_Honest_Hop` is reachable (12 steps); 2 sanity traces	hashing, natural-numbers	0	3	3/3 verified
	`lightning_aux.spthy -D=CLOCK`	Block-clock model of CLTV deadlines: channel/HTLC lifecycle, claim windows, preimage timing, exclusive outcomes; 3 reachability traces	hashing, natural-numbers	6	3	9/9 verified
	`lightning_aux.spthy -D=CLTV`	CLTV-delta arithmetic: positive gap, non-empty claim window, staggered path	hashing, natural-numbers	3	0	3/3 verified
	`lightning_aux.spthy -D=TIMEOUT`	Four-rule model without timeout ordering: the early-timeout race is reachable (`Early_Timeout_Race`)	hashing, natural-numbers	0	1	1/1 verified
`make necessity`	`lightning.spthy -D=NECESSITY` with one of `-D=NO_IMC`, `-D=NO_OTF`, `-D=NO_RBT`	One liveness restriction (T3, T2b or T1) removed per run; the 5 liveness lemmas re-checked, 600 s limit each; counterexample `Necessity_IMC_Honest_Intermediary_Loses` (with `-D=NO_IMC`)	hashing, signing, natural-numbers	15 checks	1	10 of 15 checks verified, 5 without result; counterexample verified (28 steps)
Total	verify + general + aux	16 of the 24 general-mode proofs re-prove default-mode lemmas in the liveness-free mode		79	21	100 lemma proofs, all verified (plus the necessity runs)

`lightning.spthy` is our main model. It contains the channel layer of Sec. V-A1, multi-hop routing with the single generic rule `Forward_HTLC` (Sec. V-A2) and the wormhole rule `AttackerSkip`, all under one Dolev-Yao adversary that can compromise long-term keys. In its default mode (`make verify`), all 55 lemmas are verified: 32 safety lemmas about channels and payments (Sec. V-C, V-D); 13 lemmas (six theorems and seven helpers) showing that the forwarders at positions 1 to 6 of a route cannot be bypassed unless some key was compromised (Sec. VI-A); and 10 exists-trace lemmas. These show that the model is not vacuous (channel updates, cooperative closes, refunds and honest multi-hop payments are reachable), that the assumptions of our safety lemmas matter (a payment without a matching invoice can settle once the receiver's key is compromised; a compromised intermediary can lose), and that the wormhole is reachable. `make general` runs the same file without the three liveness restrictions and proves the bypass theorem for routes of any length, `Any_Hop_Skip_Requires_Compromise` (Sec. VI); since removing restrictions only adds traces, this all-traces result also holds in the default mode. `lightning_aux.spthy` holds five small, self-contained models, one per flag: the PTLC countermeasure and its HTLC baseline (Sec. VI-E), two timing models (block clock, CLTV delta) that make explicit the deadlines the main model abstracts into untimed timeout events, and the early-timeout race, the trace pattern that `Timeout_Race_Blocked` excludes in the main model for uncompromised forwarders. `make necessity` removes one

liveness restriction at a time (Sec. V-D).



[S43]

Alice learns x from Carol directly, so Bob never sees it. Wormhole_Steals_Exactly_The_Fees shows the loss is exactly the path fees $f_1 + f_2$, not merely non-zero.

The two routing-layer failures, both reachable in `multihop.spthy`. Orange: benign (funds safe). Red: attack; red actors are dishonest.[S44]

Fig. 4: Wormhole[R53]

timeout refund, and wormhole. Each circled number matches a lemma group on the right that constrains that stage.

a) The Linear Single-Spend Discipline: We enforce the no-double-spend rule using linear facts. As mentioned before in Sec. III, a persistent fact $!F()$ is checked by a rule but not consumed, whereas a linear fact $F()$ is removed from the state when the rule fires.

In the Lightning network, the revocation secret is persistent knowledge (`!RevokedSecret`), and the revoked commitment output is a linear one-shot token (`RevokedCommitmentUnspent`), consumed when that state is published. Initially, the punishment rule only needed one thing to trigger: the persistent revocation secret alone:

```
rule Publish_Revoked_A: // WRONG: every premise persistent
  [ !ChannelRecord(ptr,$A,$B), !ValidOnChain(ptr),
    !RevokedSecret($A,n,s,c) ]
–[PublishRevoked($A,n,c), Cheat($A,n) ]→
  [ OnChainCheat($A,$B,n,c),
      RevealedSecret($A,n,s,c) ]
```
[S54]

Here, punishment is gated only by persistent `!RevokedSecret`, the rule can fire again and again for the same revoked state. If a party cheats, it lets the honest party take the channel funds many times. Lemmas like `No_Punishment_Without_Cheating` still pass, because a cheat is still required. What they miss is that punishment should happen only once. To fix this issue, we split it into two separate rules: `Revoke_Old_Secret` still stores the secret as persistent `!RevokedSecret`. It also creates a linear token `RevokedCommitmentUnspent`.[R55]

```
rule Revoke_Old_Secret:
  [ PendingA(...), PendingB(...), !Pk($A,pkA),
    !Pk($B,pkB) ]
–[StateUpdate($A,$B,ptr,n), Revoke($A,n_old,sAold)
  ]→
  [ StateA(...), StateB(...),
    !RevokedSecret($A,n_old,sAold,cA), //
      knowledge: persistent
    RevokedCommitmentUnspent(ptr,$A,$B,n_old,cA) ]
      // output: linear
```
[S45]

`Publish_Revoked` must consume that token, so the same revoked commitment can be punished only once:

#	Group	Lemmas	Kind
1	Invoice & preimage	Invoice_Authenticates_Settlement, Preimage_Secret_Until_Released, Forged_Invoice_Requires_Key_Compromise, Invoice_Released_Once	∀
2	Lock causality	Forward1_Requires_Offer, Forward2_Requires_Forward1, HTLC_On_Opened_Channel	∀
3	Release causality	Fulfill_Requires_Forward2, Claim_Requires_Release, Settle_Requires_Receiver_Release	∀
4	Atomicity	Payment_Atomicity_Under_Liveness, Settle_Excludes_Sender_Refund	∀
	non-vacuity	Multihop_Payment_Possible	∃
5	Value & fee	EndToEnd_Value_Conservation, Fee_Conservation_Hop1/2, Fee_Strictly_Positive_Hop1/2, Receiver_Paid_Invoice_Amount	∀
	non-vacuity	Fees_Charged_On_Path_Possible	∃
6	Refund / griefing	Refund_Requires_Timeout, Loss_Requires_Inaction	∀
	reachable	Honest_Intermediary_Refunds	∃
7	Wormhole fee theft	Wormhole_Fee_Theft_Reachable, Wormhole_Steals_Exactly_The_Fees	∃
8	Intermediary safety	Intermediary_Never_Loses_Under_Liveness, Timeout_Race_Blocked	∀

∀: all-traces safety; ∃: exists-trace witness.

TABLE II: Lemma groups of `multihop.spthy`, keyed to the circled numbers in Fig. 3a and 4.[S46]

```
rule Publish_Revoked_A: // FIXED: linear token gates the spend
  [ !ChannelRecord(ptr,$A,$B), !ValidOnChain(ptr),
    !RevokedSecret($A,n,s,c),
    RevokedCommitmentUnspent(ptr,$A,$B,n,c) ]
–[PublishRevoked($A,n,c), Cheat($A,n) ]→
  [ OnChainCheat($A,$B,n,c),
      RevealedSecret($A,n,s,c) ]
```
[S47]

Persistent knowledge of the revocation secret is retained via `!RevokedSecret`. Only spending the funds is linear.[R56]

The same critical flaw we found in our initial model of the routing layer. With `!ChannelConnect`, one channel could add unboundedly many HTLCs on the same output.[R57] We fixed it by adding `Free(ptr)`. `Lock_Funds_And_Open` produced it once the channel is open and consumed when an HTLC is added on that channel.[S48]

```
rule Lock_Funds_And_Open:
  [ ReadyToLock(...), Wallet($A, fA),
    Wallet($B, fB) ]
–[ChannelOpen($A,$B,ptr), FundsLocked($A,$B,ptr)
  ]→
  [ StateA(...), StateB(...), !ValidOnChain(ptr),
    !ChannelConnect(ptr,$A,$B),
      !ChannelConnect(ptr,$B,$A),
    Free(ptr) ] // one-shot HTLC slot
```
[S49]

Adding a linear premise can remove some exists-trace witnesses.[R58] So we re-ran the full concrete theory after the fix; all 43 lemmas still verify.[S50]

C. Protocol Properties

We state the guarantees of the concrete theory as lemmas.[R59] Each is named after the Tamarin property that establishes it. Adversarial capability is explicitly modeled as long-term key compromise(`Compromise_Ltk`). We apply rules to honest executions explicitly, not globally.

Red updates for page 10

[S43] struck *"Alice and Carol are non-adjacent and ... that no honest party receives."*

Figure content shows the old 4-party multihop.spthy trace (Alice colludes, amounts chained, Bob's outgoing HTLC redeemed); none of this is the verified trace. Redraw as described in the TODO note at 'Fig. 4: Wormhole'.

[S44] struck *"Alice learns x from Carol directly, ... attack; red actors are dishonest."*

Old caption: Alice is honest in Wormhole_Fee_Theft_Reachable, Wormhole_Steals_Exactly_The_Fees does not exist in lightning.spthy (amounts are not chained, so no 'loss is exactly the path fees' claim), and multihop.spthy is not the verified model. Replaced by the new caption in the note at 'Fig. 4: Wormhole'.

[S45] struck *"1 rule Revoke_Old_Secret: 2 PendingA(...), PendingB(...), ... persistent 6 RevokedCommitmentUnspent(ptr,$A,$B,n_old,cA) output: linear"*

Listing is not the lightning.spthy rule (StateUpdate takes n_new, rule revokes both parties, Eq checks missing); replaced by the verbatim rule in the note at 'It also creates a linear token'.

[S46] struck *"Group Lemmas Kind 1 Invoice preimage ... in Fig. 3a and 4."*

Old Table II (multihop.spthy): many of its lemmas (e.g. Forward1_Requires_Offer, EndToEnd_Value_Conservation, Wormhole_Steals_Exactly_The_Fees) do not exist in lightning.spthy, and Forged_Invoice_Requires_Key_Compromise is exists-trace, not all-traces. Replaced by the new Table II (red insert after this page).

[S47] struck *"1 rule Publish_Revoked_A: FIXED: linear token ... PublishRevoked($A,n,c), Cheat($A,n) 6 OnChainCheat($A,$B,n,c), RevealedSecret($A,n,s,c)"*

Not the exact lightning.spthy text (variables n_old, sAold, oldCommitA); replaced by the verbatim rule in the note at 'Persistent knowledge of the revocation secret'.

[S48] struck *"We fixed it by adding Free(ptr). ... is added on that channel."*

lightning.spthy uses Mutex(ptr), which is also given back when the HTLC resolves; replaced by the note at 'could add unboundedly many HTLCs'.

[S49] struck *"1 rule Lock_Funds_And_Open: 2 ReadyToLock(...), Wallet($A,~fA), ... 6 Free(ptr) one-shot HTLC slot"*

Not the lightning.spthy rule (Free(ptr) is now Mutex(ptr); NewStateBuilt and !ChannelRecord missing); replaced by the verbatim rule in the note at 'could add unboundedly many HTLCs'.

[S50] struck *"So we re-ran the full concrete ... all 43 lemmas still verify."*

Old theory and count; replaced by the note at 'Adding a linear premise can remove' (55 lemmas of lightning.spthy).

[R53] *at "Fig. 4: Wormhole"*

TODO (redraw Fig. 4 from the verified trace of Wormhole_Fee_Theft_Reachable in lightning.spthy):

- Five lifelines, left to right: Alice (sender, honest), Att1 (compromised, red), Indiana (honest forwarder), Att2 (compromised, red), Bob (receiver, honest). Att1 and Att2 are compromised: AttackerSkip requires !Compromised for its entry party (Att1) and its exit party (Att2).
- The lemma fixes no order of #t1 to #t8; draw the events in causal order.
- #t1 `InvoiceCreated(Bob, Alice, h(x))`: invoice from Bob to Alice.
- Lock, solid arrows left to right: #t2 `HTLCOffered(Alice, Att1, ptrAliceAtt1, h(x))`; #t3 `HTLCForwarded(Att1, Alice, Indiana, ptrAliceAtt1, ptrAtt1Indiana, h(x))`; #t4 `HTLCForwarded(Indiana, Att1, Att2, ptrAtt1Indiana, ptrIndianaAtt2, h(x))`; #t5 `HTLCForwarded(Att2, Indiana, Bob, ptrIndianaAtt2, ptrAtt2Bob, h(x))`. Label the arrows HTLC(h(x)) only: the model does not chain amounts or deadlines along the route, so drop HTLC(5, y, 3t), HTLC(4, y, 2t), HTLC(3, y, t).
- #t6 `HTLCFulfilled(Bob, Att2, ptrAtt2Bob, h(x))`: Bob redeems with x and is paid.
- Red dashed arrow from Att2 to Att1, passing over Indiana, labelled "x relayed by AttackerSkip"; Att1 then claims Alice's HTLC on ptrAliceAtt1.
- #t7 `SenderSettled(Alice, Att1, Bob, ptrAliceAtt1, x)`: Alice settles.
- #t8 `Refunded(Indiana, ptrIndianaAtt2, h(x))`: Indiana's outgoing HTLC to Att2 is refunded on timeout. Mark Indiana "no fee earned" (the lemma excludes any FeeEarned event of Indiana).
- Keep the circled 7 (Table II, group 7). Remove "Bob's outgoing HTLC is redeemed" and "Alice pays f1+f2 that no honest party receives": neither is in the verified trace.

New caption: Fig. 4: Wormhole (Wormhole_Fee_Theft_Reachable, lightning.spthy). Att1 and Att2 are compromised

(AttackerSkip requires this) and not adjacent; Alice, Indiana and Bob are honest. Bob redeems Att2's HTLC, AttackerSkip relays x from Att2 to Att1 around Indiana, and Att1 claims Alice's HTLC, so Alice settles and Bob is paid. Indiana's outgoing HTLC is refunded and Indiana earns no fee.

[R54] *at "rule Publish_Revoked_A: // WRONG"*

TODO: this pre-fix listing is not in lightning.spthy. Rename its variables n, s, c to n_old, sAold, oldCommitA, so that it differs from the verbatim fixed rule only by the missing RevokedCommitmentUnspent premise.

[R55] *at "It also creates a linear token RevokedCommitmentUnspent."*

TODO: in this sentence, "we split it into two separate rules" should read "we split it into two separate facts" (one rule, Revoke_Old_Secret, creates both).

Replace the struck listing by the rule as it stands in lightning.spthy. It revokes the old states of both parties: the persistent !RevokedSecret facts are the knowledge, the linear RevokedCommitmentUnspent facts the one-shot outputs.

```
rule Revoke_Old_Secret:
    [ PendingA($A, $B, ptr, n_old, sAold, oldCommitA,
               n_new, sAnew, newCommitA, sigAnew, fA_new, fB_new),
      PendingB($A, $B, ptr, n_old, sBold, oldCommitB,
               n_new, sBnew, newCommitB, sigBnew, fA_new, fB_new),
      !Pk($A, pkA), !Pk($B, pkB) ]
    --[ StateUpdate($A, $B, ptr, n_new),
        Revoke($A, n_old, sAold),
        Revoke($B, n_old, sBold),
        Eq(verify(sigAnew, newCommitA, pkA), true),
        Eq(verify(sigBnew, newCommitB, pkB), true) ]->
    [ StateA($A, $B, ptr, n_new, sAnew, newCommitA, fA_new, fB_new),
      StateB($A, $B, ptr, n_new, sBnew, newCommitB, fA_new, fB_new),
      !RevokedSecret($A, n_old, sAold, oldCommitA),
      !RevokedSecret($B, n_old, sBold, oldCommitB),
      RevokedCommitmentUnspent(ptr, $A, $B, n_old, oldCommitA),
      RevokedCommitmentUnspent(ptr, $B, $A, n_old, oldCommitB) ]
```

[R56] *at "Persistent knowledge of the revocation secret is retained"*

Replace the struck listing by the rule as it stands in lightning.spthy (Publish_Revoked_B is symmetric). Keep "FIXED: linear token gates the spend" as the listing caption; the rule itself has no comment.

```
rule Publish_Revoked_A:
    [ !ChannelRecord(ptr, $A, $B),
      !ValidOnChain(ptr),
      !RevokedSecret($A, n_old, sAold, oldCommitA),
      RevokedCommitmentUnspent(ptr, $A, $B, n_old, oldCommitA) ]
    --[ PublishRevoked($A, n_old, oldCommitA),
        Cheat($A, n_old) ]->
    [ OnChainCheat($A, $B, n_old, oldCommitA),
      RevealedSecret($A, n_old, sAold, oldCommitA) ]
```

[R57] *at "could add unboundedly many HTLCs on the same output."*

We fixed it with a linear slot Mutex(ptr). Lock_Funds_And_Open produces it once, when the channel opens; Sender_Offer_HTLC and Forward_HTLC consume it when they add an HTLC on that channel; Receiver_Fulfill_HTLC, Intermediary_Settle, Sender_Timeout_Refund and Intermediary_Timeout_Refund give it back when that HTLC is redeemed or refunded. HTLCs are thus added to a channel one at a time, and the channel can be reused once its HTLC has resolved.

```
rule Lock_Funds_And_Open:
    [ ReadyToLock($A, $B, ptr, sA0, sB0, commitA0, commitB0),
      Wallet($A, ~fA), Wallet($B, ~fB) ]
    --[ ChannelOpen($A, $B, ptr), FundsLocked($A, $B, ptr),
        NewStateBuilt($A, $B, ptr, '0', ~fA, ~fB) ]->
    [ StateA($A, $B, ptr, '0', sA0, commitA0, ~fA, ~fB),
      StateB($A, $B, ptr, '0', sB0, commitB0, ~fA, ~fB),
      !ValidOnChain(ptr),
      !ChannelRecord(ptr, $A, $B),
      !ChannelConnect(ptr, $A, $B),
      !ChannelConnect(ptr, $B, $A),
      Mutex(ptr) ]
```

[R58] *at "Adding a linear premise can remove"*

Replace the struck sentence by: So we re-ran the whole of lightning.spthy after the fix (make verify): all 55 lemmas verify, namely 32 all-traces safety lemmas, 13 hop-bypass lemmas, 9 exists-trace lemmas found automatically, and Wormhole_Fee_Theft_Reachable, replayed from a saved 116-step proof.

[R59] *at "the guarantees of the concrete theory"*

Replace "of the concrete theory" by "of lightning.spthy", and after "(Compromise_Ltk)" add: In addition, two compromised

parties on the same payment can relay a preimage to each other around the parties between them (rule AttackerSkip, Sec. V-E).

New Table II (replaces the struck Table II on page 10)

#	Group	Lemmas	Kind
0	Channel layer, keys	state_update, delayed_funds, instant_funds, settlement_is_traceable, No_Punishment_Without_Cheating, Funds_Locked_Before_Update, Update_Requires_Negotiation, Ltk_Known_Implies_Compromised	A
	non-vacuity	Protocol_execution, Cooperative_Close_Execution	E
1	Invoice & preimage	Invoice_Authenticates_Settlement, Preimage_Secret_Until_Released, Preimage_Never_Known, Invoice_Has_Secret_Preimage, Invoice_Released_Once	A
	reachable	Forged_Invoice_Requires_Key_Compromise	E
2	Lock causality	Forward_Requires_Incoming, HTLC_On_Opened_Channel, Offer_Requires_Invoice	A
3	Release causality	Claim_Requires_Release, Settle_Requires_Receiver_Release, Redeem_Requires_Receiver_Release, Fulfilled_Requires_Release, Fulfilled_Source	A
4	Atomicity	Payment_Atomicity_Under_Liveness, Settle_Excludes_Sender_Refund, OneOutcomePerHTLC	A
	non-vacuity	Multihop_Routing_Reachable, Sanity_Honest_Payment_Completes, Sanity_Atomic_Payment_Completes, Distinct_Parties_Configuration	E
5	Fees	Fee_Only_On_Successful_Forward (per-hop vIn = vOut + fee holds by construction)	A
6	Refund / griefing	Refund_Requires_Timeout, Loss_Requires_Inaction	A
	reachable	Refund_Possible	E
7	Wormhole	Skipped_Requires_Compromise	A
	reachable	Wormhole_Fee_Theft_Reachable (saved 116-step proof, replayed)	E
8	Intermediary safety, timing	Intermediary_Never_Loses_Under_Liveness, Intermediary_Never_Loses_Strong, Timeout_Race_Blocked, ForwardTimeGap	A
	non-vacuity	Sanity_Compromised_Intermediary_Can_Lose	E
	necessity ‡	Necessity_IMC_Honest_Intermediary_Loses	E
9	Hop bypass	First_Hop_Skip_Requires_Compromise, Second_Hop_Skip_Requires_Compromise, Hop3_Skip_Requires_Compromise, Hop4_Skip_Requires_Compromise, Hop5_Skip_Requires_Compromise, Hop6_Skip_Requires_Compromise	A
	helpers	Forward_Unique_Per_Incoming, SettledHop_From_Forward, SettledHop_Uses_Outgoing, No_Settle_And_Refund_Same_Hop, No_Fulfill_On_Forwarded_Incoming, Redeemed_Source, SenderSettled_Uses_Fulfilled	A
	any length †	Any_Hop_Skip_Requires_Compromise	A
	helpers †	Forward_Parent, Hop_Redeem_Propagates, Sender_Channel_Redeemed, Root_First, Next_Is_Child, Route_Redeemed_Gen, Route_Redeemed_No_Compromise	A

TABLE II: Lemma groups of lightning.spthy; the circled numbers in Fig. 3 and 4 refer to groups 1 to 7. A: all-traces; E: exists-trace. The unmarked rows list the 55 lemmas of make verify, all verified: 32 all-traces safety lemmas, the 13 hop-bypass lemmas of group 9, 9 exists-trace lemmas found automatically, and Wormhole_Fee_Theft_Reachable, replayed from a saved 116-step proof. †: proved in make general (-D=GENERAL, liveness restrictions removed; Sec. VI). ‡: exists-trace, checked with T3 removed (-D=NO_IMC; Sec. V-D, Table III).

Lemma 1 (Payment Atomicity)**.** *If both intermediaries forwarded an HTLC on hash y, the receiver's HTLC on y was redeemed, and no key was compromised, then the sender's HTLC on y cannot be refunded.*[R60]

```
lemma Payment_Atomicity_Under_Liveness:
"All F1 F2 ptrSF1 ptrF1F2 ptrF2R y #i #j #k.
   ForwardHTLC(F1, ptrSF1, ptrF1F2, y) @ #i
 & ForwardHTLC(F2, ptrF1F2, ptrF2R, y) @ #j
 & Redeemed(ptrF2R, y) @ #k
 & (not Ex P #l. LtkCompromised(P) @ #l)
 ==> not (Ex #m. Refunded(ptrSF1, y) @ #m)"
```
[S51]

Lemma 2 (Intermediary Safety)**.** *If a forwarder's outgoing HTLC is redeemed while its incoming HTLC is refunded, then that forwarder's long-term key was compromised.*[R61]

```
lemma
    Intermediary\_Never\_Loses\_Under\_Liveness:
"All P ptrIn ptrOut y #f #i #j.
   ForwardHTLC(P, ptrIn, ptrOut, y) @ #f
 & Redeemed(ptrOut, y) @ #i
 & Refunded(ptrIn, y) @ #j
 ==> Ex #r. LtkCompromised(P) @ #r"
```
[S52]

Lemma 3 (Routing Causality)**.** *Every forward is preceded by an upstream offer or by the predecessor's forward, carrying the same amount, unless the upstream party's key was compromised.*[R62]

```
lemma Forward1_Requires_Offer:
"All F1 F2 S ptrF1F2 y %vIn %vOut #i.
   HTLCForwarded1(F1, F2, S, ptrF1F2, y, %vIn,
       %vOut)
    @ #i
 ==> (Ex ptrSF1 #j. HTLCOffered(S, F1, ptrSF1,
       y, %vIn)
   @ #j & #j < #i)
   | (Ex #k. LtkCompromised(S) @ #k & #k < #i)"
```

It is followed by `Forward2_Requires_Forward1` *and* `Fulfill_Requires_Forward2` *in a 3-hop path. The same goes for* `Forward1_Requires_Offer_Honest`, `Forward2_Requires_Forward1_Honest` *and* `Fulfill_Requires_Forward2_Honest`, *with the additional restriction that the parties are honest (i.e., no key compromise). This is the "honest" version of the lemmas, which is what we need for the intermediary-safety proof.*[S53]

Lemma 4 (Value Conservation)**.** *At each hop, the incoming amount is the outgoing amount plus the fee charged there. End-to-end, the sender locks strictly more than the receiver's invoice amount.*[R63]

```
lemma Fee_Conservation_Hop1:
"All F1 F2 S p y %vIn %vOut %fee #i.
   HTLCForwarded1(F1, F2, S, p, y, %vIn, %vOut)
       @ #i
 & FeeCharged(F1, y, %fee) @ #i
 ==> %vIn = %vOut %+ %fee"
```

Followed by `Fee_Conservation_Hop2`, `Fees_Charged_On_Path_Possible`, *and lastly* `lemma EndToEnd_Value_Conservation`. *End-to-end conservation: over a full linked 3-hop payment, the sender always locks the receiver's amount along with a strictly positive total fee. Value is neither created nor destroyed.*[S54]

a) Non-vacuity: Safety lemmas are useless if the model cannot run at all, so we also checked `exists-trace` witnesses. `Multihop_Payment_Possible` shows that a full payment from the invoice to sender settlement is possible.[R64][S55] `Distinct_Parties_Configuration` shows three channels among four different parties, with no key compromise and no dispute. `Fees_Charged_On_Path_Possible` shows two different intermediaries each taking a fee on the same payment, so the value lemmas are not only about a one-hop path.[S56]

D. Timing and T3[R65]

As discussed Sec. IV-A, our model uses four timing restrictions (T1, T2a, T2b, and T3). `Cltv.spthy` derives the CLTV-delta invariant from concrete block numbers. `Clock.spthy` adds block-clock lifecycle safety. `timeout.spthy` shows the early-timeout race once T2b is removed. Together with `Lemma: Timeout_Race_Blocked` in the concrete theory`Multihop_modified`, this is a two-sided check: the race is reachable without T2b and blocked with it.[S57]

We also checked T3 behavior[R66] given T1/T2b. We removed T3 (`IntermediaryMustClaim`) and re-ran. `Intermediary_Never_Loses_Under_Liveness` failed, and Tamarin found a counterexample.[S58] T2b only orders the two timeouts; it does not force the intermediary to claim its incoming HTLC. That claim is exactly T3, and no ordering rule can force an action. So T3 is load-bearing for intermediary safety.[R67] We did not make claims for T1, T2a, or T2b as for T3. We therefore only checked T3 because it is the only one of the four that is an obligation, not an ordering.[S59]

E. Griefing and Wormhole[R68]

`Honest_Intermediary_Refunds` shows an honest intermediary locking capital and then refunding on timeout, with no settlement and no key compromise. The funds are still refundable, so safety still holds. We show this trace as a reachable griefing DoS, not as a new attack.[S60] The wormhole attack [6] is an instance of the agent-skipping attack formalized symbolically by [22]. It is a known attack, so we do not claim it as new. First, after the linear single-spend fix, it is a soundness check: the model must still admit this behavior, or the fix would be too strong.[R69] Second, our value layer lets us measure the loss. `Wormhole_Fee_Theft_Reachable` shows a payment that settles while the middle hop is never redeemed and earns no fee. `Wormhole_Steals_Exactly_The_Fees` pins that loss to exactly the path fees. Both appear as (6) and (7) in Figure 4.[S61]

Red updates for page 11

[S51] struck *"If both intermediaries for- warded an ... (Ex #m. Refunded(ptrSF1, y) #m)""*

Statement of the old 3-hop model (ForwardHTLC/4, Refunded/2, 'the sender'/'the receiver'); replaced by the lightning.spthy statement in the note at 'Lemma 1 (Payment Atomicity)'.

[S52] struck *"1 lemma Intermediary_Never_Loses_Under_Liveness: 2 "All P ... 6 Ex #r. LtkCompromised(P) #r""*

Old listing (ForwardHTLC/4, Refunded/2, visible _ escapes); prose above is still correct. Replaced by the verbatim lightning.spthy statement (plus Lemma 2') in the note at 'that forwarder's long-term key'.

[S53] struck *"Every forward is preceded by an ... need for the intermediary-safety proof."*

Forward1_Requires_Offer, Forward2_Requires_Forward1, Fulfill_Requires_Forward2 and the _Honest variants do not exist in lightning.spthy, and no amount is carried; replaced by Forward_Requires_Incoming (any hop) in the note at 'Lemma 3 (Routing Causality)'.

[S54] struck *"Lemma 4 (Value Conservation). At each ... is neither created nor destroyed."*

No value-conservation lemma exists in lightning.spthy (Fee_Conservation_Hop1/2, EndToEnd_Value_Conservation, Fees_Charged_On_Path_Possible are gone; amounts are not chained along the route). Replaced by Lemma 4 = Fee_Only_On_Successful_Forward in the note at 'Lemma 4'.

[S55] struck *"Multihop_Payment_Possible shows that a full payment ... to sender settlement is possible."*

Lemma does not exist in lightning.spthy; replaced by the csf_final witnesses in the note at 'a) Non-vacuity:'.

[S56] struck *"Fees_Charged_On_Path_Possible shows two different intermediaries each ... only about a one-hop path."*

Lemma does not exist in lightning.spthy (no value lemmas); replaced by the note at 'a) Non-vacuity:'.

[S57] struck *"As discussed Sec. IV-A, our model ... T2b and blocked with it."*

Wrong for lightning.spthy: three liveness restrictions (T2a is the lemma ForwardTimeGap), old file names, and Timeout_Race_Blocked holds even with T2b removed. Replaced by the note at 'D. Timing and T3'.

[S58] struck *"We also checked T3 behavior given ... and Tamarin found a counterexample."*

All three restrictions were checked, and the counterexample is the separate lemma Necessity_IMC_Honest_Intermediary_Loses (the direct proof attempt gave no result); replaced by the note at 'We also checked T3 behavior' and new Table III.

[S59] struck *"We did not make claims for ... an obligation, not an ordering."*

No longer true: T1 and T2b were checked too (Table III); replaced by the note at 'So T3 is load-bearing'.

[S60] struck *"Honest_Intermediary_Refunds shows an honest intermediary locking ... not as a new attack."*

Honest_Intermediary_Refunds does not exist in lightning.spthy (Refund_Possible is a sender refund); replaced by the note at 'E. Griefing and Wormhole'.

[S61] struck *"Second, our value layer lets us ... and (7) in Figure 4."*

Wormhole_Steals_Exactly_The_Fees does not exist in lightning.spthy and amounts are not chained, so no 'exact loss' claim; the trace description is outdated and (6) is in Fig. 3b. Replaced by the note at 'would be too strong.'.

[R60] *at "Lemma 1 (Payment Atomicity)."*

For any two consecutive forwarders F1 and F2 on a payment with hash y (F2's incoming HTLC is F1's outgoing one): if F2's outgoing HTLC on y is redeemed and no party's key was ever compromised, then F1's incoming HTLC on y is never refunded, by anyone.

```
lemma Payment_Atomicity_Under_Liveness:
    "All F1 F2 Prev1 Next1 Prev2 Next2 ptrSF1 ptrF1F2 ptrF2R y #i #j #k.
        HTLCForwarded(F1, Prev1, Next1, ptrSF1, ptrF1F2, y) @ #i
      & HTLCForwarded(F2, Prev2, Next2, ptrF1F2, ptrF2R, y) @ #j
      & Redeemed(ptrF2R, y) @ #k
      & (not Ex P #l. LtkCompromised(P) @ #l)
     ==> not (Ex Up #m. Refunded(Up, ptrSF1, y) @ #m)"
```

Since Forward_HTLC is one generic rule, F1 and F2 may sit anywhere on a route of any length; on a three-hop route ptrSF1 is the sender's HTLC and ptrF2R the receiver's. Sanity_Atomic_Payment_Completes shows that the premises are satisfiable.

[R61] *at "that forwarder's long-term key was compromised."*

Replace the struck listing by the statement in lightning.spthy (without the _ escapes in the name):

```
lemma Intermediary_Never_Loses_Under_Liveness:
    "All P Prev Next Up ptrIn ptrOut y #i #j #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #i
      & Redeemed(ptrOut, y) @ #j
      & Refunded(Up, ptrIn, y) @ #k
     ==> Ex #l. LtkCompromised(P) @ #l"
```

Lemma 2' (Intermediary Safety, strong form). *If a forwarder's outgoing HTLC is redeemed while its incoming HTLC is refunded, then that forwarder's key was compromised before its incoming HTLC timed out.*

```
lemma Intermediary_Never_Loses_Strong:
    "All P Prev Next Up ptrIn ptrOut y #i #j #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #i
      & Redeemed(ptrOut, y) @ #j
      & Refunded(Up, ptrIn, y) @ #k
     ==> Ex #l #t. LtkCompromised(P) @ #l & TimedOut(ptrIn) @ #t & #l < #t"
```

Lemma 2' matches the honesty condition of T2b and T3, which bind a forwarder only if it is not compromised before its incoming HTLC times out. Both lemmas quantify over every forward, so they hold for every hop of a route of any length, in every trace that satisfies T1, T2b and T3.

[R62] *at "Lemma 3 (Routing Causality)."*

Every forward on hash y is preceded by the HTLC it forwards: either the sender's offer on the forwarder's incoming channel, or the predecessor's forward that locked that channel's HTLC.

```
lemma Forward_Requires_Incoming [use_induction]:
    "All Me Prev Next ptrIn ptrOut y #i.
        HTLCForwarded(Me, Prev, Next, ptrIn, ptrOut, y) @ #i
     ==> (Ex S #j. HTLCOffered(S, Me, ptrIn, y) @ #j & #j < #i)
       | (Ex P0 pin0 #j. HTLCForwarded(Prev, P0, Me, pin0, ptrIn, y) @ #j & #j < #i)"
```

Because lightning.spthy has a single generic Forward_HTLC rule, this one lemma covers every hop of a route of any length, where a model with a fixed path needs one lemma per position (and an honest variant of each). It needs no honesty premise, because a forward consumes the linear Route fact, which only Sender_Offer_HTLC and Forward_HTLC produce. On the release side, Redeemed_Source and Fulfilled_Source state where a redemption and a consumed preimage come from:

```
lemma Redeemed_Source [reuse]:
    "All p y #i. Redeemed(p, y) @ #i
     ==> (Ex o. SettledHop(p, o, y) @ #i)
       | (Ex R Q. HTLCFulfilled(R, Q, p, y) @ #i)"

lemma Fulfilled_Source [reuse]:
    "All p x #i. UsedFulfilled(p, x) @ #i
     ==> (Ex #j. Redeemed(p, h(x)) @ #j & #j < #i)
       | (Ex #j. Skipped(p, x) @ #j & #j < #i)"
```

An HTLC is redeemed only by the receiver's fulfilment or by a forwarder settling that hop, and every preimage that a forwarder or the sender takes from channel p in order to settle comes from a redemption of the HTLC on p or from a wormhole skip (Skipped, Sec. V-E).

[R63] *at "Lemma 4 (Value Conservation)."*

Lemma 4 (Fees Only on Success). *A forwarder whose key is never compromised earns the fee of a hop only after it has forwarded on that hop and its outgoing HTLC has been redeemed.*

```
lemma Fee_Only_On_Successful_Forward [heuristic={route_info_first}]:
    "All Me ptrIn ptrOut y %fee #i.
        FeeEarned(Me, ptrIn, ptrOut, y, %fee) @ #i
      & not (Ex #c. LtkCompromised(Me) @ #c)
     ==> (Ex #j. Redeemed(ptrOut, y) @ #j & #j < #i)
       & (Ex Prev Next #f. HTLCForwarded(Me, Prev, Next, ptrIn, ptrOut, y) @ #f & #f < #i)"
```

We prove no value-conservation lemma. Per hop, `vIn = vOut + fee` holds by construction (the Eq action of Forward_HTLC, Sec. V-A2), but amounts are not chained along the route: the offer, the invoice and the Route fact carry no amount, so we make no end-to-end value claim.

[R64] *at "a) Non-vacuity:"*

Replace the struck sentence on Multihop_Payment_Possible by: Multihop_Routing_Reachable shows an HTLC routed from the sender over two forwarders to the receiver, who fulfils it. Sanity_Honest_Payment_Completes and Sanity_Atomic_Payment_Completes show payments over one and over two forwarders that settle end to end, with no key compromise and no refund: every forwarder claims its incoming HTLC and the sender settles. The second witness satisfies the premises of Lemma 1.

Replace the struck sentence on Fees_Charged_On_Path_Possible by: Sanity_Compromised_Intermediary_Can_Lose shows a compromised forwarder whose outgoing HTLC is redeemed while its incoming HTLC is refunded, so the premises

of Lemma 2 are satisfiable.

[R65] *at "D. Timing and T3"*

Replace the struck first paragraph by: Our model has three liveness restrictions (Sec. IV-A): T1 = RedeemBeforeTimeout, T2b = OutgoingTimesOutFirst and T3 = IntermediaryMustClaim. T2a is no longer assumed: ForwardTimeGap is a lemma derived from T2b and verified (Sec. IV-A).

Concrete block heights are treated in two auxiliary models of lightning_aux.spthy. With -D=CLTV, a positive CLTV delta (restriction DeltaPositive) leaves every forwarder a non-empty claim window: a preimage revealed before the outgoing deadline is revealed before the incoming one (Claim_Window_Nonempty), also over two staggered hops (Staggered_Path_Safe). With -D=CLOCK, channels carry concrete deadlines on a block clock, and nine lemmas (six all-traces, three exists-trace) cover the channel and HTLC lifecycle (e.g. No_HTLC_After_Close, Claim_Window_Exists, Loss_Implies_Skipped_Claim).

The four-rule model lightning_aux.spthy -D=TIMEOUT has no ordering restriction at all. There Tamarin finds the early-timeout race (exists-trace, 6 steps): a forwarder's incoming HTLC times out and is refunded, and its outgoing HTLC is redeemed after that timeout.

```
lemma Early_Timeout_Race:
    exists-trace
    "Ex P ptrIn ptrOut y #f #k #j #i.
        ForwardHTLC(P, ptrIn, ptrOut, y) @ #f
      & TimedOut(ptrIn)     @ #k
      & Refunded(ptrIn, y)  @ #j
      & Redeemed(ptrOut, y) @ #i
      & #k < #i"
```

In lightning.spthy the race is excluded for every forwarder that is not compromised before its incoming timeout:

```
lemma Timeout_Race_Blocked:
    "not (Ex P Prev Next Up ptrIn ptrOut y #l #i #j #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #l
      & TimedOut(ptrIn)     @ #k
      & Refunded(Up, ptrIn, y)  @ #j
      & Redeemed(ptrOut, y) @ #i
      & #k < #i
      & not (Ex #c. LtkCompromised(P) @ #c & #c < #k))"
```

This is a two-sided check: the race is reachable without ordering restrictions and blocked in lightning.spthy. Timeout_Race_Blocked stays verified when any single one of T1, T2b and T3 is removed (Table III).

[R66] *at "We also checked T3 behavior"*

Replace the struck sentences by: We removed each liveness restriction in turn (make necessity: -D=NO_IMC, -D=NO_OTF or -D=NO_RBT, 600 s per check) and re-ran the five lemmas that may depend on them (Table III). Without T3, Tamarin finds a counterexample (exists-trace, 28 steps):

```
lemma Necessity_IMC_Honest_Intermediary_Loses:
  exists-trace
  "Ex S F1 R p0 p1 x #o #a #r #u.
      HTLCOffered(S, F1, p0, h(x)) @ #o
    & HTLCForwarded(F1, S, R, p0, p1, h(x)) @ #a
    & HTLCFulfilled(R, F1, p1, h(x)) @ #r
    & Refunded(S, p0, h(x)) @ #u
    & not (Ex P #k. LtkCompromised(P) @ #k)
    & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m & HTLCAdded(ptr, y2) @ #n & not (#m = #n))
    & not (Ex Me P q z #e. IntermediaryClaimed(Me, P, q, z) @ #e)
    & not (Ex A B C q z #e. SenderSettled(A, B, C, q, z) @ #e)"
```

No key is compromised, F1 forwards, R redeems F1's outgoing HTLC, F1 never claims its incoming HTLC, and S's HTLC to F1 is refunded: the honest intermediary F1 loses, which violates Lemma 2 and Lemma 2'. (The direct proof attempts of these two lemmas without T3 gave no result within 600 s; the counterexample is this separate exists-trace lemma.)

[R67] *at "So T3 is load-bearing for intermediary safety."*

Replace the struck sentences by: The two ordering restrictions are not needed in the same way. T1 is not needed by any of the five lemmas: all of them stay verified without it. T2b is not needed for intermediary safety: Lemmas 2 and 2' stay verified without it. We draw no conclusion from the cells of Table III without a result; in particular we do not claim that ForwardTimeGap or Lemma 1 needs T2b, or that Lemma 1 needs T3.

[R68] *at "E. Griefing and Wormhole"*

Replace the struck sentences by: Refund_Possible shows a griefing trace: after the invoice, the sender's HTLC is refunded and the payment never settles, with no forwarding and no key compromise. Since a refund requires a timeout (Refund_Requires_Timeout), the capital stays locked until then. This witness refunds the sender, not an intermediary. We report it as reachable griefing (denial of service), not as a new attack; it belongs to group 6 of Table II, marked (6) in Fig. 3b.

[R69] *at "would be too strong."*

Replace the struck sentences by: Second, the model shows who can mount the attack. The wormhole is the attacker rule AttackerSkip (Sec. V-A2): two compromised parties on the same payment pass the preimage x from the exit party's outgoing HTLC to the entry party's outgoing HTLC, around the hops between them. Wormhole_Fee_Theft_Reachable (exists-trace; verified by replaying a saved 116-step proof) is the five-party trace of Fig. 4, marked (7):

```
lemma Wormhole_Fee_Theft_Reachable:
  exists-trace
  "∃ Alice Bob Att1 Att2 Indiana ptrAliceAtt1 ptrAtt1Indiana
     ptrIndianaAtt2 ptrAtt2Bob x #t1 #t2 #t3 #t4 #t5 #t6 #t7 #t8.
    ((((((((((((((InvoiceCreated( Bob, Alice, h(x) ) @ #t1) ∧
                  (HTLCOffered( Alice, Att1, ptrAliceAtt1, h(x) ) @ #t2)) ∧
                 (HTLCForwarded( Att1, Alice, Indiana, ptrAliceAtt1, ptrAtt1Indiana,
                                 h(x)
                  ) @ #t3)) ∧
                (HTLCForwarded( Indiana, Att1, Att2, ptrAtt1Indiana,
                                ptrIndianaAtt2, h(x)
                 ) @ #t4)) ∧
               (HTLCForwarded( Att2, Indiana, Bob, ptrIndianaAtt2, ptrAtt2Bob,
                               h(x)
                ) @ #t5)) ∧
              (HTLCFulfilled( Bob, Att2, ptrAtt2Bob, h(x) ) @ #t6)) ∧
             (SenderSettled( Alice, Att1, Bob, ptrAliceAtt1, x ) @ #t7)) ∧
            (Refunded( Indiana, ptrIndianaAtt2, h(x) ) @ #t8)) ∧
           (¬(Indiana = Alice))) ∧
          (¬(Indiana = Bob))) ∧
         (¬(Att1 = Att2))) ∧
        (¬(∃ #c. LtkCompromised( Alice ) @ #c))) ∧
       (¬(∃ #c. LtkCompromised( Bob ) @ #c))) ∧
      (¬(∃ #c. LtkCompromised( Indiana ) @ #c))) ∧
     (¬(∃ ptr1 ptr2 yy %fee #k.
         FeeEarned( Indiana, ptr1, ptr2, yy, %fee ) @ #k))) ∧
    (¬(∃ ptr y1 y2 #m #n.
        ((HTLCAdded( ptr, y1 ) @ #m) ∧ (HTLCAdded( ptr, y2 ) @ #n)) ∧
        (¬(#m = #n))))"
```

Alice, Bob and the intermediary Indiana are not compromised. Bob is paid and Alice settles, but Indiana's outgoing HTLC is refunded and Indiana earns no fee: an honest hop is bypassed. Conversely, a skip requires a key compromise:

```
lemma Skipped_Requires_Compromise [reuse]:
    "All p x #i. Skipped(p, x) @ #i ==> Ex P #c. LtkCompromised(P) @ #c & #c < #i"
```

The hop-bypass theorems First_Hop_Skip_Requires_Compromise, Second_Hop_Skip_Requires_Compromise and Hop3_ to Hop6_Skip_Requires_Compromise state, for the forwarder at position 1 to 6 of a route, that if the sender settles while that forwarder's hop is refunded (RefundHop), some party's key was compromised; they need no assumption on the number of offers but are proved in the full model, under T1, T2b and T3 (group 9 of Table II; Indiana in Fig. 4 is at position 2). Sec. VI extends this to routes of any length (Any_Hop_Skip_Requires_Compromise), for payments with a single offer; its conclusion, like that of the fixed-position theorems, says that some party's key was compromised, not which party or when. The PTLC subsection of Sec. VI shows, on the fixed route S, H1, H2, H3, R of lightning_aux.spthy, that per-hop locks rule out the attack around an honest hop when the sender is honest (No_Wormhole_Around_Honest_Hop), whereas the same route with a shared hash lock admits it (Wormhole_Around_Honest_Hop).

New Table III (Sec. V-D, referenced by the notes on page 11)

Lemma	T3 removed (-D=NO_IMC)	T2b removed (-D=NO_OTF)	T1 removed (-D=NO_RBT)
ForwardTimeGap	verified (6)	no result	verified (6)
Timeout_Race_Blocked	verified (4)	verified (4)	verified (4)
Intermediary_Never_Loses_Under_Liveness (Lemma 2)	no result	verified (4)	verified (4)
Intermediary_Never_Loses_Strong (Lemma 2')	no result	verified (4)	verified (4)
Payment_Atomicity_Under_Liveness (Lemma 1)	no result	no result	verified (10)
Necessity_IMC_Honest_Intermediary_Loses (exists-trace)	verified (28)	–	–

TABLE III: Necessity of the liveness restrictions (make necessity: lightning.spthy with -D=NECESSITY and one restriction removed per column, 600 s per check). T1 = RedeemBeforeTimeout, T2b = OutgoingTimesOutFirst, T3 = IntermediaryMustClaim. verified (n): proved, n proof steps. no result: neither a proof nor a counterexample within 600 s; we draw no conclusion from these cells. Necessity_IMC_Honest_Intermediary_Loses is checked only with T3 removed; its trace, in which an honest intermediary loses, is a counterexample to Lemmas 2 and 2'.

Liveness as restrictions: scope and alternatives (answers Simon's concern)

Two of our three liveness assumptions, T2b and T3, are TAMARIN restrictions of the form *if A then B happens* (T1 only orders a redemption before a timeout); for instance T3 (`IntermediaryMustClaim`): if an honest forwarder's outgoing HTLC is redeemed and its incoming HTLC times out, then its incoming HTLC is redeemed. A restriction removes every trace that violates it, including traces in which B is impossible rather than merely late. A lemma proved under T3 therefore says nothing about runs in which an honest forwarder *cannot* claim in time, which is where real timing attacks live. Likewise, T2b removes every trace in which an honest forwarder's incoming HTLC times out while its outgoing HTLC has not timed out earlier. We make the scope of this assumption explicit:

- **Which results depend on it.** Removing T3 breaks intermediary safety: TAMARIN finds `Necessity_IMC_Honest_Intermediary_Loses` (28 steps). The necessity runs (Sec. V-D) show, for each restriction, which of the five liveness lemmas are still proved without it. The 24 lemmas of the any-length theorem (Sec. VI) are proved with all three liveness restrictions removed (`make general`). The wormhole trace `Wormhole_Fee_Theft_Reachable` is verified with every restriction in place (by replaying a saved 116-step proof), so the restrictions do not hide it. All other all-traces lemmas of `make verify`, including the fixed-depth bypass theorems, are proved with all three restrictions in place; apart from the five lemmas of the necessity runs, we have not checked which of them still hold without them.
- **A claim window exists (separate model).** The Clock model (`lightning_aux.spthy -D=CLOCK`) has no liveness restriction. It proves `Claim_Window_Exists`: if a forwarder's outgoing HTLC is redeemed at block bOut and its incoming HTLC is refunded at block bRef, then bOut < bRef, so the forwarder had a non-empty window in which to claim. It also proves `Loss_Implies_Skipped_Claim`: a forwarder that loses never claimed. The Clock model is not composed with `lightning.spthy`, so it supports T3 but does not discharge it.
- **TODO (revision or future work):** replace the restrictions by the progress semantics of Backes, Dreier, Kremer and Künnemann [23]. Roughly, honest processes that can continue without input from the adversary eventually do. Traces in which the forwarder cannot act are then kept instead of pruned. TAMARIN 1.10 implements this for SAPIC+ processes (option `translation-progress`), so the routing part would have to be written as SAPIC+ processes. A cheaper first step: turn T1, T2b and T3 into premises of only the lemmas that need them, so that they no longer prune traces for every other lemma.

Reference to add (same entry as in the reference-list note on p. 13): [23] M. Backes, J. Dreier, S. Kremer, and R. Künnemann, “A Novel Approach for Reasoning about Liveness in Cryptographic Protocols and its Application to Fair Exchange,” in IEEE European Symposium on Security and Privacy (EuroS&P), 2017.

VI. Scaling to N Hops[R70]

AI: OBSOLETE SECTION — dissolve it. This section existed because the old architecture had a separate concrete 3-hop theory plus an abstracted N-hop theory. After the merge, Lightning.spthy IS the N-hop model (one generic Forward_HTLC, amounts and fees along Route facts, block clock), verified in the same run as the channel lemmas. Redistribution plan: the generic Forward_HTLC listing and the fee-chaining text move into the model section (core verification); the value lemmas merge with the existing value-conservation discussion. Note the two limitation paragraphs below ('Honest Scope of the Abstraction' and 'Why the wormhole disappears in N-hop') are RESOLVED by the merge: the adversary is back in the loop (In/Out for invoices, amounts, ticks; AttackerSkip as an explicit attacker rule) and the wormhole is reachable again and quantified (Wormhole_Fee_Theft_Reachable / steals exactly the fees / skipped node refunded). These two paragraphs were the weakest of the paper; deleting them is pure gain. Do check one thing before deleting: whether anything was still sacrificed for termination (e.g. attacker-chosen reordering of Route facts) and state it precisely if so.[S62]

The concrete theory fixes the path at three hops. The obvious next question is whether these safety rules work for routes of any length. The answer was not simple by simply adding rules. Also, more steps mean more options for attackers, and Tamarin gets stuck in a loop.[S63]

A. Abstracting the Routing Layer[R71]

The first problem was the signed HTLC offer. When tracing a forwarded message, Tamarin treats every party as a potential sender. With an N- hop, the analysis loops infinitely. To solve this, we swapped the public message for a private, single-use fact passed to the next intermediary.

Listing 1: Generic Forward

```
rule Forward_HTLC:
    [ Route($Prev, $Me, ptrIn, y, amtOut + fee),
    !ChannelConnect(ptrOut, $Me, $Next),
    Fee(ptrOut) ]
    −[ HTLCForwarded($Me, $Prev, $Next, ptrIn,
        ptrOut, y,
    amtOut + fee, amtOut),
    HTLCAdded(ptrOut, y),
    ForwardHTLC($Me, ptrIn, ptrOut, y),
    FeeExpected($Me, ptrIn, ptrOut, y, fee),
    Neq(ptrIn, ptrOut), Neq($Prev, $Me), Neq($Me,
        $Next) ] →
    [ IntermediaryPending($Me, $Prev, $Next, ptrIn,
        ptrOut, y,
    amtOut + fee, amtOut, fee),
    Route($Me, $Next, ptrOut, y, amtOut) ]
```

[S64]

This generic rule applies to each hop regardless of the path length. Because the HTLC is no longer a recursive signed message, Tamarin's origin analysis no longer loops[S65]

B. Fee Chaining Across Hops

In 1, `Route` carries the outgoing amount into the next hop. Each forward therefore enforces

$$v_{\text{in}} = v_{\text{out}} + \mathit{fee}.$$

The receiver is paid the invoice amount; everything above that is path fees. So end-to-end value conservation is a property of the unbounded model. So the value lemmas thus hold for arbitrary N, not only for the fixed 3-hop theory.[S66]

a) Honest Scope of the Abstraction: The Dolev-Yao network adversary is no longer in the N-hop model because the path is inside the Route fact. So it can no longer inject, replay, or forge messages. Therefore, N-hop theory does not claim security against network attacks. It only shows that the internal N-hop state machine stays sound for arbitrary N. This includes timeouts, preimage propagation, and local transitions.[S67]

*b) **Why the wormhole disappears in N-hop**:* The wormhole is not reachable in the abstract model, and this is structural rather than accidental. Skipping a hop means sending (x) around the honest intermediary. In the concrete model, two dishonest non-adjacent parties get that path from the Dolev-Yao network adversary. In `Route`, there is no such path. As a result, the attack cannot occur.[S68]

[NOTE] These last two points are a bit annoying. I'll see how I can spin them. **[END NOTE]**[S69]

VII. Related Work[R72]

M. Grundmann and H. Hartenstein in [8] model-check a stepwise refinement of lighting in TLA+, which bounds the explored state space. [9] presents a formally verified lighting network.[S70]

Red updates for page 12

[S62] struck *"AI: OBSOLETE SECTION dissolve it. This ... state it precisely if so."*

Orange AI box describes a different model (amounts and fees carried in Route facts, a block clock, ticks, a "steals exactly the fees" lemma); none of this exists in csf_final. Its one valid check (what the routing abstraction still takes away from the adversary) is answered in the note on VI-A and in the new Limitations section (inserts after this page).

[S63] struck *"The concrete theory fixes the path ... gets stuck in a loop."*

Obsolete: csf_final has no separate fixed-path theory; one theory with one generic Forward_HTLC covers routes of any length. Replaced by the introduction of the new Sec. VI (insert after this page).

[S64] struck *"Listing 1: Generic Forward 1 rule ... Route($Me, $Next, ptrOut, y, amtOut)"*

Obsolete listing (amounts in Route, Free(ptr), ForwardHTLC/FeeExpected actions, 8-argument HTLCForwarded). The csf_final Forward_HTLC is shown verbatim in the model section (the replacement listing in the note at 'rule Forward_HTLC:' on p. 8).

[S65] struck *"This generic rule applies to each ... origin analysis no longer loops"*

Refers to the struck Listing 1; its content is kept in the replacement text for VI-A (note at "A. Abstracting the Routing Layer").

[S66] struck *"B. Fee Chaining Across Hops In ... for the fixed 3-hop theory."*

False for csf_final: the offer, the invoice and Route carry no amount, so amounts are not chained along the path and no end-to-end value lemma exists. vIn = vOut + fee holds per hop only (Eq(%vIn, %vOut %+ %fee) in Forward_HTLC). Stated in the new Limitations section.

[S67] struck *"a) Honest Scope of the Abstraction: ... preimage propagation, and local transitions."*

No longer true: one theory and one Dolev-Yao adversary (invoices, amounts and key compromise go through In/Out and Compromise_Ltk). What the private Route fact still takes away from the adversary is stated precisely in the note on VI-A and in the new Limitations section.

[S68] struck *"b) Why the wormhole disappears in ... result, the attack cannot occur."*

False for csf_final: the wormhole is reachable through AttackerSkip (Wormhole_Fee_Theft_Reachable, replayed from a saved 116-step proof) and is characterised by the new Sec. VI: a bypass requires key compromise.

[S69] struck *"[NOTE]These last two points are a ... can spin them. [END NOTE]"*

Co-author note resolved: both paragraphs are struck because they are false for csf_final, and the remaining scope limits are stated precisely in the new Limitations section.

[S70] struck *"M. Grundmann and H. Hartenstein in ... a formally verified lighting network."*

Replaced by the rewritten Sec. VII in the insert after this page (also fixes "lighting").

[R70] *at "VI. SCALING TO N HOPS"*

VI. BYPASS RESISTANCE FOR ROUTES OF ANY LENGTH

TODO: new heading; the new text of Sec. VI is in the insert after this page. Of the old Sec. VI only the idea of VI-A survives, and it moves to the model section (next note).

[R71] *at "A. Abstracting the Routing Layer"*

TODO: keep this idea, it still holds for csf_final, but move it to Sec. V-A2, right after the paragraph on Forward_HTLC, and drop this heading. Suggested text:

HTLC offers between channel peers are not network messages in our model. With the offer as a public, signed message that each hop forwards, TAMARIN's origin analysis treats every party as a potential sender of a forwarded offer, and for routes of unbounded length the analysis did not terminate. The forward obligation is therefore the private, linear fact Route, which Sender_Offer_HTLC creates and each application of Forward_HTLC consumes and re-creates for the next hop. The price is that the network adversary cannot inject, replay or modify HTLC offers between peers. It still chooses the amounts and the fee of every forward (In(%vIn), In(%fee), In(%vOut)), controls the delivery of the signed invoices, and can compromise any long-term key at any time. A forwarder compromised before its incoming HTLC times out is exempt from T2b and T3, and two compromised parties that have each added an HTLC for the same payment hash can apply AttackerSkip.

[R72] *at "VII. RELATED WORK"*

TODO: replace the body of Sec. VII by the rewritten text in the insert "Sec. VII Related Work" after this page, and add the new reference for Kiayias and Litos (entry given there) to the reference list.

NEW Sec. VI: Bypass Resistance for Routes of Any Length (replaces the old Sec. VI)

VI. BYPASS RESISTANCE FOR ROUTES OF ANY LENGTH

The wormhole of Sec. V-E lets a payment settle while an honest forwarder between two compromised parties is bypassed: its outgoing HTLC is refunded and it earns no fee. We now show that in our model a bypass requires key compromise: for the forwarder at each of the positions 1 to 6 of a route, and, for payments whose hash is offered on a single channel, for a forwarder at any position of a route of any length. Sections VI-A to VI-D concern the main theory lightning.spthy; there is no separate N-hop theory.

A. Bypass requires compromise

The only rule that produces a Fulfilled fact for an HTLC that was not redeemed is AttackerSkip. It requires the two parties at the ends of the skip, ExitAtt and EntryAtt, to be compromised, and it records Skipped(ptrSkip, x); Fulfilled_Source states that every Fulfilled fact used in a settlement stems from an earlier redemption or skip. Every skip follows a compromise (Skipped_Requires_Compromise, Sec. V-E).

A forwarder is bypassed when the payment settles at the sender while the forwarder's own hop is refunded; Intermediary_Timeout_Refund records such a refund as RefundHop(ptrIn, ptrOut, y). For the first and the second forwarder on the sender's route we prove:

```
lemma First_Hop_Skip_Requires_Compromise [heuristic={lemmas_first}]:
    "All S F1 R pSF Next pOut x #s #f #r.
        SenderSettled(S, F1, R, pSF, x) @ #s
      & HTLCForwarded(F1, S, Next, pSF, pOut, h(x)) @ #f
      & RefundHop(pSF, pOut, h(x)) @ #r
     ==> Ex P #c. LtkCompromised(P) @ #c"

lemma Second_Hop_Skip_Requires_Compromise [heuristic={lemmas_first}]:
    "All S F1 R pSF x F2 p1 Next pOut #s #f1 #f2 #r.
        SenderSettled(S, F1, R, pSF, x) @ #s
      & HTLCForwarded(F1, S, F2, pSF, p1, h(x)) @ #f1
      & HTLCForwarded(F2, F1, Next, p1, pOut, h(x)) @ #f2
      & RefundHop(p1, pOut, h(x)) @ #r
     ==> Ex P #c. LtkCompromised(P) @ #c"
```

Hop3_Skip_Requires_Compromise to Hop6_Skip_Requires_Compromise state the same for the forwarder at positions 3 to 6, with the route from the sender's channel spelled out as a chain of HTLCForwarded facts. These six theorems and their seven helper lemmas (Forward_Unique_Per_Incoming, SettledHop_From_Forward, No_Settle_And_Refund_Same_Hop, No_Fulfill_On_Forwarded_Incoming, Redeemed_Source, SenderSettled_Uses_Fulfilled, SettledHop_Uses_Outgoing), the 13 hop-bypass lemmas of make verify, are verified in the full model, with all restrictions, and need no assumption on how often a payment hash is offered. In the wormhole trace Wormhole_Fee_Theft_Reachable, Indiana is the second forwarder (F1 = Att1, F2 = Indiana): Alice settles while Indiana's outgoing HTLC is refunded, and Alice, Bob and Indiana are uncompromised. The trace satisfies the premise of Second_Hop_Skip_Requires_Compromise, and the compromise in the conclusion is that of Att1 and Att2, which AttackerSkip requires; the conclusion cannot be dropped.

B. Routes of any length

Each fixed-depth theorem names the route up to the bypassed forwarder and therefore covers one position only. The general theorem quantifies over any forward of the payment hash instead:

```
lemma Any_Hop_Skip_Requires_Compromise [heuristic={lemmas_first}]:
  all-traces
  "∀ S F1 R pSF x Me Prev Next pIn pOut #s #f #r.
    ((((SenderSettled( S, F1, R, pSF, x ) @ #s) ∧
       (HTLCForwarded( Me, Prev, Next, pIn, pOut, h(x) ) @ #f)) ∧
      (RefundHop( pIn, pOut, h(x) ) @ #r)) ∧
     (∀ A B p #o. (HTLCOffered( A, B, p, h(x) ) @ #o) ⇒ (p = pSF))) ⇒
    (∃ P #c. LtkCompromised( P ) @ #c)"
```

The last conjunct of the premise is the *single-offer premise*: every HTLC offered for h(x) is offered on the sender's channel pSF.

C. Proof

The proof is organised in seven helper lemmas, proved in the same run. *Forward_Parent*: the incoming HTLC of every forward was offered by a sender or forwarded by the previous hop. *Root_First*: every forward of a payment hash is, or is preceded by, a forward whose incoming HTLC was offered by a sender. *Next_Is_Child*: if all offers of the hash are on one channel and Prev forwarded an HTLC to Me, every later forward of the hash is Me's forward of that HTLC or comes after it. *Hop_Redeem_Propagates*: without compromise, if a forwarder's incoming HTLC is redeemed, so is its outgoing one. *Sender_Channel_Redeemed*: without compromise, if the sender settles, its own channel was redeemed. *Route_Redeemed_Gen*: without compromise and if all offers of the hash are on one channel, redemption of the incoming HTLC of a forward carries over to the incoming HTLC of every forward of the hash that does not come earlier. *Route_Redeemed_No_Compromise* combines it with Sender_Channel_Redeemed and Root_First: without compromise and under the single-offer premise, if the sender settles, the incoming HTLC of every forward of the hash is redeemed.

Root_First, Next_Is_Child and Route_Redeemed_Gen are proved by induction over traces. The theorem then takes 7 proof steps. Assume no compromise. By Route_Redeemed_No_Compromise, the bypassed forwarder's incoming HTLC pIn is redeemed. By Redeemed_Source, this redemption is either the settlement of a forward from pIn, which Forward_Unique_Per_Incoming, SettledHop_From_Forward and No_Settle_And_Refund_Same_Hop exclude given RefundHop(pIn, pOut, h(x)), or a fulfilment by the receiver, which No_Fulfill_On_Forwarded_Incoming excludes. The proof scripts of Root_First, Next_Is_Child, Route_Redeemed_Gen, Route_Redeemed_No_Compromise and of the theorem are stored in lightning.spthy; the other helpers are proved automatically.

General mode and trace inclusion. The theorem is proved in a separate mode of the same theory, `make general` (-D=GENERAL -D=NO_IMC -D=NO_OTF -D=NO_RBT -D=NECESSITY), which enables the general lemmas and removes the three liveness restrictions T1 (RedeemBeforeTimeout), T2b (OutgoingTimesOutFirst) and T3 (IntermediaryMustClaim). Removing a restriction only adds traces, so every trace of the full model is a trace of this mode, and an all-traces property proved in this mode also holds in the full model. The run proves the theorem together with all 23 [reuse] lemmas available in this mode (24/24 verified), so no lemma proved under the removed restrictions is reused; ForwardTimeGap, which is derived from T2b, is not a [reuse] lemma in this mode. Bypass resistance for single-offer payments therefore does not depend on T1, T2b or T3.

D. Caveats

- *Single-offer premise.* The theorem is proved only for payments whose hash is offered on a single channel. If the same hash is offered on several channels, several routes carry it, and the theorem says nothing about a forwarder on such a payment. The fixed-depth theorems do not need the premise, because they spell out the route from the settling sender's channel; for such payments they cover positions 1 to 6 of that route.
- *Weak conclusion.* The conclusion states only that some party's long-term key was compromised at some point of the trace. It does not say which party (for instance a neighbour of the bypassed forwarder, as in the wormhole) and does not order the compromise before the settlement or the refund. The fixed-depth theorems have the same conclusion; only Skipped_Requires_Compromise orders the compromise before the skip.
- *Liveness restrictions.* The six fixed-depth theorems are proved in the full model, with T1, T2b and T3 in place, and we have not checked them without these restrictions. For a payment whose hash is offered on several channels, bypass resistance at positions 1 to 6 therefore rests on the liveness restrictions; only Any_Hop_Skip_Requires_Compromise (single-offer payments) is shown to hold without them.

Results. make verify: Skipped_Requires_Compromise and the 13 hop-bypass lemmas verified (full model). make general: 24/24 verified, Any_Hop_Skip_Requires_Compromise in 7 steps.

NEW Sec. VI-E: Countermeasure: PTLC (end of the new Sec. VI)

E. Countermeasure: PTLC

The wormhole works because every channel of an HTLC route is locked with the same hash h(x): once x is known at one point of the route, a compromised party can use it on a channel further upstream. Malavolta et al. [6] replace the shared lock by anonymous multi-hop locks, the basis of point time-locked contracts (PTLCs): every channel has its own lock, and a forwarder can compute the secret of its incoming lock only from the secret of its outgoing lock and its own tweak, chosen by the sender. We compare both designs in two small models of lightning_aux.spthy that differ only in how the channel secrets are derived and, accordingly, in how an honest hop obtains its incoming secret (Hop_Settle): -D=PTLC (per-hop secrets) and -D=HTLC (one shared secret, the baseline).

Model. Both models fix the route S → H1 → H2 → H3 → R over channels c0, c1, c2, c3. Sender_Setup draws a fresh secret x and per-hop tweaks t1, t2, t3 and derives the channel secrets by a hash chain s_in = h(<s_out, t>): s3 = x, s2 = h(<s3, t3>), s1 = h(<s2, t2>), s0 = h(<s1, t1>); channel ci is locked with h(si). An honest hop that learns the secret of its outgoing channel derives the secret of its incoming channel with its tweak (Hop_Settle). The hash chain is a symbolic stand-in for the additive tweak of PTLCs built from adaptor signatures [6], s_in = s_out + t: in both, deriving a hop's incoming secret requires that hop's tweak. In the HTLC baseline s0 = s1 = s2 = s3 = x, i.e. one hash lock h(x) on every channel, and Hop_Settle passes the outgoing secret on unchanged. The two rules of the PTLC model that implement the per-hop locks:

```
rule Sender_Setup:
    let s3 = ~x
        s2 = h(<s3, ~t3>)
        s1 = h(<s2, ~t2>)
        s0 = h(<s1, ~t1>)
    in
    [ Fr(~x), Fr(~c0), Fr(~c1), Fr(~c2), Fr(~c3), Fr(~t1), Fr(~t2), Fr(~t3) ]
    --[ Pay($S, $H1, $H2, $H3, $R, ~c0, ~c1, ~c2, ~c3),
        Neq($S, $H1), Neq($S, $H2), Neq($S, $H3), Neq($S, $R),
        Neq($H1, $H2), Neq($H2, $H3), Neq($H1, $H3),
        Neq($H1, $R), Neq($H2, $R), Neq($H3, $R) ]->
    [ !Lock(~c0, $S, $H1, h(s0)), Open(~c0), SenderWait($S, ~c0),
      !Hop($H1, ~c0, ~c1, $H2, h(s1), ~t1),
      !Hop($H2, ~c1, ~c2, $H3, h(s2), ~t2),
```

```
      !Hop($H3, ~c2, ~c3, $R,  h(s3), ~t3),
      !RecvSecret($R, ~c3, ~x),
      Out(h(s0)), Out(h(s1)), Out(h(s2)), Out(h(s3)) ]

rule Hop_Settle:
    [ Pend($H, cIn, cOut, t), ClaimInfo(cOut, sec), !Lock(cIn, P, $H, Lin), Open(cIn) ]
    --[ Redeemed(cIn), HopSettled($H, cIn, cOut), Eq(h(h(<sec, t>)), Lin) ]->
    [ ClaimInfo(cIn, h(<sec, t>)), Out(h(<sec, t>)) ]
```

The adversary is the same in both models: Dolev–Yao plus compromise of any party at any time (rule Compromise). A compromised hop leaks its tweak (Leak_Tweak); every secret used to claim a channel is published, as it would be on chain; a compromised payee may claim its channel with any value the adversary knows and that opens the lock (Adversary_Claim). There is no skip rule: a wormhole has to be built from adversary knowledge alone. Timing is abstracted: an open HTLC may be refunded at any time (Refund).

Results. With per-hop locks, the sender's channel c0 is never redeemed while H2's outgoing channel c2 is refunded, as long as S and H2 are uncompromised, whichever of H1, H3 and R are compromised (all-traces, verified, 25 steps):

```
lemma No_Wormhole_Around_Honest_Hop:
  all-traces
  "not (
Ex S H1 H2 H3 R c0 c1 c2 c3 #p #a #b.
      Pay(S, H1, H2, H3, R, c0, c1, c2, c3) @ #p
    & Redeemed(c0) @ #a
    & Refunded(c2) @ #b
    & not (Ex #k. Compromised(H2) @ #k)
    & not (Ex #k. Compromised(S) @ #k))"
```

With the shared hash lock, the same situation is reachable (exists-trace, verified, 12 steps):

```
lemma Wormhole_Around_Honest_Hop:
  exists-trace
  "Ex S H1 H2 H3 R c0 c1 c2 c3 #p #a #b.
      Pay(S, H1, H2, H3, R, c0, c1, c2, c3) @ #p
    & Redeemed(c0) @ #a
    & Refunded(c2) @ #b
    & not (Ex #k. Compromised(H2) @ #k)
    & not (Ex #k. Compromised(S) @ #k)"
```

Two control lemmas show that the PTLC result is not vacuous. Control_Refund_Of_H2_Outgoing (exists-trace, 6 steps): H2's outgoing HTLC can be refunded while H2 is honest, so the refund in the premise is not excluded by the model itself. Control_Wormhole_If_H2_Compromised (exists-trace, 18 steps): once H2 itself is compromised, the bypass is reachable with S still uncompromised, so the safety proof depends on H2 being honest.

```
lemma Control_Refund_Of_H2_Outgoing:
  exists-trace
  "Ex S H1 H2 H3 R c0 c1 c2 c3 #p #b.
      Pay(S, H1, H2, H3, R, c0, c1, c2, c3) @ #p
    & Refunded(c2) @ #b
    & not (Ex #k. Compromised(H2) @ #k)"

lemma Control_Wormhole_If_H2_Compromised:
  exists-trace
  "Ex S H1 H2 H3 R c0 c1 c2 c3 #p #a #b #k.
      Pay(S, H1, H2, H3, R, c0, c1, c2, c3) @ #p
    & Redeemed(c0) @ #a
    & Refunded(c2) @ #b
    & Compromised(H2) @ #k
    & not (Ex #j. Compromised(S) @ #j)"
```

Sanity lemmas in both models show that a payment completes end to end without any compromise (Honest_Payment_Completes: 10 steps in PTLC, 10 in HTLC) and with both neighbours H1 and H3 of H2 compromised (Colluders_Present_Payment_Completes: 14 steps in PTLC, 8 in HTLC). All eight lemmas are verified by make aux (PTLC 5/5, HTLC 3/3).

Limitations. The two models are separate from lightning.spthy: a fixed route with three intermediaries and one honest hop under attack, no channel layer, abstract timing, and an honest Sender_Setup that chooses all locks and tweaks consistently. The hash chain abstracts the group operation of the adaptor-signature construction: it captures that a hop's incoming secret requires that hop's tweak, but not the algebra of [6] (an XOR encoding exhausted memory in TAMARIN). We therefore claim the contrast between the two lock designs on this route, not a PTLC result for routes of any length.

Sec. VII Related Work: rewritten body + new reference

VII. RELATED WORK

Grundmann and Hartenstein [8] model-check a stepwise refinement of the Lightning Network in TLA+; model checking explores a bounded state space. Our TAMARIN proofs are symbolic and unbounded: subject to the restrictions of our model (Sec. IV-A), including three liveness restrictions, they cover any number of parties, channels, channel updates and

payments, and, for bypass resistance of single-offer payments (Sec. VI), routes of any length. [9] presents a formally verified Lightning Network. TODO: one sentence on the scope and the tool of [9] and how it differs from our work.

Kiayias and Litos [24] give a composable security treatment of the Lightning Network: they define an ideal functionality for a payment network and prove, in the universal composability framework, that the Lightning protocol realises it. Their guarantee is computational and composable, and their proof is by hand. Ours are symbolic but machine-checked by TAMARIN; they include bypass resistance for routes of any length (for single-offer payments), and they contrast the wormhole under HTLCs with its absence under PTLC-style locks on a fixed route.

Malavolta et al. [6] describe the wormhole attack on HTLC-based multi-hop payments and propose anonymous multi-hop locks as a countermeasure; their instantiation with adaptor signatures is the basis of PTLCs. Our main model reproduces the wormhole (Wormhole_Fee_Theft_Reachable, Sec. V-E), and our auxiliary models check the countermeasure symbolically on a fixed route (Sec. VI-E).

Smith et al. [22] formalise agent-skipping attacks on message-forwarding protocols; the wormhole is an instance of such an attack. Our bypass theorems (Sec. VI) state the corresponding security property for Lightning routing: a forwarder at positions 1 to 6 of a route, or at any position for single-offer payments, can be skipped only if some long-term key is compromised.

New reference (TODO: add to the reference list, same entry as in the note on p. 13; numbered by first citation, after Backes et al. [23], who are cited in the liveness-scope paragraph added after p. 11):

[24] A. Kiayias and O. S. T. Litos, “A Composable Security Treatment of the Lightning Network,” in 2020 IEEE 33rd Computer Security Foundations Symposium (CSF), 2020.

TODO: add pages and DOI from IEEE Xplore.

NEW Sec. VIII: Limitations (after Related Work, before the conclusion)

VIII. LIMITATIONS

- *Single-offer premise.* The any-length theorem assumes that every HTLC for the payment hash is offered on the sender's channel; when a hash is offered more than once, only the fixed-depth theorems (positions 1 to 6) apply, and these are proved under the liveness restrictions T1, T2b and T3. Its conclusion says that some long-term key was compromised, not whose or when.
- *No end-to-end amounts.* Each Forward_HTLC reads its incoming amount, fee and outgoing amount from the network and enforces Eq(%vIn, %vOut %+ %fee), but the offer, the invoice and the Route fact carry no amount. Amounts are therefore not chained along the path, and we make no end-to-end claims about value, e.g. that the receiver obtains the invoiced amount or that the sender pays exactly the path fees.
- *HTLCs are not part of the commitment transactions.* The channel layer and the routing layer share the theory, the adversary and the channel topology (HTLCs are added only on opened channels, HTLC_On_Opened_Channel), but HTLC outputs are not folded into the commitment states, and settling or refunding an HTLC does not update the channel state. The coupling that each HTLC is resolved at most once is assumed as the restriction OneResolutionPerHTLC, not derived from the channel layer; we do not claim a composition theorem. A channel carries at most one pending HTLC at a time (Mutex(ptr)).
- *Routing abstraction.* HTLC offers travel as the private linear fact Route, not as network messages, so the network adversary cannot inject, replay or modify offers between peers; it chooses amounts and fees, controls the delivery of invoices and can compromise keys. The adversary never learns a preimage (Preimage_Never_Known), so in the main model the wormhole comes from the explicit rule AttackerSkip, which requires the parties at both ends of the skip to be compromised, rather than from adversary knowledge. (The auxiliary PTLC and HTLC models have no such rule.)
- *No onion routing, no privacy.* Every forwarder knows its predecessor, its successor and the payment hash, and the next hop is any party with which the forwarder shares a channel. We make no privacy claims.
- *Abstract timing.* The main model has no clock: Open_Timeout lets a channel time out at any point allowed by the restrictions, and the restrictions T1, T2b and T3 (the last two only for forwarders not compromised before their incoming HTLC times out) stand in for CLTV deadlines. As restrictions, they remove every trace that violates them, including traces in which an uncompromised forwarder could not have claimed in time; results proved under them say nothing about such runs (only the any-length bypass theorem is proved without them). Concrete block heights appear only in the auxiliary CLOCK and CLTV models.
- *PTLC model.* The countermeasure is checked only on the fixed route S → H1 → H2 → H3 → R, with honest lock set-up and a hash chain in place of the adaptor-signature algebra (Sec. VI-E).

NEW: Artifact Availability (unnumbered, before the references)

ARTIFACT AVAILABILITY

The artifact consists of lightning.spthy (main model: channel layer, multi-hop routing and the wormhole; 37 rules), lightning_aux.spthy (five auxiliary models, selected by -D=PTLC, -D=HTLC, -D=CLOCK, -D=CLTV and -D=TIMEOUT) and a

Makefile. `make all` runs, in this order, `make verify` (lightning.spthy, full model: 55/55 lemmas verified), `make general` (Sec. VI: 24/24), `make aux` (lightning_aux.spthy: 21/21) and `make necessity` (Sec. V-D); Table I lists what each target proves. All results were obtained with TAMARIN 1.10 and reproduced on a laptop with a 6 GB heap (MEM=6G); `make verify-one` proves one lemma per run for machines with less memory. TAMARIN 1.10 reports the Maude version used, 3.2, as unsupported; all results above were obtained with it. Every run also reports that TAMARIN's derivation checks, part of its well-formedness checks of the model, timed out; this warning does not concern the proofs.

TODO: artifact URL (anonymised for review).

References

[1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," p. 21260, 2008.

[2] J. Poon and T. Dryja, "The bitcoin lightning network: Scalable off-chain instant payments," 2016.

[3] (2026) Lightning Network Statistics | 1ML - Lightning Network Search and Analysis Engine - Bitcoin mainnet. [Online]. Available: https://web.archive.org/web/20260810134919/https://1ml.com/statistics

[4] "Lightning/bolts," Lightning Network, 2026. [Online]. Available: https://github.com/lightning/bolts

[5] S. Tikhomirov, P. Moreno-Sanchez, and M. Maffei, "A Quantitative Analysis of Security, Anonymity and Scalability for the Lightning Network," in *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, 2020, pp. 387–396.

[6] G. Malavolta, P. Moreno-Sanchez, C. Schneidewind, A. Kate, and M. Maffei, "Anonymous Multi-Hop Locks for Blockchain Scalability and Interoperability," in *Proceedings 2019 Network and Distributed System Security Symposium*. Internet Society, 2019.

[7] L. Aumayr, O. Ersoy, A. Erwig, S. Faust, K. Hostáková, M. Maffei, P. Moreno-Sanchez, and S. Riahi, "Generalized Channels from Limited Blockchain Scripts and Adaptor Signatures," in *Advances in Cryptology – ASIACRYPT 2021*, M. Tibouchi and H. Wang, Eds. Springer International Publishing, 2021, vol. 13091, pp. 635–664.

[8] M. Grundmann and H. Hartenstein, "Security of the lightning network: Model checking a stepwise refinement with TLA+," in *Integrated Formal Methods*, F. Damiani and M. Farrell, Eds. Springer Nature Switzerland, 2026, pp. 313–335.

[9] G. , R. Stefański, and O. S. T. Litos, "A Formally Verified Lightning Network," in *Financial Cryptography and Data Security*, C. Garman and P. Moreno-Sanchez, Eds. Springer Nature Switzerland, 2026, pp. 3–20.

[10] D. Dolev and A. Yao, "On the security of public key protocols," vol. 29, no. 2, pp. 198–208, 1983.

[11] B. Blanchet, "A computationally sound mechanized prover for security protocols," in *2006 IEEE Symposium on Security and Privacy (S&P'06)*, 2006, pp. 15 pp.–154.

[12] G. Barthe, B. Grégoire, S. Heraud, and S. Z. Béguelin, "Computer-Aided Security Proofs for the Working Cryptographer," in *Advances in Cryptology – CRYPTO 2011*, ser. Lecture Notes in Computer Science, P. Rogaway, Ed. Springer, 2011, pp. 71–90.

[13] D. Baelde, S. Delaune, C. Jacomme, A. Koutsos, and S. Moreau, "An Interactive Prover for Protocol Verification in the Computational Model," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 537–554.

[14] S. Jeanteur, L. Kovács, M. Maffei, and M. Rawson, "CryptoVampire: Automated Reasoning for the Complete Symbolic Attacker Cryptographic Model." IEEE Computer Society, 2024, pp. 3165–3183.

[15] B. Blanchet, "An Efficient Cryptographic Protocol Verifier Based on Prolog Rules," in *14th IEEE Computer Security Foundations Workshop (CSFW-14)*. IEEE Computer Society, 2001, pp. 82–96.

[16] S. Meier, B. Schmidt, C. Cremers, and D. Basin, "The TAMARIN Prover for the Symbolic Analysis of Security Protocols," in *Computer Aided Verification*, ser. Lecture Notes in Computer Science, N. Sharygina and H. Veith, Eds. Springer, 2013, pp. 696–701.

[17] B. Lipp, B. Blanchet, and K. Bhargavan, "A Mechanised Cryptographic Proof of the WireGuard Virtual Private Network Protocol," in *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2019, pp. 231–246.

[18] C. Cremers, M. Horvat, J. Hoyland, S. Scott, and T. van der Merwe, "A Comprehensive Symbolic Analysis of TLS 1.3," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '17. Association for Computing Machinery, 2017, pp. 1773–1788.

[19] G. Andresen, "BIP 11: M-of-N Standard Transactions," 2011-10, 2011. [Online]. Available: https://github.com/bitcoin/bips/blob/master/bip-0011.mediawiki

[20] E. Lombrozo, J. Lau, and P. Wuille, "BIP 141: Segregated witness (consensus layer)," 2015-12, 2015. [Online]. Available: https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki

[21] P. McCorry, S. Bakshi, I. Bentov, S. Meiklejohn, and A. Miller, "Pisa: Arbitration Outsourcing for State Channels," in *Proceedings of the 1st ACM Conference on Advances in Financial Technologies*, ser. AFT '19. Association for Computing Machinery, 2019, pp. 16–30.

[22] Z. Smith, H. Jonker, S. Mauw, and H. Lee. (2022) Modelling Agent-Skipping Attacks in Message Forwarding Protocols.[R73]

Appendix A
Lemma Catalogue

[TODO] SJ: This was LLM generated. We need to unsloppify it or remove it all together by submission date. I added it mostly for my own referene. **[END TODO]**

This appendix renders every lemma and query of Table II in natural language and gives its exact Tamarin statement, so that the table can be checked against the theories.

Unless stated otherwise, every lemma[R74] below is *verified* in `multihop_modified.spthy` (the concrete three-hop theory of Sec. V-C). The table's caption says "`multihop.spthy`" because that is the ancestral name of the file; the concrete theory currently contributing these lemmas is `multihop_modified.spthy`, and the two variants of the atomicity/intermediary lemmas named with the suffix `_Liveness` (rows 4 and 8) exist under exactly those names only there. We give line numbers in that file.[S71] We use the same naming convention as the table: A = all-traces safety property, E = exists-trace witness (reachability / non-vacuity).

Throughout, action facts carry their intended meaning[R75] from Sec. III: `InvoiceCreated` — the receiver issued an invoice with a given payment hash and amount; `HTLCOffered` / `HTLCForwarded1` / `HTLCForwarded2` — the corresponding party locked an HTLC (offered or forwarded) with an incoming and an outgoing amount; `HTLCFulfilled` — the receiver redeemed its final HTLC; `SenderSettled` — the sender settled its HTLC with a preimage; `IntermediaryClaimed` — an intermediary claimed its incoming HTLC; `Redeemed` / `Refunded` — the HTLC on a given channel was settled (payment) or timed out (refund); `TimedOut` — a channel timed out; `ForwardHTLC` — an intermediary forwarded an HTLC from an incoming to an outgoing channel on the same hash; `PreimageReleased` / `SecretPreimage` — the receiver released / privately holds a preimage; `FeeCharged` / `FeeEarned` — an intermediary charged / earned a fee on a hash; `ReceiverPaid` — the receiver was credited an amount; `LtkCompromised` — a party's long-term key was compromised; $K(x)$ — the adversary knows the term x.[S72]

A. Invoice and Preimage

Lemma 5 (`Invoice_Authenticates_Settlement`[R76]($\forall$))**.** *The sender can settle an HTLC only if the payment hash corresponds to an invoice the receiver actually issued, at a strictly earlier time; the only way around this is for the receiver's long-term key to have been compromised (strictly earlier as well). In short, a settlement is legitimate only when a genuine invoice backs it; a settlement without such an invoice is attributable to a compromised receiver.*

```
lemma Invoice_Authenticates_Settlement:
  "All S R1 R ptr x #i.
     SenderSettled(S, R1, R, ptr, x) @ #i
   ==> (Ex %v #j. InvoiceCreated(R, S, h(x), %v)
         @ #j & #j < #i)
     | (Ex #r. LtkCompromised(R) @ #r & #r < #i)"[S73]
```

Red updates for page 13

[S71] struck *"Unless stated otherwise, every lemma below ... line numbers in that file."*

describes multihop_modified.spthy (older three-hop lineage); replaced by the red note at "Unless stated otherwise" (csf_final files)

[S72] struck *"Throughout, action facts carry their intended ... adversary knows the term x."*

action-fact names and arities of the older lineage (HTLCForwarded1/2, ForwardHTLC, ReceiverPaid, amounts in InvoiceCreated); replaced by the red glossary note at "Throughout"

[S73] struck *"691 lemma Invoice_Authenticates_Settlement: 692 "All S ... #r. LtkCompromised(R) #r #r #i)""*

old listing (invoice with amount %v); replaced by the csf_final listing in the red note at Lemma 5

[R73] *at "Skipping Attacks in Message Forwarding Protocols"*

TODO: add these two entries, numbered by first citation: [23] is cited in the liveness-scope paragraph added after p. 11, [24] in the rewritten Sec. VII (Related Work):

[23] M. Backes, J. Dreier, S. Kremer, and R. Künnemann, "A Novel Approach for Reasoning about Liveness in Cryptographic Protocols and its Application to Fair Exchange," in IEEE European Symposium on Security and Privacy (EuroS&P), 2017.

[24] A. Kiayias and O. S. T. Litos, "A Composable Security Treatment of the Lightning Network," in *2020 IEEE 33rd Computer Security Foundations Symposium (CSF)*, 2020.

TODO: complete these entries from the publishers' records: [9] has a broken author list ("G. , R. Stefański, …"); [10] lacks the journal; [11] has a garbled page range ("15 pp.–154"); [14] and [22] lack the venue.

[R74] *at "Unless stated otherwise, every lemma"*

Unless stated otherwise, every lemma below is verified in `lightning.spthy`, the main model (channel layer, multi-hop routing in which one generic rule `Forward_HTLC` serves every hop of a route of any length, and the wormhole rule `AttackerSkip`). Lemmas of the auxiliary models are verified in `lightning_aux.spthy`, one model per preprocessor flag (`-D=PTLC`, `-D=HTLC`, `-D=CLOCK`, `-D=CLTV`, `-D=TIMEOUT`). Every listing is copied verbatim from these files with the proof script omitted; listing line numbers refer to `lightning.spthy`. Sections A–I follow groups 1–9 of Table II and Section K group 0; the lemmas not printed below are given at the end of this appendix (additions to Sections A–H, and Sections I–K).

All results were obtained with Tamarin 1.10 and are reproduced by `make all`: `make verify` proves all 55 lemmas of the full model, i.e. with all nine restrictions, including the liveness restrictions T1, T2b and T3, in place (32 all-traces safety lemmas, 13 hop-bypass lemmas, 9 exists-trace lemmas found automatically, and `Wormhole_Fee_Theft_Reachable`, replayed from a saved 116-step proof); `make general` proves the 24 lemmas of the any-length theorem in the model without the three liveness restrictions; `make aux` proves the 21 lemmas of `lightning_aux.spthy`; `make necessity` removes one liveness restriction at a time.

[R75] *at "Throughout, action facts carry their intended meaning"*

Throughout, action facts carry their intended meaning from Sec. III (y = h(x) is a payment hash, ptr a channel): `InvoiceCreated(R, S, y)` — receiver R issued an invoice for y to sender S (the invoice `<'invoice', R, S, y>` is signed by R and carries no amount); `SecretPreimage(R, x)` / `PreimageReleased(R, x)` — R generated / released the preimage x; `HTLCOffered(S, F1, ptr, y)` — the sender offered an HTLC on y to its first forwarder F1 on channel ptr; `OfferFor(S, R, ptr, y)` — annotation of the same step naming the invoice's receiver R; `HTLCAdded(ptr, y)` — an HTLC on y was added to channel ptr (by an offer or a forward); `HTLCForwarded(Me, Prev, Next, ptrIn, ptrOut, y)` — forwarder Me, holding an incoming HTLC from Prev on ptrIn, added an outgoing HTLC to Next on ptrOut (one rule for every hop); `FeeCharged(Me, y, %fee)` — the fee Me charges at that forward; `HTLCFulfilled(R, Prev, ptr, y)` — the receiver redeemed its incoming HTLC; `IntermediaryClaimed(Me, Prev, ptrIn, x)` / `FeeEarned(Me, ptrIn, ptrOut, y, %fee)` — forwarder Me redeemed its incoming HTLC with x / earned its fee; `SenderSettled(S, F1, R, ptr, x)` — the sender settled its HTLC with x; `Redeemed(ptr, y)` / `Refunded(Up, ptr, y)` — the HTLC on ptr was redeemed / refunded, after its timeout, to Up, the party that had offered it; `TimedOut(ptr)` — the HTLC timeout of channel ptr was reached; `SettledHop(ptrIn, ptrOut, y)` / `RefundHop(ptrIn, ptrOut, y)` — annotations: the hop from ptrIn to ptrOut settled / had its outgoing HTLC refunded; `UsedFulfilled(ptr, x)` — annotation: a settlement consumed the fulfilment of the HTLC on ptr; `FulfilledProduced(ptr, x)` — a fulfilment for ptr was produced by an intermediary settlement or by a skip; `Skipped(ptr, x)` — annotation of `AttackerSkip`: compromised parties relayed x onto ptr (wormhole); `ChannelOpen(A, B, ptr)` — A and B opened channel ptr; `KeyGen(A, ltk)` — A generated its long-term key ltk; `LtkCompromised(A)` — A's long-term key was compromised; *K*(x) — the adversary knows the term x.

[R76] *at "Lemma 5 (Invoice_Authenticates_Settlement"*

Description unchanged; the invoice carries no amount.

csf_final (`lightning.spthy`, l. 592):

```
lemma Invoice_Authenticates_Settlement:
    "All S F1 R ptr x #i.
        SenderSettled(S, F1, R, ptr, x) @ #i
     ==> (Ex #j. InvoiceCreated(R, S, h(x)) @ #j & #j < #i)
       | (Ex #r. LtkCompromised(R) @ #r & #r < #i)"
```

Lemma 6 (Preimage_Secret_Until_Released[R77] (∀)). *As long as the receiver holds a secret preimage and has not released it, the adversary does not know it: the first moment the adversary can learn the preimage is strictly after the receiver has released it. Formally, if the receiver generated a secret preimage and the adversary later learns it, then the receiver must have released that preimage at a strictly earlier point in the trace.*

```
lemma Preimage_Secret_Until_Released:
   "All R x #i #j.
      SecretPreimage(R, x) @ #i
    & K(x) @ #j
    & #i < #j
    ==> Ex #k. PreimageReleased(R, x) @ #k & #k <
        #j"
```

Lemma 7 (Forged_Invoice_Requires_Key_Compromise[R78] (∃)). ~~*The sender can settle an HTLC even though the receiver never created the corresponding invoice — but only in traces in which the receiver's long-term key was compromised. This is the attack trace showing that a forged invoice cannot be cashed in without breaking the receiver.*~~

```
lemma Forged_Invoice_Requires_Key_Compromise:
   exists-trace
   "Ex S F1 R ptr x #i #r.
      SenderSettled(S, F1, R, ptr, x) @ #i
    & LtkCompromised(R) @ #r
    & not (Ex %v #j. InvoiceCreated(R, S, h(x),
        %v) @ #j)"[S74]
```

Remark 3. ~~*In the theory this lemma is declared* `exists-trace` *(kind E), whereas Table II groups it with all-traces lemmas (kind A). The intended reading is the reachability one above; if the table is meant to report an all-traces guarantee, that guarantee is in fact already the contrapositive of* `Invoice_Authenticates_Settlement`*. The classification in the table should be corrected to E.*[S75]~~

Lemma 8 (Invoice_Released_Once[R79] (∀)). *The receiver releases a given preimage at most once: if the same receiver releases the same preimage at two times in a trace, those two times coincide, i.e.there is a single release event.* ~~*This holds because the preimage is fresh per invoice, and even if the adversary replays a public* `fulfill` *message it cannot make the receiver release the preimage a second time.*[S76]~~

```
lemma Invoice_Released_Once [reuse]:
   "All R x #i #j.
      PreimageReleased(R, x) @ #i
    & PreimageReleased(R, x) @ #j
    ==> #i = #j"
```

B. Lock Causality[R80]

~~**Lemma 9** (Forward1_Requires_Offer (∀)). *The first intermediary can lock (forward) an HTLC on its outgoing channel only if the sender previously offered it an HTLC with the same payment hash and the same incoming amount on its incoming channel, at a strictly earlier time — unless the sender's key was compromised. In other words, forwarding is causally preceded by an upstream offer.*~~

```
lemma Forward1_Requires_Offer:
   "All F1 F2 S ptrF1F2 y %vIn %vOut #i.
      HTLCForwarded1(F1, F2, S, ptrF1F2, y, %vIn,
          %vOut) @ #i
    ==> (Ex ptrSF1 #j. HTLCOffered(S, F1, ptrSF1,
        y, %vIn) @ #j & #j < #i)
      | (Ex #k. LtkCompromised(S) @ #k & #k < #i)"[S77]
```

~~**Lemma 10** (Forward2_Requires_Forward1 (∀)). *The second intermediary can forward an HTLC to the receiver only if the first intermediary previously forwarded an HTLC with the same hash and with a matching amount, at a strictly earlier time — unless the first intermediary's key was compromised.*~~

```
lemma Forward2_Requires_Forward1:
   "All F2 R F1 ptrF2R y %vIn %vOut #i.
      HTLCForwarded2(F2, R, F1, ptrF2R, y, %vIn,
          %vOut) @ #i
    ==> (Ex S ptrF1F2 %vPrev #j.
          HTLCForwarded1(F1, F2, S, ptrF1F2, y,
              %vPrev, %vIn) @ #j & #j < #i)
      | (Ex #k. LtkCompromised(F1) @ #k & #k < #i)"[S78]
```

Lemma 11 (HTLC_On_Opened_Channel[R81] (∀)). *An HTLC is only ever added on a channel that has been opened beforehand: whenever an HTLC is added on a channel, two parties opened that channel at a strictly earlier time.*

```
lemma HTLC_On_Opened_Channel [reuse,
    use_induction]:
   "All ptr y #i.
      HTLCAdded(ptr, y) @ #i
    ==> Ex X Y #j. ChannelOpen(X, Y, ptr) @ #j &
        #j < #i"
```

C. Release Causality

~~**Lemma 12** (Fulfill_Requires_Forward2 (∀)). *The receiver can fulfill (redeem) an HTLC only if the second intermediary previously forwarded it an HTLC with the same hash and the same amount on the same channel, at a strictly earlier time — unless that intermediary's key was compromised.*~~

```
lemma Fulfill_Requires_Forward2:
   "All R F2 ptrF2R y %v #i.
      HTLCFulfilled(R, F2, ptrF2R, y, %v) @ #i
    ==> (Ex F1 %vIn #j. HTLCForwarded2(F2, R, F1,
        ptrF2R, y, %vIn, %v) @ #j & #j < #i)
      | (Ex #r. LtkCompromised(F2) @ #r & #r < #i)"[S79]
```

Lemma 13 (Claim_Requires_Release[R82] (∀)). *An intermediary can claim its incoming HTLC with a preimage only after (strictly later than) the receiver has released that preimage: the release of the preimage is a prerequisite for any intermediary settlement.*

```
lemma Claim_Requires_Release:
   "All MidP InP ptrIn R x #i #j.
      IntermediaryClaimed(MidP, InP, ptrIn, x) @ #i
    & SecretPreimage(R, x) @ #j
    ==> Ex #k. PreimageReleased(R, x) @ #k & #k <
        #i"
```

Lemma 14 (Settle_Requires_Receiver_Release (∀)). *When the sender settles an HTLC on a genuine invoice, the receiver must have released the corresponding preimage strictly*

Red updates for page 14

[S74] struck *"The sender can settle an HTLC ... InvoiceCreated(R, S, h(x), %v) #j)""*

description overstates an exists-trace lemma; old listing (amount %v); replaced by the red note at Lemma 7

[S75] struck *"Remark 3. In the theory this ... should be corrected to E."*

resolved by the new Table II, which classifies Forged_Invoice_Requires_Key_Compromise as an exists-trace (E) lemma

[S76] struck *"This holds because the preimage is ... the preimage a second time."*

csf_final has no public fulfil message; replaced by the justification in the red note at Lemma 8

[S77] struck *"Lemma 9 (Forward1_Requires_Offer The first inter- ... #k. LtkCompromised(S) #k #k #i)""*

no csf_final counterpart (hop-indexed HTLCForwarded1 with amounts); replaced by Forward_Requires_Incoming, red note at "B. Lock Causality"

[S78] struck *"Lemma 10 (Forward2_Requires_Forward1 The sec- ond ... #k. LtkCompromised(F1) #k #k #i)""*

no csf_final counterpart (HTLCForwarded2, amounts); replaced by Forward_Requires_Incoming, red note at "B. Lock Causality"

[S79] struck *"Lemma 12 (Fulfill_Requires_Forward2 The re- ceiver ... #r. LtkCompromised(F2) #r #r #i)""*

no csf_final counterpart (fixed second intermediary, amounts); csf_final release-causality lemmas are listed in the red insert after p. 17 (additions to C)

[R77] *at "Lemma 6 (Preimage_Secret_Until_Released"*

Statement unchanged; TODO: renumber the listing to `lightning.spthy` l. 542–547. Add: *In this model the preimage is never sent to the network: fulfilments travel between hops as state facts, and `AttackerSkip` relays them without the adversary learning x. Hence `Preimage_Never_Known` (additions to Sec. A) holds, and this lemma holds trivially.*

[R78] *at "Lemma 7 (Forged_Invoice_Requires_Key_Compromise"*

Reachability witness. There is a trace in which the sender settles an HTLC for receiver R although R never created an invoice for h(x); R's long-term key is compromised in this trace, and no forwarding takes place. Together with Lemma 5, which requires such a compromise to precede the settlement, this shows that the compromise case of Lemma 5 is not vacuous.

csf_final (`lightning.spthy`, l. 598):

```
lemma Forged_Invoice_Requires_Key_Compromise:
    exists-trace
    "Ex S F1 R ptr x #i #r.
        SenderSettled(S, F1, R, ptr, x) @ #i
      & LtkCompromised(R) @ #r
      & not (Ex #j. InvoiceCreated(R, S, h(x)) @ #j)
      & not (Ex Me Prev Next pi po y #u. HTLCForwarded(Me, Prev, Next, pi, po, y) @ #u)"
```

[R79] *at "Lemma 8 (Invoice_Released_Once"*

Statement unchanged; TODO: renumber the listing to `lightning.spthy` l. 508–512. New justification: *This holds because `Receiver_Fulfill_HTLC` consumes the linear fact `ReceiverInv(R, S, x)`, which `Receiver_Create_Invoice` creates once per fresh preimage x.*

[R80] *at "B. Lock Causality"*

Lemma (Forward_Requires_Incoming (∀)). *A forwarder adds an outgoing HTLC only if an HTLC on the same hash was added earlier to its incoming channel, either offered to it by a sender or forwarded to it by its predecessor. The lemma covers every hop of a route of any length and needs no honesty assumption.*

Statement (`lightning.spthy`, l. 576):

```
lemma Forward_Requires_Incoming [use_induction]:
    "All Me Prev Next ptrIn ptrOut y #i.
        HTLCForwarded(Me, Prev, Next, ptrIn, ptrOut, y) @ #i
     ==> (Ex S #j. HTLCOffered(S, Me, ptrIn, y) @ #j & #j < #i)
       | (Ex P0 pin0 #j. HTLCForwarded(Prev, P0, Me, pin0, ptrIn, y) @ #j & #j < #i)"
```

[R81] *at "Lemma 11 (HTLC_On_Opened_Channel"*

Statement unchanged; TODO: renumber the listing to `lightning.spthy` l. 554–557. `HTLCAdded` is emitted by `Sender_Offer_HTLC` and `Forward_HTLC`.

[R82] *at "Lemma 13 (Claim_Requires_Release"*

Statement unchanged; TODO: renumber the listing to `lightning.spthy` l. 570–574 and write its first line as `lemma Claim_Requires_Release [use_induction]:.`

earlier: a settlement never precedes the receiver's release of the preimage. [R83]

```
lemma Settle_Requires_Receiver_Release:
    "All S F1 R ptr x %v #i #j.
       SenderSettled(S, F1, R, ptr, x) @ #i
     & InvoiceCreated(R, S, h(x), %v) @ #j
     ==> Ex #k. PreimageReleased(R, x) @ #k & #k < #i"
```
[S80]

D. Atomicity

Lemma 15 (Payment_Atomicity_Under_Liveness (∀)). [R84] *If both intermediaries have forwarded the HTLCs that chain a three-hop payment (locking it end-to-end), the receiver's final HTLC has been redeemed, and no party's long-term key is compromised, then the sender's HTLC can never be refunded: once the payment is locked all the way through and released at the receiver, the sender's money is spent for good. This is the atomicity of the payment under the liveness restrictions.*

```
lemma Payment_Atomicity_Under_Liveness:
    "All F1 F2 ptrSF1 ptrF1F2 ptrF2R y #i #j #k.
       ForwardHTLC(F1, ptrSF1, ptrF1F2, y) @ #i
     & ForwardHTLC(F2, ptrF1F2, ptrF2R, y) @ #j
     & Redeemed(ptrF2R, y) @ #k
     & (not Ex P #l. LtkCompromised(P) @ #l)
     ==> not (Ex #m. Refunded(ptrSF1, y) @ #m)"
```
[S81]

Lemma 16 (Settle_Excludes_Sender_Refund (∀)). [R85] *If the sender has settled an HTLC with a preimage, then that same HTLC can never be refunded later: settlement and refund of the sender's HTLC are mutually exclusive.*

```
lemma Settle_Excludes_Sender_Refund:
    "All S F1 R ptr x #i.
       SenderSettled(S, F1, R, ptr, x) @ #i
     ==> not (Ex #j. HTLCRefunded(S, ptr, h(x)) @ #j)"
```
[S82]

Lemma 17 (Multihop_Payment_Possible (∃)). [R86] *Non-vacuity witness. There exists a complete trace realizing a successful three-hop payment, in order: invoice creation, the sender's offer, both forwards, the receiver fulfilling its HTLC, both intermediaries claiming their incoming HTLCs, and the sender's final settlement. This shows the safety lemmas speak about a protocol that can actually run.*

```
lemma Multihop_Payment_Possible:
    exists-trace
    "Ex S F1 F2 R ptrSF1 ptrF1F2 ptrF2R x %v %vS
        %vMid
       #t1 #t2 #t3 #t4 #t5 #t6 #t7 #t8.
       InvoiceCreated(R, S, h(x), %v) @ #t1
     & HTLCOffered(S, F1, ptrSF1, h(x), %vS) @ #t2
     & HTLCForwarded1(F1, F2, S, ptrF1F2, h(x),
         %vS, %vMid) @ #t3
     & HTLCForwarded2(F2, R, F1, ptrF2R, h(x),
         %vMid, %v) @ #t4
     & HTLCFulfilled(R, F2, ptrF2R, h(x), %v) @ #t5
     & IntermediaryClaimed(F2, F1, ptrF1F2, x) @ #t6
     & IntermediaryClaimed(F1, S, ptrSF1, x) @ #t7
     & SenderSettled(S, F1, R, ptrSF1, x) @ #t8
     & #t1 < #t2 & #t2 < #t3 & #t3 < #t4 & #t4 < #t5
     & #t5 < #t6 & #t6 < #t7 & #t7 < #t8"
```
[S83]

E. Value and Fee [R87]

Lemma 18 (Fee_Conservation_Hop1 (∀)). *At the first forwarding step, the amount the first intermediary receives on its incoming HTLC equals the amount it locks on its outgoing HTLC plus the fee it charges at that hop. Value is conserved per hop.*

```
lemma Fee_Conservation_Hop1:
    "All F1 F2 S p y %vIn %vOut %fee #i.
       HTLCForwarded1(F1, F2, S, p, y, %vIn, %vOut)
         @ #i
     & FeeCharged(F1, y, %fee) @ #i
     ==> %vIn = %vOut %+ %fee"
```

Lemma 19 (Fee_Conservation_Hop2 (∀)). *Same as Fee_Conservation_Hop1, for the second forwarding step: the second intermediary's incoming amount equals its outgoing amount plus the fee it charges there.*

```
lemma Fee_Conservation_Hop2:
    "All F2 R F1 p y %vIn %vOut %fee #i.
       HTLCForwarded2(F2, R, F1, p, y, %vIn, %vOut)
         @ #i
     & FeeCharged(F2, y, %fee) @ #i
     ==> %vIn = %vOut %+ %fee"
```

Lemma 20 (Fee_Strictly_Positive_Hop1 (∀)). *The routed amount strictly decreases at the first forward: the incoming amount is the outgoing amount plus a strictly positive delta, i.e.the intermediary never forwards more value than it receives and always takes a strictly positive fee.*

```
lemma Fee_Strictly_Positive_Hop1:
    "All F1 F2 S p y %vIn %vOut #i.
       HTLCForwarded1(F1, F2, S, p, y, %vIn, %vOut)
         @ #i
     ==> Ex %d. %vIn = %vOut %+ %d"
```

Lemma 21 (Fee_Strictly_Positive_Hop2 (∀)). *Same as Fee_Strictly_Positive_Hop1, for the second forward: the routed amount strictly decreases by a strictly positive delta.*

```
lemma Fee_Strictly_Positive_Hop2:
    "All F2 R F1 p y %vIn %vOut #i.
       HTLCForwarded2(F2, R, F1, p, y, %vIn, %vOut)
         @ #i
     ==> Ex %d. %vIn = %vOut %+ %d"
```

Lemma 22 (EndToEnd_Value_Conservation (∀)). *Across a full linked three-hop payment, the sender locks exactly the receiver's amount plus a (strictly positive) total fee: value is neither created nor destroyed along the whole path.*

```
lemma EndToEnd_Value_Conservation:
    "All S F1 F2 R pS pF pR y %vS %vM %v #o #f1 #f2.
       HTLCOffered(S, F1, pS, y, %vS) @ #o
     & HTLCForwarded1(F1, F2, S, pF, y, %vS, %vM) @
         #f1
     & HTLCForwarded2(F2, R, F1, pR, y, %vM, %v) @
         #f2
     ==> Ex %d. %vS = %v %+ %d"
```

Lemma 23 (Receiver_Paid_Invoice_Amount (∀)). *The receiver is paid exactly the amount it invoiced: whenever the receiver is credited for a payment hash, that hash corresponds to an invoice it created, with precisely this amount, at an* [S84]

Red updates for page 15

[S80] struck *"629 lemma Settle_Requires_Receiver_Release: 630 "All S ... PreimageReleased(R, x) #k #k #i""*

old listing (invoice with amount %v); replaced by the red note at Lemma 14

[S81] struck *"If both intermediaries have forwarded the ... (Ex #m. Refunded(ptrSF1, y) #m)""*

description and listing of the fixed three-hop version (ForwardHTLC); replaced by the red note at Lemma 15

[S82] struck *"685 lemma Settle_Excludes_Sender_Refund: 686 "All S ... #j. HTLCRefunded(S, ptr, h(x)) #j)""*

old listing (HTLCRefunded does not exist in csf_final); replaced by the red note at Lemma 16

[S83] struck *"Lemma 17 (Multihop_Payment_Possible Non- vacuity witness. ... #t6 #t6 #t7 #t7 #t8""*

no csf_final counterpart (HTLCForwarded1/2, amounts); replaced by Multihop_Routing_Reachable and Sanity_Atomic_Payment_Completes, red note at Lemma 17

[S84] struck *"Lemma 18 (Fee_Conservation_Hop1 At the first ... precisely this amount, at an"*

Lemmas 18-23 have no csf_final counterpart (amounts are not chained along the path; no HTLCForwarded1/2, no ReceiverPaid); replaced by the red note at "E. Value and Fee"

[R83] *at "a settlement never precedes the receiver's release"*

Lemma 14: description unchanged; the invoice carries no amount.

csf_final (`lightning.spthy`, l. 564):

```
lemma Settle_Requires_Receiver_Release:
    "All S F1 R ptr x #i #j.
        SenderSettled(S, F1, R, ptr, x) @ #i
      & InvoiceCreated(R, S, h(x)) @ #j
     ==> Ex #k. PreimageReleased(R, x) @ #k & #k < #i"
```

[R84] *at "Lemma 15 (Payment_Atomicity_Under_Liveness"*

If two forwarders F1 and F2 forward consecutive HTLCs on the same hash y (the outgoing channel of F1 is the incoming channel of F2), the HTLC on F2's outgoing channel is redeemed, and no party's long-term key is ever compromised, then the HTLC on F1's incoming channel is never refunded: in a run without any key compromise, once the payment is redeemed downstream of two forwarders, the party upstream of them cannot be refunded. The lemma is proved in the full model, i.e. under the liveness restrictions; it also verifies with T1 removed (Table III).

csf_final (`lightning.spthy`, l. 625):

```
lemma Payment_Atomicity_Under_Liveness:
    "All F1 F2 Prev1 Next1 Prev2 Next2 ptrSF1 ptrF1F2 ptrF2R y #i #j #k.
        HTLCForwarded(F1, Prev1, Next1, ptrSF1, ptrF1F2, y) @ #i
      & HTLCForwarded(F2, Prev2, Next2, ptrF1F2, ptrF2R, y) @ #j
      & Redeemed(ptrF2R, y) @ #k
      & (not Ex P #l. LtkCompromised(P) @ #l)
     ==> not (Ex Up #m. Refunded(Up, ptrSF1, y) @ #m)"
```

[R85] *at "Lemma 16 (Settle_Excludes_Sender_Refund"*

TODO: in the description replace "can never be refunded later" by "is never refunded" (the statement excludes a refund at any time).

csf_final (`lightning.spthy`, l. 587):

```
lemma Settle_Excludes_Sender_Refund [heuristic={sender_pending}]:
    "All S F1 R ptr x #i.
        SenderSettled(S, F1, R, ptr, x) @ #i
     ==> not (Ex #j. Refunded(S, ptr, h(x)) @ #j)"
```

[R86] *at "Lemma 17 (Multihop_Payment_Possible"*

Replace Lemma 17 by two non-vacuity witnesses.

Lemma (Multihop_Routing_Reachable (∃)). *The routing phase of a payment over two forwarders is reachable: the sender offers, both forwarders forward and the receiver fulfils, with no other offer or forward, at most one HTLC per channel, and no intermediary claim or sender settlement.*

Statement (`lightning.spthy`, l. 644):

```
lemma Multihop_Routing_Reachable:
    exists-trace
    "Ex S F1 F2 R p0 p1 p2 x #t2 #t3 #t4 #t5.
        HTLCOffered(S, F1, p0, h(x)) @ #t2
```

```
    & HTLCForwarded(F1, S, F2, p0, p1, h(x)) @ #t3
    & HTLCForwarded(F2, F1, R, p1, p2, h(x)) @ #t4
    & HTLCFulfilled(R, F2, p2, h(x)) @ #t5
    & #t2 < #t3 & #t3 < #t4 & #t4 < #t5
    & not (Ex A B p y #u. HTLCOffered(A, B, p, y) @ #u & not (A = S & B = F1))
    & not (Ex Me Prev Next pi po y #u. HTLCForwarded(Me, Prev, Next, pi, po, y) @ #u
            & not (Me = F1 & Prev = S & Next = F2)
            & not (Me = F2 & Prev = F1 & Next = R))
    & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m & HTLCAdded(ptr, y2) @ #n & not (#m = #n))
    & not (Ex Me P q z #e. IntermediaryClaimed(Me, P, q, z) @ #e)
    & not (Ex A B C q z #e. SenderSettled(A, B, C, q, z) @ #e)"
```

Lemma (Sanity_Atomic_Payment_Completes (∃)). *An honest payment over two forwarders completes end to end: offer, both forwards, fulfilment, both intermediary claims and the sender's settlement occur, with no key compromise, at most one HTLC per channel, exactly these claims and this settlement, and no refund. It shows that the premise of Lemma 15 is satisfiable.*

Statement (`lightning.spthy`, l. 1123):

```
lemma Sanity_Atomic_Payment_Completes:
  exists-trace
  "Ex S F1 F2 R p0 p1 p2 x #o #a #b #r #c1 #c2 #s.
      HTLCOffered(S, F1, p0, h(x)) @ #o
    & HTLCForwarded(F1, S, F2, p0, p1, h(x)) @ #a
    & HTLCForwarded(F2, F1, R, p1, p2, h(x)) @ #b
    & HTLCFulfilled(R, F2, p2, h(x)) @ #r
    & IntermediaryClaimed(F2, F1, p1, x) @ #c2
    & IntermediaryClaimed(F1, S, p0, x) @ #c1
    & SenderSettled(S, F1, R, p0, x) @ #s
    & not (Ex P #k. LtkCompromised(P) @ #k)
    & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m & HTLCAdded(ptr, y2) @ #n & not (#m = #n))
    & (All Me P q z #e. IntermediaryClaimed(Me, P, q, z) @ #e ==> (#e = #c1 | #e = #c2))
    & (All A B C q z #e. SenderSettled(A, B, C, q, z) @ #e ==> #e = #s)
    & not (Ex U q z #e. Refunded(U, q, z) @ #e)"
```

[R87] *at "E. Value and Fee"*

TODO: retitle the subsection "E. Fees" and replace Lemmas 18–24 by the following text.

Amounts are local to a hop: `Forward_HTLC` receives the incoming amount, the fee and the outgoing amount as inputs and emits `Eq(%vIn, %vOut %+ %fee)`, which the `Equality` restriction enforces, so per-hop value conservation holds by construction. The offer, the invoice and the route carry no amount, so amounts are not chained along the path and the model makes no end-to-end value statement. The fee guarantee is `Fee_Only_On_Successful_Forward` (Additions to E, end of this appendix): an uncompromised forwarder earns its fee only after its outgoing HTLC was redeemed.

earlier time.

```
lemma Receiver_Paid_Invoice_Amount:
    "All R y %v #i.
       ReceiverPaid(R, y, %v) @ #i
     ==> Ex S #j. InvoiceCreated(R, S, y, %v) @ #j
           & #j < #i"
```

Lemma 24 (Fees_Charged_On_Path_Possible (∃)). *Non-vacuity witness. There exists a trace in which two* distinct *intermediaries each charge a fee on a payment with the same hash, showing that the value/fee lemmas apply to a genuine multi-hop path (and not only to a single hop).*

```
lemma Fees_Charged_On_Path_Possible:
    exists-trace
    "Ex F1 F2 y %f1 %f2 #i #j.
       FeeCharged(F1, y, %f1) @ #i
     & FeeCharged(F2, y, %f2) @ #j
     & not (F1 = F2)"
```
[S85]

F. Refund and Griefing [R88]

Lemma 25 (Refund_Requires_Timeout (∀)) [R89]. *Every refund of an HTLC is preceded by the timeout of its channel: an HTLC is refunded only after that channel has timed out.*

```
lemma Refund_Requires_Timeout [reuse]:
    "All ptr y #i.
       Refunded(ptr, y) @ #i
     ==> Ex #j. TimedOut(ptr) @ #j & #j < #i"
```
[S86]

Lemma 26 (Loss_Requires_Inaction (∀)) [R90]. *If a forwarder's outgoing HTLC is redeemed while its incoming HTLC is refunded, then its incoming HTLC is never redeemed afterwards. An intermediary therefore loses funds only through its own inaction — failing to redeem its incoming HTLC before it times out — and never because the same HTLC was both redeemed and refunded (the two outcomes are exclusive).*

```
lemma Loss_Requires_Inaction:
    "All P ptrIn ptrOut y #i #j #k.
       ForwardHTLC(P, ptrIn, ptrOut, y) @ #i
     & Redeemed(ptrOut, y) @ #j
     & Refunded(ptrIn, y) @ #k
     ==> not (Ex #l. Redeemed(ptrIn, y) @ #l)"
```
[S87]

Lemma 27 (Honest_Intermediary_Refunds (∃)). *Reachability witness. There exists a trace in which an honest intermediary (no key compromise) forwards an HTLC and its outgoing HTLC is subsequently refunded on timeout, with no settlement anywhere: an honest intermediary's capital can be locked up and merely returned without a payment, which is exactly the griefing denial-of-service scenario. The funds remain refundable, so safety still holds.*

```
lemma Honest_Intermediary_Refunds:
    exists-trace
    "Ex F ptrIn ptrOut y #i #j.
       ForwardHTLC(F, ptrIn, ptrOut, y) @ #i
     & Refunded(ptrOut, y) @ #j
     & not (Ex a b rr p x #k. SenderSettled(a, b,
         rr, p, x) @ #k)
     & not (Ex P #l. LtkCompromised(P) @ #l)"
```
[S88]

G. Wormhole Fee Theft

Lemma 28 (Wormhole_Fee_Theft_Reachable (∃)) [R91]. *Reachability witness (attack). There exists a complete three-hop payment in which the payment settles and the receiver is paid, yet the middle hop's HTLC is never redeemed and neither intermediary ever earns a fee, with no key compromise. This is the wormhole: an honest intermediary forwards value but collects nothing, because the preimage is relayed around it by two colluding endpoints.*

```
lemma Wormhole_Fee_Theft_Reachable:
    exists-trace
    "Ex S F1 F2 R ptrSF1 ptrF1F2 ptrF2R x %v %vS
       %vMid
       #t1 #t2 #t3 #t4 #t5 #t6.
       InvoiceCreated(R, S, h(x), %v) @ #t1
     & HTLCOffered(S, F1, ptrSF1, h(x), %vS) @ #t2
     & HTLCForwarded1(F1, F2, S, ptrF1F2, h(x),
         %vS, %vMid) @ #t3
     & HTLCForwarded2(F2, R, F1, ptrF2R, h(x),
         %vMid, %v) @ #t4
     & HTLCFulfilled(R, F2, ptrF2R, h(x), %v) @ #t5
     & SenderSettled(S, F1, R, ptrSF1, x) @ #t6
     & #t1 < #t2 & #t2 < #t3 & #t3 < #t4 & #t4 <
         #t5 & #t5 < #t6
     & not (Ex #i. Redeemed(ptrF1F2, h(x)) @ #i)
     & not (Ex p1 p2 yy %f #j. FeeEarned(F1, p1,
         p2, yy, %f) @ #j)
     & not (Ex p1 p2 yy %f #k. FeeEarned(F2, p1,
         p2, yy, %f) @ #k)
     & not (Ex P #l. LtkCompromised(P) @ #l)"
```
[S89]

Lemma 29 (Wormhole_Steals_Exactly_The_Fees (∃)). *The same reachable wormhole, quantified. The sender locks exactly the receiver's amount plus the sum of the two path fees, both fees are charged, and yet neither intermediary ever earns them: the value stolen from the honest intermediary is exactly the sum of the path fees — not merely some positive loss.*

```
lemma Wormhole_Steals_Exactly_The_Fees:
    exists-trace
    "Ex S F1 F2 R ptrSF1 ptrF1F2 ptrF2R x %v %vS
       %vMid %f1 %f2
       #t1 #t2 #t3 #t4 #t5 #t6 #c1 #c2.
       InvoiceCreated(R, S, h(x), %v) @ #t1
     & HTLCOffered(S, F1, ptrSF1, h(x), %vS) @ #t2
     & HTLCForwarded1(F1, F2, S, ptrF1F2, h(x),
         %vS, %vMid) @ #t3
     & HTLCForwarded2(F2, R, F1, ptrF2R, h(x),
         %vMid, %v) @ #t4
     & HTLCFulfilled(R, F2, ptrF2R, h(x), %v) @ #t5
     & SenderSettled(S, F1, R, ptrSF1, x) @ #t6
     & FeeCharged(F1, h(x), %f1) @ #c1
     & FeeCharged(F2, h(x), %f2) @ #c2
     & %vS = %v %+ %f1 %+ %f2
     & #t2 < #t3 & #t3 < #t4 & #t4 < #t5 & #t5 < #t6
     & not (Ex p1 p2 yy %f #i. FeeEarned(F1, p1,
         p2, yy, %f) @ #i)
     & not (Ex p1 p2 yy %f #j. FeeEarned(F2, p1,
         p2, yy, %f) @ #j)
     & not (Ex P #k. LtkCompromised(P) @ #k)"
```
[S90]

H. Intermediary Safety

Lemma 30 (Intermediary_Never_Loses_Under_Liveness (∀)). *If an intermediary's outgoing HTLC is redeemed while its incoming HTLC is refunded, then its long-term key was compromised. Equivalently, an honest (non-compromised)* [S91]

Red updates for page 16

[S85] struck *"earlier time. 764 lemma Receiver_Paid_Invoice_Amount: 765 ... #j 774 not (F1 F2)""*

rest of Lemma 23 and Lemma 24: no csf_final counterpart (no ReceiverPaid; Fees_Charged_On_Path_Possible not in csf_final); see the red note at "E. Value and Fee" on p. 15

[S86] struck *"714 lemma Refund_Requires_Timeout [reuse]: 715 "All ... #j. TimedOut(ptr) #j #j #i""*

old listing (Refunded without the refunded party); replaced by the red note at Lemma 25

[S87] struck *"706 lemma Loss_Requires_Inaction: 707 "All P ... (Ex #l. Redeemed(ptrIn, y) #l)""*

old listing (ForwardHTLC); replaced by the red note at Lemma 26

[S88] struck *"Lemma 27 (Honest_Intermediary_Refunds Reach- ability witness. ... (Ex P #l. LtkCompromised(P) #l)""*

no csf_final counterpart; replaced by Refund_Possible (a different, weaker witness), red note at "F. Refund and Griefing"

[S89] struck *"Reach- ability witness (attack). There exists ... (Ex P #l. LtkCompromised(P) #l)""*

describes the 3-hop, no-compromise trace of the older lineage; replaced by the 5-party csf_final witness in the red note at Lemma 28

[S90] struck *"Lemma 29 (Wormhole_Steals_Exactly_The_Fees The same reachable ... (Ex P #k. LtkCompromised(P) #k)""*

no csf_final counterpart: amounts are not chained along the path, so the loss cannot be quantified as the path fees

[S91] struck *"Equivalently, an honest (non-compromised)"*

start of a sentence that overstates the lemma (continues on p. 17); replaced by the red note at Lemma 30 on p. 17

[R88] *at "F. Refund and Griefing"*

TODO: keep the title “F. Refund and Griefing” (group 6 of the new Table II, “Refund / griefing”). `lightning.spthy` has no counterpart of Lemma 27 (an honest forwarder whose outgoing HTLC is refunded with no settlement anywhere). The refund witness of the model is the weaker Refund_Possible, which Sec. V-E reports as reachable griefing of the sender: its HTLC stays locked until the timeout (Refund_Requires_Timeout):

Lemma (Refund_Possible (∃)). *A payment can be refunded: the receiver issues an invoice, the sender offers an HTLC on its hash, and the sender’s HTLC is refunded after its timeout, with no settlement, no key compromise, no forwarding, and at most one HTLC per channel.*

Statement (`lightning.spthy`, l. 530):

```
lemma Refund_Possible:
    exists-trace
    "Ex S F1 R p y #i #j #k.
        InvoiceCreated(R, S, y) @ #j
      & HTLCOffered(S, F1, p, y) @ #k
      & Refunded(S, p, y) @ #i
      & #j < #k & #k < #i
      & not (Ex A MidP B p2 x #l. SenderSettled(A, MidP, B, p2, x) @ #l)
      & not (Ex P #r. LtkCompromised(P) @ #r)
      & not (Ex Me Prev Next pi po y2 #t1. HTLCForwarded(Me, Prev, Next, pi, po, y2) @ #t1)
      & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m & HTLCAdded(ptr, y2) @ #n & not (#m = #n))"
```

[R89] *at "Lemma 25 (Refund_Requires_Timeout"*

Description unchanged.

csf_final (`lightning.spthy`, l. 613):

```
lemma Refund_Requires_Timeout [reuse]:
    "All Up ptr y #i.
        Refunded(Up, ptr, y) @ #i
     ==> Ex #j. TimedOut(ptr) @ #j & #j < #i"
```

[R90] *at "Lemma 26 (Loss_Requires_Inaction"*

TODO: in the description replace “is never redeemed afterwards” by “is never redeemed” (the statement excludes a redemption at any time).

csf_final (`lightning.spthy`, l. 606):

```
lemma Loss_Requires_Inaction:
    "All P Prev Next Up ptrIn ptrOut y #i #j #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #i
      & Redeemed(ptrOut, y) @ #j
      & Refunded(Up, ptrIn, y) @ #k
```

```
    ==> not (Ex #l. Redeemed(ptrIn, y) @ #l)"
```

[R91] *at "Lemma 28 (Wormhole_Fee_Theft_Reachable"*

Reachability witness (attack). There is a trace on the route Alice → Att1 → Indiana → Att2 → Bob in which Bob issues the invoice and redeems his incoming HTLC and Alice settles her HTLC, yet the outgoing HTLC of the intermediary Indiana is refunded and Indiana earns no fee. Alice, Bob and Indiana are never compromised, and Att1 and Att2 are distinct parties. This is the wormhole [6]: the preimage is relayed around Indiana by rule `AttackerSkip`, which only two compromised parties on the same payment can apply. Tamarin does not find this trace automatically: `make verify` replays a saved 116-step proof. In the PTLC model of Section J the corresponding wormhole around an honest hop is unreachable (`No_Wormhole_Around_Honest_Hop`), while the HTLC baseline reaches it (`Wormhole_Around_Honest_Hop`).

csf_final (`lightning.spthy`, l. 660):

```
lemma Wormhole_Fee_Theft_Reachable:
  exists-trace
  "∃ Alice Bob Att1 Att2 Indiana ptrAliceAtt1 ptrAtt1Indiana
     ptrIndianaAtt2 ptrAtt2Bob x #t1 #t2 #t3 #t4 #t5 #t6 #t7 #t8.
    ((((((((((((((InvoiceCreated( Bob, Alice, h(x) ) @ #t1) ∧
                  (HTLCOffered( Alice, Att1, ptrAliceAtt1, h(x) ) @ #t2)) ∧
                 (HTLCForwarded( Att1, Alice, Indiana, ptrAliceAtt1, ptrAtt1Indiana,
                                 h(x)
                  ) @ #t3)) ∧
                (HTLCForwarded( Indiana, Att1, Att2, ptrAtt1Indiana,
                                ptrIndianaAtt2, h(x)
                 ) @ #t4)) ∧
               (HTLCForwarded( Att2, Indiana, Bob, ptrIndianaAtt2, ptrAtt2Bob,
                               h(x)
                ) @ #t5)) ∧
              (HTLCFulfilled( Bob, Att2, ptrAtt2Bob, h(x) ) @ #t6)) ∧
             (SenderSettled( Alice, Att1, Bob, ptrAliceAtt1, x ) @ #t7)) ∧
            (Refunded( Indiana, ptrIndianaAtt2, h(x) ) @ #t8)) ∧
           (¬(Indiana = Alice))) ∧
          (¬(Indiana = Bob))) ∧
         (¬(Att1 = Att2))) ∧
        (¬(∃ #c. LtkCompromised( Alice ) @ #c))) ∧
       (¬(∃ #c. LtkCompromised( Bob ) @ #c))) ∧
      (¬(∃ #c. LtkCompromised( Indiana ) @ #c))) ∧
     (¬(∃ ptr1 ptr2 yy %fee #k.
         FeeEarned( Indiana, ptr1, ptr2, yy, %fee ) @ #k))) ∧
    (¬(∃ ptr y1 y2 #m #n.
        ((HTLCAdded( ptr, y1 ) @ #m) ∧ (HTLCAdded( ptr, y2 ) @ #n)) ∧
        (¬(#m = #n))))"
```

intermediary never loses funds: whenever its outgoing HTLC is redeemed, under the liveness restrictions it also redeems its incoming HTLC.[S92]

```
lemma Intermediary_Never_Loses_Under_Liveness:[R92]
  "All P ptrIn ptrOut y #f #i #j.
     ForwardHTLC(P, ptrIn, ptrOut, y) @ #f
   & Redeemed(ptrOut, y) @ #i
   & Refunded(ptrIn, y) @ #j
  ==> Ex #r. LtkCompromised(P) @ #r"[S93]
```

Lemma 31 (`Timeout_Race_Blocked`[R93]**).** *There is no trace in which an intermediary's incoming HTLC times out strictly before its outgoing HTLC is redeemed. The early-timeout race — an incoming refund happening while the outgoing HTLC can still be settled — is blocked by the timing restrictions (in particular `T2b`), in the* honest *model.*[S94]

```
lemma Timeout_Race_Blocked:
  "not (Ex P ptrIn ptrOut y #l #i #j #k.
     ForwardHTLC(P, ptrIn, ptrOut, y) @ #l
   & TimedOut(ptrIn) @ #k
   & Refunded(ptrIn, y) @ #j
   & Redeemed(ptrOut, y) @ #i
   & #k < #i)"[S95]
```

Red updates for page 17

[S92] struck *"intermediary never loses funds: whenever its ... also redeems its incoming HTLC."*

overstates the lemma (it does not force a redemption); replaced by the red note at Lemma 30

[S93] struck *"721 lemma Intermediary_Never_Loses_Under_Liveness: 722 "All P ... 726 Ex #r. LtkCompromised(P) #r""*

old listing (ForwardHTLC, Refunded without the refunded party); replaced by the red note at Lemma 30

[S94] struck *"There is no trace in which ... T2b), in the honest model."*

wrong for csf_final: the lemma verifies with T2b removed (necessity runs); replaced by the red note at Lemma 31

[S95] struck *"740 lemma Timeout_Race_Blocked: 741 "not (Ex ... y) #i 746 #k #i)""*

old listing (ForwardHTLC, no compromise guard); replaced by the red note at Lemma 31

[R92] *at "721 lemma Intermediary_Never_Loses_Under_Liveness"*

Lemma 30, replacement for the struck sentence: *Equivalently, an honest intermediary whose outgoing HTLC is redeemed never has its incoming HTLC refunded. The stronger `Intermediary_Never_Loses_Strong` (Additions to H) places the compromise before the incoming timeout.*

csf_final (`lightning.spthy`, l. 618):

```
lemma Intermediary_Never_Loses_Under_Liveness:
    "All P Prev Next Up ptrIn ptrOut y #i #j #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #i
      & Redeemed(ptrOut, y) @ #j
      & Refunded(Up, ptrIn, y) @ #k
     ==> Ex #l. LtkCompromised(P) @ #l"
```

[R93] *at "Lemma 31 (Timeout_Race_Blocked"*

No intermediary that is not compromised before its incoming timeout has its incoming HTLC refunded when that timeout happens strictly before its outgoing HTLC is redeemed. The lemma also verifies with any one of T1, T2b and T3 removed (4 steps each, Table III), so it does not depend on any single liveness restriction. In the abstract TIMEOUT model of `lightning_aux.spthy`, where either channel may time out at any point and the outgoing HTLC may be redeemed at any time, the race is reachable (`Early_Timeout_Race`, ∃, 6 steps).

csf_final (`lightning.spthy`, l. 633):

```
lemma Timeout_Race_Blocked:
    "not (Ex P Prev Next Up ptrIn ptrOut y #l #i #j #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #l
      & TimedOut(ptrIn)     @ #k
      & Refunded(Up, ptrIn, y)  @ #j
      & Redeemed(ptrOut, y) @ #i
      & #k < #i
      & not (Ex #c. LtkCompromised(P) @ #c & #c < #k))"
```

Appendix A additions: csf_final lemmas missing from the catalogue

The following lemmas complete the catalogue; the section letters follow the groups of Table II. Statements are verbatim (proof scripts omitted); ∀ = all-traces, ∃ = exists-trace. Unless a flag is given they belong to `lightning.spthy` and are proved by `make verify`.

Additions to A (Invoice and Preimage)

Preimage_Never_Known (∀) [`lightning.spthy`, l. 1141]. *The adversary never learns a receiver's preimage (x is never output in this model); Lemma 6 therefore holds trivially.*

```
lemma Preimage_Never_Known:
    "All R x #i. SecretPreimage(R, x) @ #i ==> not (Ex #j. K(x) @ #j)"
```

Invoice_Has_Secret_Preimage (∀) [`lightning.spthy`, l. 549]. *Every invoice is created together with the receiver's secret preimage of its hash.*

```
lemma Invoice_Has_Secret_Preimage [reuse]:
    "All R S x #i.
        InvoiceCreated(R, S, h(x)) @ #i
     ==> SecretPreimage(R, x) @ #i"
```

Additions to B (Lock Causality)

Offer_Requires_Invoice (∀) [`lightning.spthy`, l. 1115]. *An HTLC is offered for receiver R only against an invoice that R created earlier, or R's long-term key is compromised.*

```
lemma Offer_Requires_Invoice:
```

```
    "All S R ptr y #o.
        OfferFor(S, R, ptr, y) @ #o
     ==> (Ex #j. InvoiceCreated(R, S, y) @ #j & #j < #o)
       | (Ex #r. LtkCompromised(R) @ #r)"
```

Additions to C (Release Causality)

Fulfilled_Requires_Release (∀) [lightning.spthy, l. 559]. *Every fulfilment produced by an intermediary settlement or a skip is preceded by a receiver's release of x.*

```
lemma Fulfilled_Requires_Release [reuse, use_induction, heuristic=S]:
    "All ptr x #i.
        FulfilledProduced(ptr, x) @ #i
     ==> Ex R #k. PreimageReleased(R, x) @ #k & #k < #i"
```

Redeem_Requires_Receiver_Release (∀) [lightning.spthy, l. 582]. *Every redeemed HTLC on y comes with a receiver's release of a preimage x with h(x) = y.*

```
lemma Redeem_Requires_Receiver_Release [use_induction]:
    "All ptr y #i.
        Redeemed(ptr, y) @ #i
     ==> Ex R x #k. PreimageReleased(R, x) @ #k & h(x) = y"
```

Fulfilled_Source (∀) [lightning.spthy, l. 1148]. *A fulfilment consumed by a settlement stems from an earlier redemption of that HTLC or from an earlier skip.*

```
lemma Fulfilled_Source [reuse]:
    "All p x #i. UsedFulfilled(p, x) @ #i
     ==> (Ex #j. Redeemed(p, h(x)) @ #j & #j < #i)
       | (Ex #j. Skipped(p, x) @ #j & #j < #i)"
```

Additions to D (Atomicity)

OneOutcomePerHTLC (∀) [lightning.spthy, l. 498]. *No HTLC is both redeemed and refunded.*

```
lemma OneOutcomePerHTLC [reuse]:
    "All Up ptr y #i #j.
       Redeemed(ptr, y) @ #i & Refunded(Up, ptr, y) @ #j ==> F"
```

Sanity_Honest_Payment_Completes (∃) [lightning.spthy, l. 1075]. *An honest payment over one forwarder completes end to end, with no compromise and no refund.*

```
lemma Sanity_Honest_Payment_Completes:
  exists-trace
  "Ex S F1 R p0 p1 x #o #a #r #c #s.
      HTLCOffered(S, F1, p0, h(x)) @ #o
    & HTLCForwarded(F1, S, R, p0, p1, h(x)) @ #a
    & HTLCFulfilled(R, F1, p1, h(x)) @ #r
    & IntermediaryClaimed(F1, S, p0, x) @ #c
    & SenderSettled(S, F1, R, p0, x) @ #s
    & not (Ex P #k. LtkCompromised(P) @ #k)
    & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m & HTLCAdded(ptr, y2) @ #n & not (#m = #n))
    & (All Me P q z #e. IntermediaryClaimed(Me, P, q, z) @ #e ==> #e = #c)
    & (All A B C q z #e. SenderSettled(A, B, C, q, z) @ #e ==> #e = #s)
    & not (Ex U q z #e. Refunded(U, q, z) @ #e)"
```

Distinct_Parties_Configuration (∃) [lightning.spthy, l. 514]. *Three channels among four distinct parties can be opened with no compromise, update, publication, close, cheat or forward.*

```
lemma Distinct_Parties_Configuration:
    exists-trace
    "Ex S F1 F2 R ptrSF1 ptrF1F2 ptrF2R #t1 #t2 #t3.
        ChannelOpen(S, F1, ptrSF1) @ #t1
      & ChannelOpen(F1, F2, ptrF1F2) @ #t2
      & ChannelOpen(F2, R, ptrF2R) @ #t3
      & not (S = F1) & not (S = F2) & not (S = R)
      & not (F1 = F2) & not (F1 = R) & not (F2 = R)
      & not (Ex P #i. LtkCompromised(P) @ #i)
      & not (Ex P Q p n #j. StateUpdate(P, Q, p, n) @ #j)
      & not (Ex P n #k. PublishState(P, n) @ #k)
      & not (Ex P Q p n #l. Close(P, Q, p, n) @ #l)
      & not (Ex P n #m. Cheat(P, n) @ #m)
      & not (Ex Me Prev Next pi po y #n. HTLCForwarded(Me, Prev, Next, pi, po, y) @ #n)
      & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m & HTLCAdded(ptr, y2) @ #n & not (#m = #n))"
```

Additions to E (Fees)

Fee_Only_On_Successful_Forward (∀) [lightning.spthy, l. 1066]. *An uncompromised forwarder earns its fee only after its outgoing HTLC was redeemed and after it forwarded on that hop.*

```
lemma Fee_Only_On_Successful_Forward [heuristic={route_info_first}]:
    "All Me ptrIn ptrOut y %fee #i.
        FeeEarned(Me, ptrIn, ptrOut, y, %fee) @ #i
```

```
      & not (Ex #c. LtkCompromised(Me) @ #c)
    ==> (Ex #j. Redeemed(ptrOut, y) @ #j & #j < #i)
      & (Ex Prev Next #f. HTLCForwarded(Me, Prev, Next, ptrIn, ptrOut, y) @ #f & #f < #i)"
```

Additions to G (Wormhole)

Skipped_Requires_Compromise (∀) [lightning.spthy, l. 1154]. *Every wormhole skip happens after some compromise.*

```
lemma Skipped_Requires_Compromise [reuse]:
    "All p x #i. Skipped(p, x) @ #i ==> Ex P #c. LtkCompromised(P) @ #c & #c < #i"
```

Additions to H (Intermediary Safety, Timing)

ForwardTimeGap (∀) [lightning.spthy, l. 426]. *If both channels of a forward time out and the forwarder is not compromised before its incoming timeout, the outgoing HTLC times out strictly before the incoming one. T2a is not a restriction in this model: it is this lemma, derived from T2b. Under -D=NECESSITY it is declared without [reuse].*

```
lemma ForwardTimeGap [reuse]:
    "All P Prev Next ptrIn ptrOut y #f #j #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #f
      & TimedOut(ptrOut) @ #j
      & TimedOut(ptrIn)  @ #k
      & not (Ex #c. LtkCompromised(P) @ #c & #c < #k)
     ==> #j < #k"
```

Intermediary_Never_Loses_Strong (∀) [lightning.spthy, l. 1106]. *A forwarder whose outgoing HTLC is redeemed while its incoming HTLC is refunded was compromised before its incoming HTLC timed out.*

```
lemma Intermediary_Never_Loses_Strong:
    "All P Prev Next Up ptrIn ptrOut y #i #j #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #i
      & Redeemed(ptrOut, y) @ #j
      & Refunded(Up, ptrIn, y) @ #k
     ==> Ex #l #t. LtkCompromised(P) @ #l & TimedOut(ptrIn) @ #t & #l < #t"
```

Sanity_Compromised_Intermediary_Can_Lose (∃) [lightning.spthy, l. 1091]. *Non-vacuity of Lemma 30: a compromised forwarder can lose (its outgoing HTLC is fulfilled while its incoming HTLC is refunded).*

```
lemma Sanity_Compromised_Intermediary_Can_Lose:
  exists-trace
  "Ex S F1 R p0 p1 x #o #a #r #u #k.
      HTLCOffered(S, F1, p0, h(x)) @ #o
    & HTLCForwarded(F1, S, R, p0, p1, h(x)) @ #a
    & HTLCFulfilled(R, F1, p1, h(x)) @ #r
    & Refunded(S, p0, h(x)) @ #u
    & LtkCompromised(F1) @ #k
    & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m & HTLCAdded(ptr, y2) @ #n & not (#m = #n))
    & not (Ex Me P q z #e. IntermediaryClaimed(Me, P, q, z) @ #e)
    & not (Ex A B C q z #e. SenderSettled(A, B, C, q, z) @ #e)"
```

Necessity_IMC_Honest_Intermediary_Loses (∃) — `-D=NO_IMC -D=NECESSITY`, `make necessity`, 28 steps [lightning.spthy, l. 1847]. *Without T3 an honest forwarder loses: its outgoing HTLC is fulfilled by the receiver while its incoming HTLC is refunded to the sender, with no key compromise, no claim and no settlement. Hence T3 is necessary for intermediary safety.*

```
lemma Necessity_IMC_Honest_Intermediary_Loses:
  exists-trace
  "Ex S F1 R p0 p1 x #o #a #r #u.
      HTLCOffered(S, F1, p0, h(x)) @ #o
    & HTLCForwarded(F1, S, R, p0, p1, h(x)) @ #a
    & HTLCFulfilled(R, F1, p1, h(x)) @ #r
    & Refunded(S, p0, h(x)) @ #u
    & not (Ex P #k. LtkCompromised(P) @ #k)
    & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m & HTLCAdded(ptr, y2) @ #n & not (#m = #n))
    & not (Ex Me P q z #e. IntermediaryClaimed(Me, P, q, z) @ #e)
    & not (Ex A B C q z #e. SenderSettled(A, B, C, q, z) @ #e)"
```

TODO: `Necessity_OTF_Timeout_Order_Broken` (`-D=NO_OTF`) is in the file but is not part of `make necessity` and has no result; report nothing for it.

I. Hop Bypass: Skipping a Forwarder Requires a Compromise

If the payment settles at the sender while a forwarder's own hop is refunded, some party's key was compromised. The conclusion does not say which party, or when. The lemmas for positions 1–6 need no single-offer premise; `Any_Hop_Skip_Requires_Compromise` covers any position on a route of any length under a single-offer premise (every offer on h(x) uses the sender's channel pSF).

First_Hop_Skip_Requires_Compromise (∀) [lightning.spthy, l. 1208]. *The first forwarder cannot be bypassed without a compromise.*

```
lemma First_Hop_Skip_Requires_Compromise [heuristic={lemmas_first}]:
```

```
    "All S F1 R pSF Next pOut x #s #f #r.
        SenderSettled(S, F1, R, pSF, x) @ #s
      & HTLCForwarded(F1, S, Next, pSF, pOut, h(x)) @ #f
      & RefundHop(pSF, pOut, h(x)) @ #r
     ==> Ex P #c. LtkCompromised(P) @ #c"
```

Second_Hop_Skip_Requires_Compromise (∀) [lightning.spthy, l. 1221]. *The second forwarder (Indiana's position in Lemma 28) cannot be bypassed without a compromise.*

```
lemma Second_Hop_Skip_Requires_Compromise [heuristic={lemmas_first}]:
    "All S F1 R pSF x F2 p1 Next pOut #s #f1 #f2 #r.
        SenderSettled(S, F1, R, pSF, x) @ #s
      & HTLCForwarded(F1, S, F2, pSF, p1, h(x)) @ #f1
      & HTLCForwarded(F2, F1, Next, p1, pOut, h(x)) @ #f2
      & RefundHop(p1, pOut, h(x)) @ #r
     ==> Ex P #c. LtkCompromised(P) @ #c"
```

Hop3_Skip_Requires_Compromise (∀) [lightning.spthy, l. 1231]. *The forwarder at position 3 cannot be bypassed without a compromise.*

```
lemma Hop3_Skip_Requires_Compromise [heuristic={lemmas_first}]:
    "All S R x pSF pOut Next F1 F2 F3 p1 p2 #s #r #f1 #f2 #f3.
        SenderSettled(S, F1, R, pSF, x) @ #s
      & HTLCForwarded(F1, S, F2, pSF, p1, h(x)) @ #f1
      & HTLCForwarded(F2, F1, F3, p1, p2, h(x)) @ #f2
      & HTLCForwarded(F3, F2, Next, p2, pOut, h(x)) @ #f3
      & RefundHop(p2, pOut, h(x)) @ #r
     ==> Ex P #c. LtkCompromised(P) @ #c"
```

Hop4_Skip_Requires_Compromise (∀) [lightning.spthy, l. 1242]. *The forwarder at position 4 cannot be bypassed without a compromise.*

```
lemma Hop4_Skip_Requires_Compromise [heuristic={lemmas_first}]:
    "All S R x pSF pOut Next F1 F2 F3 F4 p1 p2 p3 #s #r #f1 #f2 #f3 #f4.
        SenderSettled(S, F1, R, pSF, x) @ #s
      & HTLCForwarded(F1, S, F2, pSF, p1, h(x)) @ #f1
      & HTLCForwarded(F2, F1, F3, p1, p2, h(x)) @ #f2
      & HTLCForwarded(F3, F2, F4, p2, p3, h(x)) @ #f3
      & HTLCForwarded(F4, F3, Next, p3, pOut, h(x)) @ #f4
      & RefundHop(p3, pOut, h(x)) @ #r
     ==> Ex P #c. LtkCompromised(P) @ #c"
```

Hop5_Skip_Requires_Compromise (∀) [lightning.spthy, l. 1254]. *The forwarder at position 5 cannot be bypassed without a compromise.*

```
lemma Hop5_Skip_Requires_Compromise [heuristic={lemmas_first}]:
    "All S R x pSF pOut Next F1 F2 F3 F4 F5 p1 p2 p3 p4 #s #r #f1 #f2 #f3 #f4 #f5.
        SenderSettled(S, F1, R, pSF, x) @ #s
      & HTLCForwarded(F1, S, F2, pSF, p1, h(x)) @ #f1
      & HTLCForwarded(F2, F1, F3, p1, p2, h(x)) @ #f2
      & HTLCForwarded(F3, F2, F4, p2, p3, h(x)) @ #f3
      & HTLCForwarded(F4, F3, F5, p3, p4, h(x)) @ #f4
      & HTLCForwarded(F5, F4, Next, p4, pOut, h(x)) @ #f5
      & RefundHop(p4, pOut, h(x)) @ #r
     ==> Ex P #c. LtkCompromised(P) @ #c"
```

Hop6_Skip_Requires_Compromise (∀) [lightning.spthy, l. 1267]. *The forwarder at position 6 cannot be bypassed without a compromise.*

```
lemma Hop6_Skip_Requires_Compromise [heuristic={lemmas_first}]:
    "All S R x pSF pOut Next F1 F2 F3 F4 F5 F6 p1 p2 p3 p4 p5 #s #r #f1 #f2 #f3 #f4 #f5 #f6.
        SenderSettled(S, F1, R, pSF, x) @ #s
      & HTLCForwarded(F1, S, F2, pSF, p1, h(x)) @ #f1
      & HTLCForwarded(F2, F1, F3, p1, p2, h(x)) @ #f2
      & HTLCForwarded(F3, F2, F4, p2, p3, h(x)) @ #f3
      & HTLCForwarded(F4, F3, F5, p3, p4, h(x)) @ #f4
      & HTLCForwarded(F5, F4, F6, p4, p5, h(x)) @ #f5
      & HTLCForwarded(F6, F5, Next, p5, pOut, h(x)) @ #f6
      & RefundHop(p5, pOut, h(x)) @ #r
     ==> Ex P #c. LtkCompromised(P) @ #c"
```

Helper lemmas of the hop-bypass family (all ∀, [reuse]):

```
lemma Forward_Unique_Per_Incoming [reuse, heuristic={route_first}]:
    "All M1 P1 N1 M2 P2 N2 pIn o1 o2 y #i #j.
        HTLCForwarded(M1, P1, N1, pIn, o1, y) @ #i
      & HTLCForwarded(M2, P2, N2, pIn, o2, y) @ #j
     ==> #i = #j"
```

```
lemma SettledHop_From_Forward [reuse, heuristic={route_first}]:
    "All pIn pOut y #i. SettledHop(pIn, pOut, y) @ #i
     ==> Ex M P N #j. HTLCForwarded(M, P, N, pIn, pOut, y) @ #j & #j < #i"
```

```
lemma No_Settle_And_Refund_Same_Hop [reuse, heuristic={route_first}]:
    "All pIn pOut y #i #j. SettledHop(pIn, pOut, y) @ #i & RefundHop(pIn, pOut, y) @ #j ==> F"

lemma No_Fulfill_On_Forwarded_Incoming [reuse, heuristic={route_first}]:
    "All R Q M P N p o y #i #j.
        HTLCFulfilled(R, Q, p, y) @ #i & HTLCForwarded(M, P, N, p, o, y) @ #j ==> F"

lemma Redeemed_Source [reuse]:
    "All p y #i. Redeemed(p, y) @ #i
     ==> (Ex o. SettledHop(p, o, y) @ #i)
       | (Ex R Q. HTLCFulfilled(R, Q, p, y) @ #i)"

lemma SenderSettled_Uses_Fulfilled [reuse]:
    "All S F1 R p x #i. SenderSettled(S, F1, R, p, x) @ #i ==> UsedFulfilled(p, x) @ #i"

lemma SettledHop_Uses_Outgoing [reuse]:
    "All pIn pOut y #i. SettledHop(pIn, pOut, y) @ #i
     ==> Ex x. UsedFulfilled(pOut, x) @ #i & h(x) = y"
```

Any_Hop_Skip_Requires_Compromise (∀) — `make general` (`-D=GENERAL -D=NO_IMC -D=NO_OTF -D=NO_RBT -D=NECESSITY`), 7 steps [`lightning.spthy`, l. 1803]. *A single-offer payment that settles while any forwarder's own hop is refunded requires a compromise. Proved in the model without the three liveness restrictions; removing restrictions only adds traces, so the statement also holds in the full model.*

```
lemma Any_Hop_Skip_Requires_Compromise [heuristic={lemmas_first}]:
  all-traces
  "∀ S F1 R pSF x Me Prev Next pIn pOut #s #f #r.
    ((((SenderSettled( S, F1, R, pSF, x ) @ #s) ∧
       (HTLCForwarded( Me, Prev, Next, pIn, pOut, h(x) ) @ #f)) ∧
      (RefundHop( pIn, pOut, h(x) ) @ #r)) ∧
     (∀ A B p #o. (HTLCOffered( A, B, p, h(x) ) @ #o) ⇒ (p = pSF))) ⇒
    (∃ P #c. LtkCompromised( P ) @ #c)"
```

Helper chain of the general theorem (all ∀, `-D=GENERAL`; listed by name only): `Forward_Parent`, `Hop_Redeem_Propagates`, `Sender_Channel_Redeemed`, `Root_First`, `Next_Is_Child`, `Route_Redeemed_Gen`, `Route_Redeemed_No_Compromise`. `make general` re-proves every lemma it reuses in the same mode (24/24 verified).

J. Auxiliary Models (lightning_aux.spthy, make aux, 21/21 verified)

PTLC (`-D=PTLC`): fixed route S → H1 → H2 → H3 → R; each channel has its own secret, derived from the next one by the hash chain `s_in = h(<s_out, t>)` with a per-hop fresh tweak t (a symbolic stand-in for the adaptor-signature points of [6]); locks are hashes of the secrets. HTLC baseline (`-D=HTLC`): the same route with one secret x for every channel.

-D=	Lemma	Kind	Steps
PTLC	`Honest_Payment_Completes`	∃	10
PTLC	`Colluders_Present_Payment_Completes`	∃	14
PTLC	`No_Wormhole_Around_Honest_Hop`	∀	25
PTLC	`Control_Refund_Of_H2_Outgoing`	∃	6
PTLC	`Control_Wormhole_If_H2_Compromised`	∃	18
HTLC	`Honest_Payment_Completes`	∃	10
HTLC	`Colluders_Present_Payment_Completes`	∃	8
HTLC	`Wormhole_Around_Honest_Hop`	∃	12
CLOCK	`HTLC_Needs_Open_Channel`	∀	36
CLOCK	`No_HTLC_After_Close`	∀	367
CLOCK	`Claim_Window_Exists`	∀	79
CLOCK	`Transitive_Preimage_Before_Upstream_Deadline`	∀	187
CLOCK	`Loss_Implies_Skipped_Claim`	∀	861
CLOCK	`Outcome_Exclusive`	∀	2
CLOCK	`Redeem_Reachable`	∃	7
CLOCK	`Refund_Reachable`	∃	7
CLOCK	`Honest_Flow_Possible`	∃	14
CLTV	`CLTV_Gap_Is_Positive`	∀	2
CLTV	`Claim_Window_Nonempty`	∀	2
CLTV	`Staggered_Path_Safe`	∀	2
TIMEOUT	`Early_Timeout_Race`	∃	6

No_Wormhole_Around_Honest_Hop (∀) [`lightning_aux.spthy`, l. 122]. *In the PTLC model (fixed route, hash-chain locks), the sender's channel is never redeemed while the outgoing channel of an uncompromised H2 is refunded, unless the sender is compromised.*

```
lemma No_Wormhole_Around_Honest_Hop:
  all-traces
  "not (
Ex S H1 H2 H3 R c0 c1 c2 c3 #p #a #b.
      Pay(S, H1, H2, H3, R, c0, c1, c2, c3) @ #p
    & Redeemed(c0) @ #a
    & Refunded(c2) @ #b
    & not (Ex #k. Compromised(H2) @ #k)
    & not (Ex #k. Compromised(S) @ #k))"
```

Wormhole_Around_Honest_Hop (∃) [`lightning_aux.spthy`, l. 252]. *With HTLCs, that wormhole around an uncompromised H2 is reachable.*

```
lemma Wormhole_Around_Honest_Hop:
  exists-trace
  "Ex S H1 H2 H3 R c0 c1 c2 c3 #p #a #b.
      Pay(S, H1, H2, H3, R, c0, c1, c2, c3) @ #p
    & Redeemed(c0) @ #a
    & Refunded(c2) @ #b
    & not (Ex #k. Compromised(H2) @ #k)
    & not (Ex #k. Compromised(S) @ #k)"
```

K. Channel Layer and Keys (group 0 of Table II, make verify)

Ltk_Known_Implies_Compromised (∀) [`lightning.spthy`, l. 502]. *The adversary knows a long-term key only if that key was compromised.*

```
lemma Ltk_Known_Implies_Compromised [reuse]:
    "All A ltk #i #j.
        KeyGen(A, ltk) @ #i
      & !KU(ltk) @ #j
     ==> Ex #k. LtkCompromised(A) @ #k"
```

∀: `state_update`, `delayed_funds`, `instant_funds`, `settlement_is_traceable`, `No_Punishment_Without_Cheating`, `Funds_Locked_Before_Update`, `Update_Requires_Negotiation`. ∃: `Protocol_execution`, `Cooperative_Close_Execution`. Together with Sections A–I and the additions above, this covers all 55 lemmas of `make verify`.